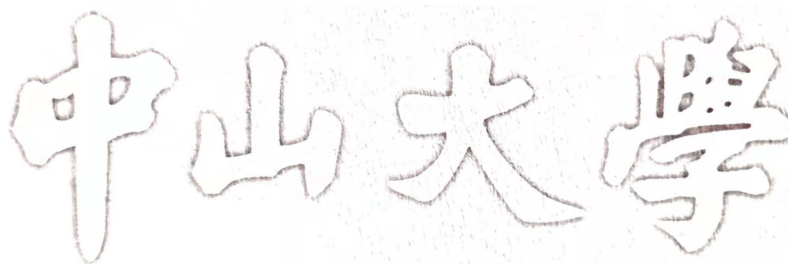


密级: 公开

编号: [学号]



# 工程 硕士专业学位论文

基于并行计算的金融大规模数据异常检测系统  
研究与实现

Research and Implementation of a  
Parallel-Computing-Based Anomaly Detection  
System for Large-Scale Financial Data

学位申请人: 黄裕文

导师姓名及职称: 黄聃 教授

专业、领域名称: 计算机技术

2026 年 5 月 4 日



中山大学硕士学位论文

基于并行计算的金融大规模数据异常检测系统研究与实现

Research and Implementation of a  
Parallel-Computing-Based Anomaly  
Detection System for Large-Scale  
Financial Data

专业: 计算机技术  
学位申请人: 黄裕文  
指导教师: 黄聃 教授

论文答辩委员会主席: \_\_\_\_\_  
成员: \_\_\_\_\_  
\_\_\_\_\_  
\_\_\_\_\_  
\_\_\_\_\_

二〇二六年五月



# 原创性及使用授权声明

## 论文原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究作出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律结果由本人承担。

学位论文作者签名：

日期： 年 月 日

## 学位论文使用授权声明

本人完全了解中山大学有关保留、使用学位论文的规定，即：学校有权保留学位论文并向国家主管部门或其指定机构送交论文的电子版和纸质版；有权将学位论文用于非赢利目的的少量复制并允许论文进入学校图书馆、院系资料室被查阅；有权将学位论文的内容编入有关数据库进行检索；可以采用复印、缩印或其他方法保存学位论文；可以为存在馆际合作关系的兄弟高校用户提供文献传递服务和交换服务。

保密论文保密期满后，适用本声明。

学位论文作者签名：

日期： 年 月 日

导师签名：

日期： 年 月 日



## 摘要

面向金融风控、反欺诈、反洗钱与交易监测，异常检测系统需要在大规模数据中识别少量高风险样本，同时满足吞吐率和时延要求。此类任务异常比例低、数据形态多样、业务时效性强，离线可用的方法若缺少系统化执行支撑，难以直接用于近实时链路。随着交易规模扩大，瓶颈从单一模型精度转向异常评分复杂、候选召回访问开销、CPU 与 GPU 数据交换以及多 GPU 扩展效率等端到端开销，局部算子或检索模块加速难以单独消除链路瓶颈。

针对上述问题，本文设计并实现 PyTOD-FlashANNS 协同异常检测系统 FlashTOD。PyTOD 是基于张量操作的异常评分框架，FlashANNS 是面向大规模外存近似最近邻检索的候选召回系统。FlashTOD 先通过 FlashANNS 在大规模样本中生成候选集合，再由 PyTOD 在候选范围内完成张量化异常评分，并输出异常分数和前  $k$  个结果 (Top- $k$ ) 风险对象。通过候选预算调节评分范围，FlashTOD 在检测效果和系统开销之间形成可控折中。该系统的重点是组织两类已有组件之间的数据流、接口和执行顺序，而不是提出新的异常检测理论模型。

工程实现中，单 GPU 路径主要压缩 CPU-GPU 往返、候选重组和阶段同步。前期利用 FAISS 的 GPU 检索实现 (FAISS-GPU) 较完善的近似最近邻检索 API，对候选召回接口、设备侧输入输出路径、候选 ID 与距离返回格式以及 PyTOD 候选评分衔接进行初步验证；在完整 FlashTOD 路径中，则以 FlashANNS 作为主要 ANN 候选召回后端，并结合其异步 I/O-计算流水线、页锁定内存 (pinned memory)、RAPIDS GPU 数据处理生态和 GPU 侧数据路径组织，减少主机侧反复介入。多 GPU 路径以降低跨卡归并和共享 I/O 竞争为目标，采用索引复制式 replicated 架构维持每张 GPU 的本地召回-评分闭环，使 CPU 主要承担请求调度、微批形成和最终小规模结果汇总；在需要跨卡归并时，借助 NVIDIA 集合通信库 (NCCL) 组织设备间通信，并结合拓扑感知的 I/O 通道 (I/O lane) 绑定与准入控制缓解共享路径冲突。

实验使用约 1M 真实或半真实金融数据验证有效性，并使用 10M、50M 与 100M 高保真合成数据测试吞吐、延迟和扩展趋势。1M 与 10M 分别表示约 100 万和 1000 万条样本；若仅按预处理后  $d$  维 float32 统一向量主体计算，二者约对应  $4d$  MB 和  $40d$  MB，实际落盘规模还受索引、标签和日志配置影响。在本文设定的数据规模、硬件平台和候选预算条件下，完整融合方案 PR-AUC 为 0.906，接近

PyTOD 全量评分的 0.932; QPS 由 2700 提升至 4280, 约提升 58.5%。单卡完整优化方案将 QPS 从 3180 提升至 4550, 并将 p99 延迟从 67.0 ms 降至 45.5 ms; replicated 路径下 1/2/4 GPU 的 QPS 分别为 4550、7780 和 13620, 4 GPU 约为 1 GPU 的 2.99 倍。结果表明, FlashTOD 在效果保持、单卡路径压缩和多卡吞吐扩展方面具有可验证的工程收益, 相关结论仍受实验条件约束。

关键词: 异常检测, 候选召回, FlashANNS, GPU 并行计算, 金融风控



## ABSTRACT

In financial risk control, fraud detection, anti-money laundering, and transaction monitoring, outlier detection systems need to identify a small number of high-risk samples from large-scale data while meeting throughput and latency requirements. Such tasks usually involve low anomaly ratios, heterogeneous data forms, and strict timeliness constraints, so an offline method cannot be directly used in a near-real-time pipeline without system-level execution support. As transaction volume grows, the bottleneck shifts from single-model accuracy to end-to-end costs, including expensive anomaly scoring, candidate retrieval overhead, CPU–GPU data movement, and limited multi-GPU scalability.

To address these issues, this thesis designs and implements *FlashTOD*, a collaborative outlier detection system built with PyTOD and FlashANNS. PyTOD is an anomaly scoring framework based on tensor operations, and FlashANNS is a candidate retrieval system for out-of-core approximate nearest neighbor search. FlashTOD first uses FlashANNS to generate a candidate set from large-scale samples, then applies PyTOD to perform tensorized anomaly scoring within the candidate set, and finally outputs anomaly scores and Top- $k$  risky objects. By controlling the candidate budget, FlashTOD provides a tunable trade-off between detection effectiveness and system cost. The focus is the organization of data flow, interfaces, and execution order between two existing components, rather than a new theoretical model for outlier detection.

In the engineering implementation, the single-GPU path compresses CPU–GPU round trips, candidate repacking, and stage synchronization. In the preliminary implementation, FAISS-GPU is used as a mature ANN API to validate the candidate retrieval interface, device-side input/output path, candidate ID and distance format, and the connection to PyTOD candidate scoring. In the complete FlashTOD path, FlashANNS is used as the main ANN candidate retrieval backend, together with its asynchronous I/O–compute pipeline, pinned memory, the RAPIDS GPU data-processing ecosystem, and GPU-side data-path organization, to reduce repeated host-side intervention. The multi-GPU path targets cross-GPU merging overhead and shared I/O contention. It uses an index-replicated architecture to keep a local retrieval–scoring loop on each GPU, limits the CPU to request scheduling, micro-batch formation, and final small-result collection,

employs NCCL for inter-device communication when cross-GPU merging is needed, and applies topology-aware I/O lane binding with admission control to mitigate shared-path conflicts.

Experiments use real or semi-real financial data at around 1M scale for effectiveness evaluation, and high-fidelity synthetic data at 10M, 50M, and 100M scales for throughput, latency, and scalability stress tests. Here 1M and 10M denote about one million and ten million samples, respectively; counting only the preprocessed  $d$ -dimensional float32 vector body, they correspond to about  $4d$  MB and  $40d$  MB of storage, while the actual on-disk size also depends on index, label, and log configurations. Under the evaluated data scales, hardware platform, and candidate budgets, the full fusion scheme achieves a PR-AUC of 0.906, close to the 0.932 of the PyTOD full-scoring baseline. QPS increases from 2700 to 4280, a 58.5% improvement. The full single-GPU optimization increases QPS from 3180 to 4550 and reduces p99 latency from 67.0 ms to 45.5 ms. Along the replicated path, the QPS values on 1, 2, and 4 GPUs are 4550, 7780, and 13620, with 4 GPUs reaching about  $2.99\times$  the 1-GPU throughput. These results show verifiable engineering gains in effectiveness preservation, single-GPU path compression, and multi-GPU throughput scaling, while the conclusions remain bounded by the evaluated conditions.

**Keywords:** Outlier Detection, Candidate Retrieval, FlashANNS, GPU Parallel Computing, Financial Risk Control

## 目 录

|                                           |      |
|-------------------------------------------|------|
| 摘 要.....                                  | I    |
| ABSTRACT .....                            | III  |
| 符号和缩略语说明.....                             | VIII |
| 插图清单.....                                 | IX   |
| 附表清单.....                                 | XI   |
| 第一章 绪论.....                               | 1    |
| 1.1 研究背景与意义.....                          | 1    |
| 1.2 本文研究内容.....                           | 3    |
| 1.3 研究目标与技术路线.....                        | 7    |
| 1.4 本文主要工作与技术贡献.....                      | 9    |
| 1.5 论文组织结构.....                           | 10   |
| 第二章 相关研究工作与关键技术基础.....                    | 11   |
| 2.1 异常检测方法研究现状.....                       | 11   |
| 2.2 TOD/PyTOD 的 GPU 张量化执行基础.....          | 13   |
| 2.3 FlashANNS 候选召回系统.....                 | 16   |
| 2.4 GPU 数据处理、FAISS-GPU 初步验证与候选召回后端支撑..... | 18   |
| 2.5 多 GPU 调度与在线推理启示.....                  | 21   |
| 2.6 本章小结.....                             | 22   |
| 第三章 FlashTOD 检测模型与数据准备.....               | 23   |
| 3.1 金融异常检测任务建模.....                       | 23   |
| 3.2 实验数据集选取与构建.....                       | 25   |
| 3.3 数据预处理与特征构造.....                       | 30   |
| 3.4 PyTOD 异常评分模块设计.....                   | 31   |
| 3.5 FlashANNS 候选召回模块设计.....               | 32   |
| 3.6 FlashTOD 执行流程.....                    | 34   |
| 3.7 基础实验与初步验证.....                        | 35   |
| 3.8 本章小结.....                             | 36   |

|                               |    |
|-------------------------------|----|
| 第四章 单卡场景下的系统设计与数据路径优化         | 38 |
| 4.1 单卡系统需求与总体架构               | 38 |
| 4.2 单卡场景下的瓶颈分析                | 39 |
| 4.3 GPU 主导的端到端执行链路设计          | 42 |
| 4.4 异步 I/O—计算流水线融合            | 43 |
| 4.5 GPU 数据处理与候选组织优化           | 44 |
| 4.6 候选召回后端的初步验证与完整路径接入        | 45 |
| 4.7 单卡执行链路的工程增强               | 45 |
| 4.8 单卡实验与性能分析                 | 46 |
| 4.9 本章小结                      | 48 |
| 第五章 多 GPU 并行扩展与归并优化           | 49 |
| 5.1 多 GPU 扩展需求分析              | 49 |
| 5.2 多 GPU 扩展策略设计原则            | 50 |
| 5.3 数据并行与索引复制的多 GPU 执行架构      | 51 |
| 5.4 CPU 控制面调度与负载均衡            | 52 |
| 5.5 跨卡归并场景下的 GPU 侧归并与 NCCL 通信 | 54 |
| 5.6 拓扑感知 I/O 通道绑定与准入控制        | 54 |
| 5.7 多 GPU 运行时支撑               | 55 |
| 5.8 多 GPU 扩展实验与结果分析           | 55 |
| 5.9 本章小结                      | 58 |
| 第六章 综合实验设计与结果分析               | 59 |
| 6.1 实验环境与软硬件配置                | 59 |
| 6.2 数据集与任务设置                  | 61 |
| 6.3 评价指标与比较原则                 | 62 |
| 6.4 核心结果展示                    | 65 |
| 6.5 结果讨论                      | 69 |
| 6.6 局限性分析                     | 70 |
| 6.7 本章小结                      | 71 |
| 第七章 全文总结与展望                   | 72 |
| 7.1 全文总结                      | 72 |
| 7.2 未来工作展望                    | 73 |

|                       |    |
|-----------------------|----|
| 参考文献.....             | 75 |
| 附录 A 实验脚本与日志归档说明..... | 78 |
| 附录 B 图表补充说明.....      | 81 |
| 附录 C 缩写、变量与配置对照.....  | 85 |
| 在学期间完成的相关学术成果.....    | 86 |
| 致 谢.....              | 87 |

## 符号和缩略语说明

|                  |                                                                  |
|------------------|------------------------------------------------------------------|
| ANN              | Approximate Nearest Neighbor, 近似最近邻                              |
| AUC              | Area Under Curve, 曲线下面积                                          |
| BTO              | Basic Tensor Operations, 基础张量算子                                  |
| CPU              | Central Processing Unit, 中央处理器                                   |
| DMA              | Direct Memory Access, 直接内存访问                                     |
| FO               | Functional Operations, 功能算子                                      |
| GDS              | GPUDirect Storage, GPU 直连存储访问机制                                  |
| GPU              | Graphics Processing Unit, 图形处理器                                  |
| H2D / D2H        | Host-to-Device / Device-to-Host, 主机到设备 / 设备到主机传输                 |
| I/O              | Input/Output, 输入/输出                                              |
| KNN              | K-Nearest Neighbors, $k$ 近邻                                      |
| NIC              | Network Interface Card, 网络接口卡                                    |
| NUMA             | Non-Uniform Memory Access, 非一致内存访问                               |
| PCIe             | Peripheral Component Interconnect Express, 外设组件高速互连              |
| PR-AUC           | Precision-Recall Area Under Curve, 精确率—召回率曲线下面积                  |
| QPS              | Queries Per Second, 每秒查询数                                        |
| RMM              | RAPIDS Memory Manager, RAPIDS 内存管理器                              |
| ROC-AUC          | Receiver Operating Characteristic Area Under Curve, 受试者工作特征曲线下面积 |
| SSD              | Solid-State Drive, 固态硬盘                                          |
| Top- $k$ overlap | 近似排序结果与基线排序结果的前 $k$ 重合度                                          |
| adaptive_ratio   | 多 GPU 动态分流比例策略                                                   |
| bounded batching | 在时延约束下受控执行的微批处理策略                                                |
| rerank           | 对粗排候选执行更高精度精排计算的步骤                                               |

## 插图清单

|       |                                                                      |    |
|-------|----------------------------------------------------------------------|----|
| 图 1-1 | 金融异常检测业务场景总览图（本文绘制） .....                                            | 2  |
| 图 1-2 | 数据规模增长与系统瓶颈迁移示意图（本文绘制） .....                                         | 2  |
| 图 1-3 | 金融大规模异常检测系统标准处理流程图（本文根据金融异常检测流程整理绘制） .....                           | 4  |
| 图 1-4 | FlashTOD 系统概念图（本文绘制） .....                                           | 5  |
| 图 2-1 | TOD 系统总览图（根据文献 <sup>[6]</sup> 整理） .....                              | 15 |
| 图 2-2 | 典型异常检测算法到张量算子的映射（根据文献 <sup>[6]</sup> 整理） .....                       | 15 |
| 图 2-3 | FlashANNS 异步 I/O—计算流水线原理（根据文献 <sup>[7]</sup> 整理） .....               | 16 |
| 图 2-4 | RAPIDS 在本文系统中的数据处理位置（本文绘制） .....                                     | 20 |
| 图 2-5 | FAISS-GPU 的 CPU/GPU 互操作路径（根据 FAISS-GPU 文档与本文系统接口整理） .....            | 20 |
| 图 3-1 | 金融异常检测任务建模图（本文绘制） .....                                              | 24 |
| 图 3-2 | PyTOD 异常评分模块流程图（本文绘制） .....                                          | 32 |
| 图 3-3 | FlashANNS 候选召回执行流程图（本文绘制） .....                                      | 33 |
| 图 3-4 | 模块间中间结果传递示意图（本文绘制） .....                                             | 35 |
| 图 3-5 | 不同数据集上的候选召回—候选评分初步有效性验证结果（实验数据绘制） .....                              | 36 |
| 图 4-1 | 单卡异常检测系统总体架构图（本文绘制） .....                                            | 39 |
| 图 4-2 | 原始链路与 GPU 主导链路对比图（本文绘制） .....                                        | 41 |
| 图 4-3 | FlashANNS 异步 I/O—计算流水线与本文执行链路融合（本文根据文献 <sup>[7]</sup> 和系统实现整理） ..... | 44 |
| 图 4-4 | 1M 数据规模下单卡各阶段耗时分解（实验数据绘制） .....                                      | 47 |
| 图 4-5 | 10M 数据规模下单卡各阶段耗时分解（实验数据绘制） .....                                     | 47 |
| 图 4-6 | 单卡执行链路递进式消融结果（实验数据绘制；该图为递进式消融，非完全独立的单因素消融） .....                     | 48 |
| 图 5-1 | 数据并行与索引复制多 GPU 架构（本文绘制） .....                                        | 52 |
| 图 5-2 | CPU 控制面调度与微批示意（本文绘制） .....                                           | 53 |
| 图 5-3 | 拓扑感知 I/O 通道示意（本文绘制） .....                                            | 55 |
| 图 5-4 | 多 GPU 扩展趋势与共享路径瓶颈分析（实验数据绘制） .....                                    | 56 |

---

|       |                                  |    |
|-------|----------------------------------|----|
| 图 5-5 | 平均延迟与 p99 延迟变化（实验数据绘制） .....     | 57 |
| 图 5-6 | I/O 通道配置对扩展性的影响（实验数据绘制） .....    | 57 |
| 图 6-1 | 实验平台拓扑示意图（本文根据实验平台配置绘制） .....    | 61 |
| 图 6-2 | 数据规模层级与实验类型关系图（本文绘制） .....       | 62 |
| 图 6-3 | 指标体系结构图（本文绘制） .....              | 64 |
| 图 6-4 | 不同基线方案下检测效果与吞吐率对比（实验数据绘制） .....  | 66 |
| 图 6-5 | FlashTOD 与基线系统吞吐对比（实验数据绘制） ..... | 67 |
| 图 6-6 | FlashTOD 效果与性能折中（实验数据绘制） .....   | 68 |
| 图 6-7 | 单卡优化与多 GPU 扩展收益对比（实验数据绘制） .....  | 68 |
| 图 6-8 | 不同规模下系统表现趋势（实验数据绘制） .....        | 68 |



附表清单

表 1-1 本文研究目标与技术路线对应关系..... 8

表 2-1 各类异常检测方法复杂度与系统适配性比较..... 13

表 2-2 候选召回系统比较..... 18

表 3-1 实验数据集总体信息..... 27

表 3-2 数据集属性与实验用途说明..... 28

表 3-3 异常评分指标与算子映射..... 32

表 3-4 候选召回模块主要参数..... 34

表 4-1 单卡主要瓶颈与对应优化项..... 41

表 5-1 FlashTOD 多 GPU 部署适用边界..... 51

表 5-2 多 GPU 扩展结果汇总..... 58

表 6-1 实验平台与软件环境配置..... 60

表 6-2 对比方案与基线设置..... 64

表 6-3 核心结果汇总..... 66

表 6-4 后续外部基线补充计划..... 71



## 第一章 绪论

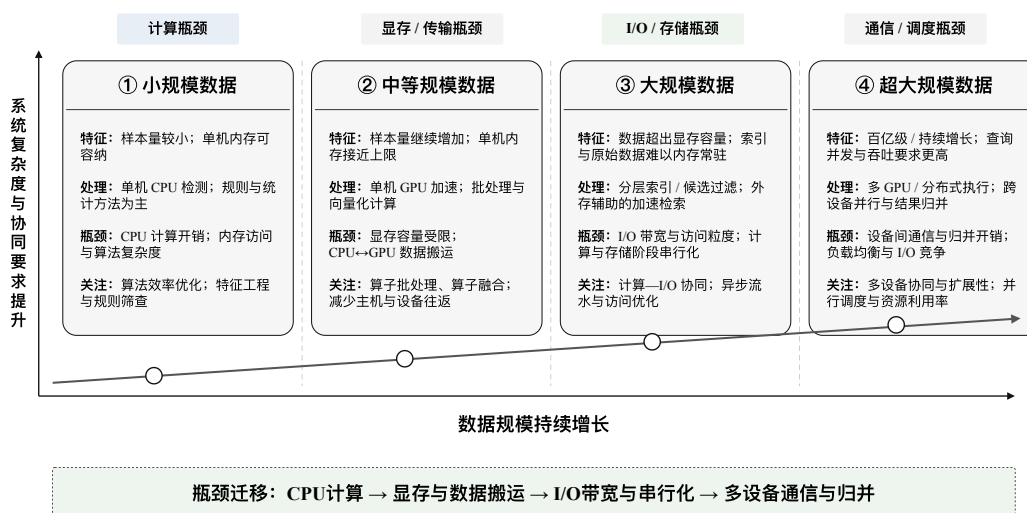
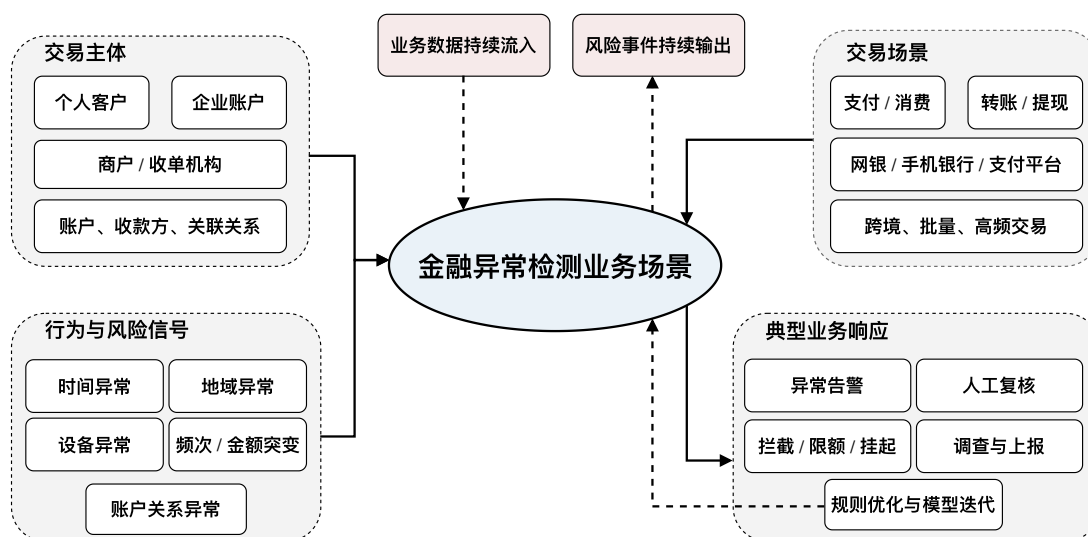
### 1.1 研究背景与意义

异常检测是金融风控、反欺诈、反洗钱、交易监测与平台治理等场景中的基础能力，其目标是在大量正常样本中识别出少量偏离总体分布的高风险对象。与传统监督学习任务相比，金融异常检测通常面临标签稀缺、异常样本极度不平衡、异常模式持续变化以及业务时效性要求高等现实约束。金融风险预测任务已不再局限于静态表格建模，而是越来越多地结合关系结构与时间演化信息来刻画欺诈交易、洗钱和逃税等多类风险问题<sup>[1]</sup>。因此，问题的关键不只在检测模型能否识别异常，还在于整条检测链路能否在真实业务环境中以可接受的吞吐率与时延稳定运行<sup>[2]</sup>。随着金融数据规模由百万级扩展至千万级乃至更高量级，异常检测所面对的矛盾已从单一模型计算逐步延伸到数据准备、候选检索、异常评分、设备间通信、日志追踪与在线扩展等多个环节的协同组织。

如图 1-1 所示，在金融业务场景中，异常检测通常服务于实时交易风控、支付反欺诈、账户行为审计、洗钱交易排查与交易路径审查等任务。这类任务同时受到两方面约束。其一，输入数据规模大、维度高且更新频繁，交易行为、账户状态、地理位置、设备指纹、操作轨迹与时间统计特征往往需要共同参与异常判定；其二，系统必须在有限时间内输出可执行的异常评分或告警结果，延迟过高会削弱业务决策时效，吞吐不足则容易在高峰期形成检测堆积<sup>[2]</sup>。也就是说，金融异常检测不仅要求检测结果可用，还要求系统在高并发条件下持续可运行。在高吞吐数据处理场景中，系统瓶颈往往并不只来自单个算子的计算代价，还会进一步体现在数据驻留位置、查询执行模型以及 CPU-GPU 协同组织方式上<sup>[3-4]</sup>。

传统基于中央处理器（Central Processing Unit, CPU）的异常检测系统在小规模数据环境下能够完成较为稳定的训练与推理，但当样本规模持续增大时，距离计算、排序、候选组织与统计聚合等操作带来的时间与空间开销会迅速放大<sup>[2]</sup>。如图 1-2 所示，瓶颈会随数据规模变化发生迁移：在早期，问题主要表现为算子计算慢；而在单算子加速之后，CPU 与图形处理器（Graphics Processing Unit, GPU）之间的数据传输、检索与评分之间的中间结果组织，以及多 GPU 条件下的任务调度与结果聚合，会逐渐成为新的限制因素。这意味着，仅对某个检测器或某类算子做局部加速，并不足以支撑金融场景下的大规模异常检测。

在这一背景下，近似最近邻（Approximate Nearest Neighbor, ANN）检索为异



常检测系统提供了更具工程可行性的组织方式。对于金融大规模异常检测任务，并不一定需要对全量样本直接执行高代价评分；更现实的路径是先借助高吞吐候选召回模块从海量数据中筛出规模可控的候选集合，再对候选集合执行更精细的异常评分。这种“候选召回—异常评分”的两阶段路线能够有效压缩评分范围，并为后续吞吐扩展预留空间。在大规模向量检索场景下，近似最近邻搜索问题并不仅限于索引结构设计，还包括硬件加速、磁盘-内存混合访问和数据路径优化等系统性问题<sup>[5]</sup>。但系统是否真正受益，取决于候选质量、候选组织代价、跨阶段数据搬运、输入/输出（Input/Output, I/O）与计算重叠程度，以及多 GPU 扩展时的归并方式，而不是简单地把两个模块首尾相接。

基于上述认识，本文从系统实现角度切入金融大规模异常检测问题，聚焦于 PyTOD 异常评分框架（下文简称 PyTOD）与 FlashANNS 候选召回系统（面向外存驻留或超显存向量索引访问路径的 GPU 驱动近似最近邻检索系统，下文简称 FlashANNS）的融合。PyTOD 作为沿 TOD 路线实现的 Python 异常检测框架，通过张量算子抽象承担异常评分模块<sup>[6]</sup>；FlashANNS 则借助异步 I/O—计算流水线提供高吞吐候选召回能力<sup>[7]</sup>。沿着这一组合继续向下看，单卡场景的关键在于能否让查询特征、候选结果与评分中间张量尽量驻留 GPU，减少主机侧反复介入；多卡场景的关键则在于能否让每张 GPU 完成本地闭环，并把 CPU 的职责收敛到控制面，避免重新回到数据平面。这也构成了本文的主要切入点。

从这个意义上说，围绕金融大规模异常检测系统展开研究，既有方法层面的价值，也有明确的工程意义。它将研究重心从单一检测器的性能比较推进到候选召回、异常评分和并行执行机制的协同设计，也为金融风控场景下构建高吞吐、低时延、可扩展的异常检测系统提供了更贴近实际部署的技术路线。

## 1.2 本文研究内容

### 1.2.1 本文研究对象与系统处理链路

在金融大规模异常检测场景中，待解决的问题已经超出单一检测模型的局部性能优化，转而落在候选检索、异常评分、数据路径组织、设备协同和结果追踪共同构成的系统链路上。面对连续到达、时延敏感和峰值流量明显的业务条件，系统既要保持对少量高风险样本的识别能力，又要在大规模数据条件下维持稳定吞吐与可接受延迟。基于这一判断，本文的研究没有停留在单一算法或单一检索组件的独立优化上，而是把研究对象落在面向金融大规模异常检测任务的系统链路上。

从处理流程上看，本文关注的系统链路如图 1-3 所示，可概括为：对原始金融交易记录、账户行为统计和上下文特征进行规范化与向量化表示；基于向量化表

示执行高吞吐候选召回，以缩小后续异常评分所需处理的样本范围；在候选集合上执行张量化异常评分，输出异常分数或排序结果；最后结合日志、统计指标与告警规则完成结果输出与追踪。后续各章围绕这条“向量化表示—候选召回—异常评分—结果输出与追踪”链路在大规模条件下如何稳定落地的问题，展开讨论和阐述。

在这一处理链路上，本文选择由 PyTOD 与 FlashANNS 协同构成的异常检测系统作为具体研究对象，并将其命名为 FlashTOD，其中 Flash 取自候选召回前端 FlashANNS，TOD 对应 PyTOD 所沿用的 TOD 路线。PyTOD 将异常检测过程抽象为可复用的张量算子，适合作为异常评分模块；FlashANNS 则通过异步 I/O—计算流水线提供高吞吐候选召回能力，适合作为候选召回前端，如图 1-4 所示。二者的角色分工相对清晰，但真正落到工程实现时，阶段边界上的开销并不会因为模块职责明确而自动消失。若查询特征先在 CPU 上构造，再传输至 GPU 检索，而候选结果又回到 CPU 重新组织后再送回 GPU 评分，那么瓶颈会很快转移到 CPU↔GPU 数据搬运与阶段同步上；进一步地，在多 GPU 场景中，若将多个 GPU 的局部结果重新集中到 CPU 归并，CPU 还可能再次成为系统瓶颈。

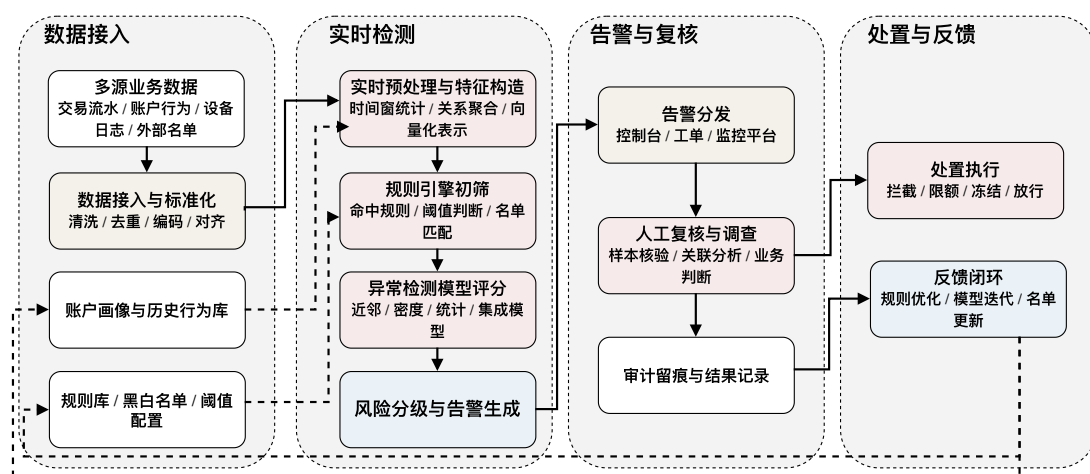


图 1-3 金融大规模异常检测系统标准处理流程图（本文根据金融异常检测流程整理绘制）

图 1-4 在本章中用于说明 FlashTOD 的基本概念和两阶段关系，系统整体架构、层次划分与执行边界将在第三章进一步展开。需要说明的是，图中 FAISS-GPU 表示前期 ANN 候选召回接口和设备侧检索路径的初步验证组件；完整 FlashTOD 路径中的主要 ANN 候选召回后端为 FlashANNS。

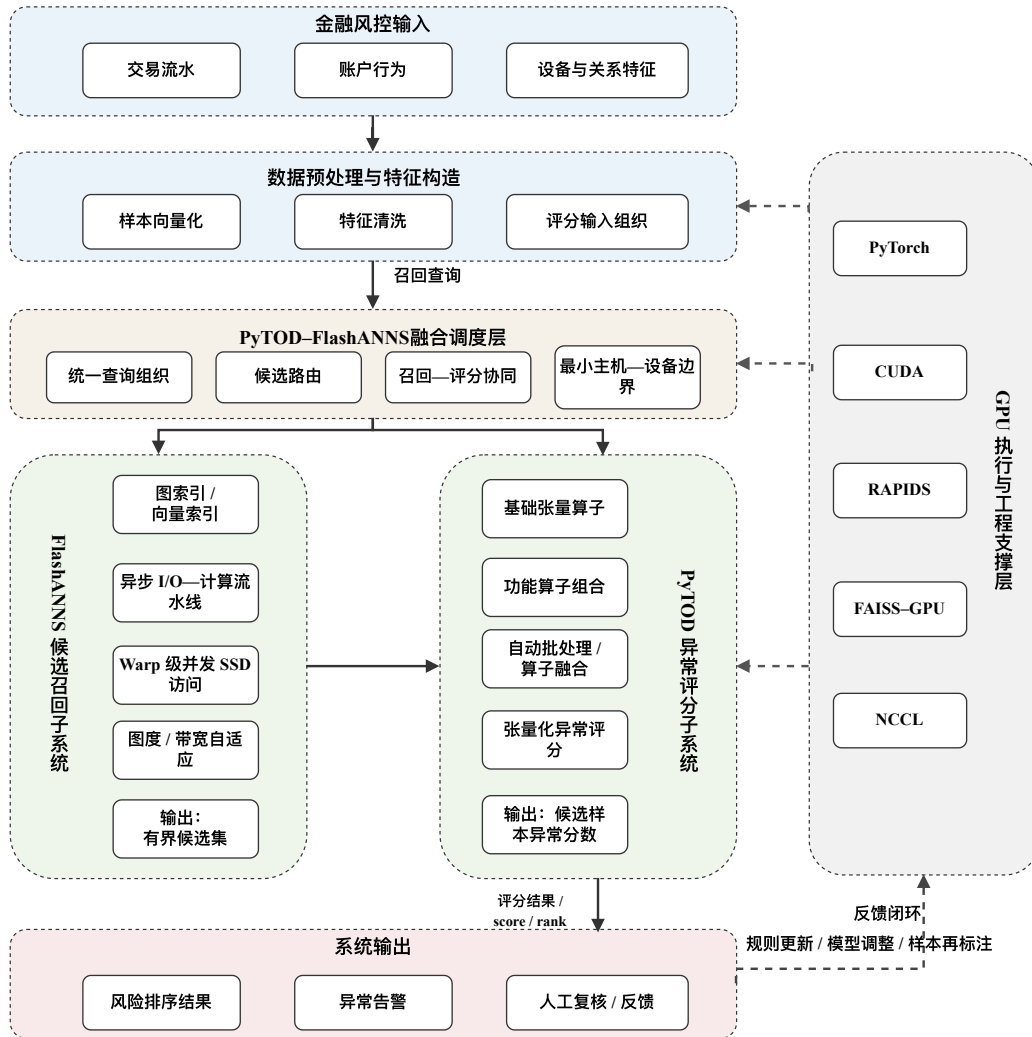


图 1-4 FlashTOD 系统概念图（本文绘制）

### 1.2.2 本文关注的核心系统问题

面向金融场景的大规模异常检测系统，其性能并不取决于某个模块的速度，而是主要由异常评分复杂度、候选召回开销、数据路径组织成本以及多 GPU 扩展代价共同决定。本文研究内容将进一步聚焦于这几个彼此关联的核心问题。

**异常评分阶段的高复杂度计算问题** 许多异常检测方法依赖样本之间的距离、近邻关系或统计偏离程度进行评分，尤其是基于近邻、密度或局部异常因子的检测方法，需要大量距离计算、排序与聚合操作<sup>[2]</sup>。当数据规模上升到百万级以上时，全量距离矩阵的显式构造会带来极高计算开销与显存压力。PyTOD 通过张量算子抽象提升了异常评分阶段的执行效率，但其优化重点在于异常评分过程的高效计算，而非决定哪些样本应进入后续评分阶段。在大规模异常检测系统中，评分复杂度问题必须放在候选召回已经收缩样本范围的前提下联合讨论。

**候选召回阶段的存储访问与吞吐瓶颈问题** 候选召回的作用在于为后续评分提供规模可控且质量可接受的候选集合。在大规模数据环境中，召回模块既要保证候选质量，又要控制访问代价；一旦索引规模超过显存容量并进入外存驻留（out-of-core）路径，I/O 与计算之间能否有效重叠，就会直接决定系统吞吐上限。也就是说，大规模候选召回的性能上限并不只取决于近邻搜索本身，还同时受数据组织方式、外存访问路径与硬件加速策略共同影响<sup>[5]</sup>。

**CPU-GPU 数据传输与阶段同步问题** 在工程系统中，阶段边界往往比单个算子更容易积累开销。若查询特征与候选结果频繁在 CPU 与 GPU 之间往返，那么即便候选召回和异常评分本身已经被加速，端到端吞吐仍会被主机到设备/设备到主机传输（Host-to-Device / Device-to-Host, H2D / D2H）、格式转换与阶段同步拖慢。单卡场景下，如何压缩主机侧参与范围、让更多中间结果停留在 GPU 上，是影响系统效率的关键问题。若中间结果无法在设备侧连续组织，PCIe 传输、对象转换和边界同步很容易迅速抵消局部模块已经获得的加速收益<sup>[3-4]</sup>。

**多 GPU 扩展中的结果聚合与共享 I/O 瓶颈问题** 多 GPU 扩展并不必然带来线性收益：一方面，索引复制、查询分发、设备间通信与结果归并都会引入额外开销；另一方面，多卡并发访问共享外设组件高速互连（Peripheral Component Interconnect Express, PCIe）、非一致内存访问（Non-Uniform Memory Access, NUMA）与固态硬盘（Solid-State Drive, SSD）路径时，I/O 竞争还可能放大尾延迟。因此，多 GPU 扩展并不是简单地增加 GPU 的数量，而是需要研究在何种组织方式下增加



GPU 的数量才真正有利于系统吞吐。在多 GPU 系统中，带宽不均衡和远程访问开销会直接影响扩展效率，因此 NUMA 拓扑与共享路径组织不应被视为次要工程细节[8]。

### 1.2.3 本文研究边界与主线

综合来看，本文的研究边界在于面向金融大规模异常检测场景，研究系统的实现与组织方式。全文围绕以前端候选召回压缩评分范围，以 PyTOD 承担张量化异常评分模块，以 GPU 主导执行链路压缩 CPU↔GPU 数据路径，并在此基础上进一步讨论多 GPU 条件下的数据并行、结果归并与共享 I/O 约束的主线，在后续章节中将分别讨论系统构建、单卡链路压缩和多卡扩展问题。

## 1.3 研究目标与技术路线

本文将研究目标落在一个可部署、可扩展的金融大规模异常检测系统上。在保证检测效果可接受的前提下，系统需要能够面向大规模金融数据稳定运行，并兼顾较高吞吐率、较低端到端时延以及多 GPU 条件下的扩展能力。与单纯比较异常检测算法效果不同，本文更关注候选召回、异常评分、数据路径组织和多 GPU 扩展之间的协同关系，即如何将异常检测任务组织为一条能够在真实工程环境中持续运行的系统链路。

围绕上述目标，本文的技术路线可概括为“统一建模—候选召回—张量化评分—数据路径优化—多 GPU 扩展”五个连续环节。首先，对金融交易记录、账户行为统计和图结构关系信息进行预处理与特征构造，将异构金融数据统一映射为固定维度向量表示，为后续候选召回和异常评分提供一致的输入接口。其次，以 FlashANNS 作为候选召回前端，在大规模样本中快速筛选与查询样本相关的候选集合，从而避免直接对全量样本执行高代价评分。再次，以 PyTOD 作为异常评分模块，在候选集合上完成距离型、局部密度型或统计型异常评分，并输出异常分数、风险排序结果及必要的日志信息。

在单卡性能优化层面，工作重点放在数据路径压缩上。在 FlashANNS 异步 I/O—计算流水线基础上构建 GPU 主导执行链路，使查询特征、候选结果与评分中间张量尽量保持在 GPU 侧，仅在必要边界通过页锁定内存（pinned memory）与主机交互；同时引入 RAPIDS 开源 GPU 数据科学与数据处理生态（下文简称 RAPIDS）<sup>[9-11]</sup>，对特征处理、中间结果组织和内存资源进行工程支撑。在候选召回后端演进上，本文前期利用 FAISS（Facebook AI Similarity Search）的 GPU 检索实现（下文简称 FAISS-GPU）<sup>[12-13]</sup> 较完善的 ANN API 完成候选召回接口、候选 ID

与距离返回、设备侧输入输出路径以及 PyTOD 候选评分衔接的初步验证；在后续完整 FlashTOD 路径中，则以 FlashANNS 作为主要 ANN 候选召回组件，以支持大规模外存驻留或超显存索引条件下的高吞吐候选召回。对应解决的是单卡场景下最容易被忽视、但对端到端性能影响最直接的链路开销问题。

在多 GPU 扩展层面，优先采用数据并行与索引复制策略，使每张 GPU 独立完成本地候选召回与异常评分闭环，而 CPU 主要承担控制面职责；当索引容量或部署条件使完整复制不再可行时，再采用设备侧归并与 NVIDIA 集合通信库（NVIDIA Collective Communications Library, NCCL）通信作为兜底<sup>[14]</sup>，并结合拓扑感知 I/O 通道（I/O lane）控制降低共享路径冲突，在控制实现复杂度的同时尽可能把扩展收益保留下来。

为保持全文结构清晰，本文将上述技术路线分别落实到后续章节，如 1-1：第三章完成金融异常检测任务建模、数据准备、PyTOD 异常评分模块和 FlashANNS 候选召回模块设计；第四章讨论单卡场景下的 GPU 主导执行链路、数据路径压缩和工程后端优化；第五章进一步讨论多 GPU 数据并行、索引复制、CPU 控制面调度、设备侧归并和拓扑感知 I/O 控制；第六章从检测效果、吞吐率、时延、资源利用率和扩展能力等角度对系统进行综合验证。

表 1-1 本文研究目标与技术路线对应关系

| 技术路线环节            | 核心任务                                                  | 对应章节 |
|-------------------|-------------------------------------------------------|------|
| 金融数据统一建模与向量化表示    | 将交易记录、账户行为和图节点关系整理为统一输入接口                             | 第三章  |
| FlashANNS 候选召回    | 在大规模样本中快速生成规模可控的候选集合                                  | 第三章  |
| PyTOD 张量化异常评分     | 在候选集合上完成距离、密度或统计型异常评分                                 | 第三章  |
| 单卡 GPU 主导执行链路     | 压缩 CPU 与 GPU 之间的数据搬运和阶段同步开销                           | 第四章  |
| 候选召回后端验证与完善       | 以前期 FAISS-GPU API 验证候选召回接口，完整路径以 FlashANNS 作为主要候选召回后端 | 第四章  |
| 多 GPU 数据并行与索引复制   | 使每张 GPU 独立完成本地候选召回与异常评分                               | 第五章  |
| 设备侧归并与拓扑感知 I/O 控制 | 降低跨卡归并和共享 I/O 路径带来的扩展损耗                               | 第五章  |
| 综合实验验证            | 从检测效果、吞吐率、时延和扩展性验证系统表现                                | 第六章  |

## 1.4 本文主要工作与技术贡献

围绕上述目标，本文完成了以下几方面工作：构建 FlashTOD，形成候选召回与异常评分协同执行的系统主干；在单卡场景下设计 GPU 主导执行链路并压缩数据路径；引入 RAPIDS 完成数据处理路径重构，并以前期 FAISS-GPU API 验证候选召回与评分衔接；在多 GPU 场景下给出数据并行与索引复制的主方案，并补充 GPU 侧归并与拓扑感知 I/O 控制作为兜底机制。

本文的主要工作与技术贡献主要体现在以下四个方面：

1. 面向金融大规模异常检测的 FlashTOD 协同系统设计。本文围绕金融风控场景下大规模异常检测的吞吐、时延和扩展需求，将 FlashANNs 候选召回与 PyTOD 张量化异常评分组织到统一执行链路中，形成候选召回—异常评分协同处理的系统框架。该设计的重点不在于提出新的异常检测模型，而在于明确两类已有组件在大规模金融数据处理链路中的角色边界、接口关系和执行顺序。
2. 候选召回与异常评分之间的数据接口组织。针对直接拼接候选召回模块和异常评分模块可能带来的候选重组、阶段同步和 CPU-GPU 数据往返问题，本文对查询向量、候选 ID、局部距离、排序信息和评分输入张量的传递方式进行了系统化组织，使候选召回结果能够更低成本地进入 PyTOD 评分阶段，从而减少模块边界上的额外开销。
3. 单 GPU 场景下的数据路径压缩与工程优化。本文围绕候选召回和异常评分之间的热路径，构建 GPU 主导的端到端执行链路，并结合页锁定内存、异步 stream、双缓冲和 RAPIDS 数据处理后端，对查询准备、候选组织和评分中间张量传递过程进行优化。本文前期利用 FAISS-GPU 较成熟的 GPU ANN API 对候选召回接口和 PyTOD 候选评分链路进行初步验证，随后在完整系统中以 FlashANNs 作为主要候选召回后端，并围绕其异步 I/O—计算流水线、设备侧候选组织和 PyTOD 评分接口进行数据路径压缩。这里的贡献是面向本文系统的工程组织与实现验证，而不是对 FAISS-GPU、FlashANNs 或页锁定内存等通用技术本身的重新提出。
4. 多 GPU 场景下的部署策略与扩展性验证。本文以数据并行和索引复制 (replicated) 架构作为吞吐优先场景下的主方案，使每张 GPU 尽可能独立完成本地候选召回与异常评分闭环；在索引容量或部署条件受限时，进一步讨论设备侧归并、NCCL 通信和拓扑感知 I/O 控制作为兜底路径。该部分贡献强调多 GPU 部署策略在本文系统中的适配、实现和实验验证，而不是重新定义多 GPU 扩展机制。

## 1.5 论文组织结构

全文按照“问题提出—相关研究—系统构建—单卡优化—多 GPU 扩展—实验验证—总结展望”的逻辑展开。第一章界定研究背景、研究内容、技术路线与全文结构，为后续章节建立问题范围；第二章梳理异常检测、候选召回、GPU 数据处理与多 GPU 扩展相关研究，为方案设计提供技术背景；第三章给出 FlashTOD 的检测模型、数据准备方式以及系统基础执行流程；第四章聚焦单卡场景，讨论 GPU 主导执行链路以及数据路径压缩方法；第五章进一步讨论多 GPU 扩展方案、控制面与数据面分工以及跨卡归并和 I/O 约束；第六章从检测效果、吞吐、时延与扩展性等维度验证系统表现，并据此分析适用边界；第七章对全文工作进行总结，并给出后续研究方向。

## 第二章 相关研究工作与关键技术基础

本章不按地域简单划分国内外研究现状，而是按照本文系统链路涉及的关键技术环节展开：异常检测方法、GPU 张量化异常评分、ANN 候选召回、GPU 数据处理路径和多 GPU 扩展。各节在介绍代表性工作的同时，重点归纳其系统边界和与本文工作的关系，从而为后续 FlashTOD 的系统设计提供技术背景和问题来源。

### 2.1 异常检测方法研究现状

异常检测旨在从大量正常样本中识别出少数偏离总体分布的对象；其研究长期以来围绕距离型、密度型、统计型、集成型以及深度学习型方法展开<sup>[2,6,15-16]</sup>。在金融场景下，异常不仅可能表现为单笔交易的数值偏离，也可体现在账户行为、设备特征、时序模式或多实体交互关系的结构性异常中。相应地，金融欺诈检测综述和图异常检测综述都表明，方法选择不能只考察离线识别能力，还必须兼顾可解释性、推理代价，以及在高吞吐、低时延条件下的可执行性<sup>[17-18]</sup>。面向金融风险的异常建模正在从静态表格描述逐步转向结合关系结构与时间演化的信息建模，而图异常检测研究也进一步说明了复杂图场景下异常定义与方法边界的多样性<sup>[1,19]</sup>。基于这一判断，本文在梳理这些方法时，更关注它们在统一 GPU 评分模块中的可组织性、批处理可行性和系统代价，以及是否便于与候选样本召回（下文简称候选召回）链路衔接。代表性评分方法可概括为基于角度、近邻密度、统计分布、线性投影和深度表征的多类路线，表 2-1 进一步比较了各类方法在复杂度、可解释性以及大规模金融场景适配性上的差异。

距离型方法是金融异常检测中最常见的一类路线，其核心思想是通过样本与近邻之间的距离、局部连通结构或几何关系来刻画异常程度<sup>[2,20]</sup>。例如，基于  $k$  近邻的异常检测通常将样本与其第  $k$  个近邻的距离、平均近邻距离或邻域集合结构作为风险评分依据；基于角度、可达距离或局部密度的代表性方法则进一步刻画样本与邻域之间的相对偏离关系。对于本研究而言，这类方法的重要性主要体现在两方面：一方面，其业务语义相对直观，便于与金融风控中偏离正常行为模式的概念相对应；另一方面，其评分过程天然围绕距离、排序和局部聚合展开，因而具备被张量化组织的潜力。然而，这类方法也恰恰最容易暴露大规模系统中的复杂度问题。TOD 指 GPU-accelerated Outlier Detection via Tensor Operations，即基于张量操作的 GPU 加速异常检测框架或思想；其对经典异常检测算法的梳理表明，

多种近邻驱动 (proximity-based) 方法在朴素实现下至少具有二次时间或空间复杂度, 这直接限制了它们在近实时金融场景中的可部署性<sup>[6]</sup>。

从显存占用看,  $k$ NN、LOF 和 ABOD 等方法真正容易触发 GPU 超显存的部分, 通常并不是输入矩阵  $X \in \mathbb{R}^{n \times d}$  本身, 而是全量样本距离计算过程中的中间张量。若直接构造样本两两距离矩阵, 最低也需要  $O(n^2)$  级别的距离存储; 若在朴素实现中显式物化  $n \times n \times d$  的成对差分张量, 则显存需求还会随特征维度  $d$  继续放大。因此, 即使输入向量主体只有数十 MiB 到数 GiB, 全量距离计算也可能迅速达到数百 GiB、TiB 甚至 PiB 级别。第三章结合本文当前实验数据规模给出了对应的显存估算, 这也成为后续选择“先候选召回, 再精细评分”两阶段路线的重要背景。在金融欺诈检测场景中, 类不平衡、噪声干扰和稀疏联系等问题会进一步放大基于关系结构建模的难度, 这也说明异常检测不能只从单一分类精度角度理解<sup>[21-22]</sup>。

与距离型方法相比, 密度型和统计型方法更强调样本在局部分布或全局分布中的稀有程度。密度型方法通常通过比较样本局部密度与邻域密度之间的差异来识别异常; 统计型方法则通过特征分布、边缘概率或直方图结构来评估异常程度<sup>[6,23]</sup>。若仅从公式复杂度来看, 统计型方法往往显得更轻量, 密度型方法则介于距离型与统计型之间。然而, 一旦将问题置于系统实现层面, 这种区分便显得不够充分: 密度型方法仍然需要处理近邻结构, 而统计型方法对数据预处理、分布表达和批量组织方式十分敏感, 其大规模效率同样受到数据路径与资源组织方式的强烈影响。因此, 这两类方法虽可为评分设计提供参考, 却难以单独回答“如何让异常检测稳定运行在大规模金融链路上”这一系统性问题。

集成型方法与深度学习型方法代表了另外两种常见思路: 前者通过组合多个检测器或多个子空间模型来提高稳健性, 后者则利用深层表征能力处理更复杂的数据分布和非线性模式<sup>[2,6]</sup>。这些方法具有其价值, 但与本文所面对的系统目标并不完全契合。集成方法往往会进一步抬升推理成本和链路复杂度; 深度模型虽然表达能力更强, 但通常伴随着更高的在线部署成本、更复杂的训练依赖以及较弱的业务可解释性。面向金融风险预测的深度图模型虽然具备更强表示能力, 但同时也面临数据构建、计算效率和可解释性等方面的现实约束<sup>[1]</sup>。就本文的系统目标而言, 更合适的仍是一类能够融入统一 GPU 执行路径、并与候选召回自然衔接的评分模块。因此, 后续研究聚焦于“候选召回 + 张量化异常评分”的两阶段系统路线, 并没有继续探索更复杂的端到端模型, 这一取舍的核心在于明确研究边界和工程目标。

表 2-1 各类异常检测方法复杂度与系统适配性比较

| 方法类别  | 代表方法                                             | 复杂度与系统特征                        | 可解释性 | 大规模金融场景适配性                     |
|-------|--------------------------------------------------|---------------------------------|------|--------------------------------|
| 距离型   | kNN、ABOD、COF                                     | 依赖近邻搜索与距离计算，朴素实现下时间和空间开销较高      | 强    | 适合作为评分模块，但通常需要候选召回和 GPU 加速共同支撑 |
| 密度型   | LOF、LOCI                                         | 需要局部密度估计与邻域聚合，复杂度随近邻结构迅速上升      | 较强   | 适合描述相对异常，但系统效率仍受近邻组织和数据路径制约    |
| 统计型   | HBOS、COPOD、ECOD                                  | 依赖边缘分布、分位统计或直方图结构，单次评分较轻量       | 中等   | 适合作为基线或辅助评分模块，但对复杂金融异常模式覆盖有限   |
| 集成型   | 特征装袋 (Feature bagging)、检测器集成 (Detector ensemble) | 通过组合多个检测器提升稳健性，但推理成本和系统复杂度进一步增大 | 中等   | 适合离线分析或稳健性增强，不利于高吞吐近实时链路       |
| 深度学习型 | Autoencoder、GNN、时序模型                             | 表达能力强，但训练与部署依赖重，在线解释和维护成本高      | 较弱   | 适合复杂模式建模，但需要更高工程投入，不是本文主路线     |

## 2.2 TOD/PyTOD 的 GPU 张量化执行基础

为解决传统异常检测算法难以直接迁移到 GPU 并高效扩展的问题，TOD 提出了一种基于张量操作的 GPU 加速异常检测系统。TOD 将异常检测过程分解为一组基础张量算子与功能算子，并在深度学习框架 PyTorch 等现代基础设施上统一执行，而不再为每种检测器分别编写定制化 GPU 实现<sup>[6]</sup>。TOD 的系统架构（如图 2-1 所示）主要包含三个层次：底层是可由 GPU 高效执行的基础张量算子 (Basic Tensor Operations, BTOs)；中间层是由这些基础算子组合而成的功能算子 (Functional Operations, FOs)；上层则对应具体的异常检测算法及其组合。图 2-2 进一步展示了典型异常检测方法到张量算子的映射关系，例如 kNN、LOF、COPOD 等算法可被重写为 `cdist`、`topk`、聚合与统计运算的组合<sup>[6]</sup>。对于本研究而言，这一抽象的核心价值在于它将评分过程重组为一条标准化、可批处理、且易于并行组织的执行链路。

TOD 进一步给出的可证明量化 (provable quantization)、自动批处理 (automatic batching) 以及多 GPU 支持 (multi-GPU support)<sup>[6]</sup>，对应了本文关心的三个问题：

算子开销能否被压缩，大规模评分能否被拆分为可流水的小批次，以及评分任务能否在多个设备之间继续扩展。TOD 的相关实验表明，在异常检测任务中，GPU 时间往往占总运行时间的绝大部分，而通过算子融合与批处理优化，可以减少大规模中间矩阵的物化和搬运开销，从而提升整体效率<sup>[6]</sup>。这些机制的重要性在于它们提供了一个更实际的判断：在大规模金融数据场景中，决定评分模块能否落地的，关键在于它是否能够被张量化、批处理化并继续向多 GPU 延展。

PyTOD 可视为 TOD 的对应开源实现与工程化延展，其价值在于将张量化异常检测从论文级概念推进为可复用的软件能力。对本文而言，其价值更在于为基于近邻、密度和统计分布的多类评分方法提供统一评分接口，使得这些方法能够在同一 GPU 评分模块上被组织和比较；同时，它将 TOD 所强调的张量算子抽象与 GPU 批处理思路落实为可调用的软件能力，为后续将异常评分嵌入候选召回链路提供了方法学基础。另一方面，本文将 PyTOD 更稳妥地定位为“评分模块”，而非“完整检测系统”，因为其重点在于异常评分的 GPU 化，而不直接覆盖大规模外存候选召回、GPU-SSD 数据路径以及多设备在线调度等更高层系统问题。

在 TOD/PyTOD 之前，已有工作尝试针对特定异常检测环节做 GPU 加速，例如将基于距离的离群检测（distance-based outlier detection）直接映射到 GPU，或将离群检测中的特征选择步骤改写为 GPU 并行实现<sup>[24-25]</sup>。这类研究说明 GPU 对异常检测具有实际加速价值，但大多仍停留在单一检测器或局部算子层面。相比之下，TOD/PyTOD 所代表的路线更具有普适性，它并不围绕某一个检测器做高度特化优化，而是围绕异常评分算子的张量化抽象、批处理执行和多 GPU 扩展提供统一支撑。也正因为如此，后续选择 PyTOD 作为异常评分模块。

因此，本文将 PyTOD 定位为评分模块：它以统一 GPU 执行语义承接近邻、密度、统计分布和线性投影等多类评分方法，自动批处理与多 GPU 支持也有助于缓解大规模评分中的显存压力。候选召回、外存索引和端到端调度并不属于它的覆盖范围，这部分由 FlashANNs 与后续系统设计补足。



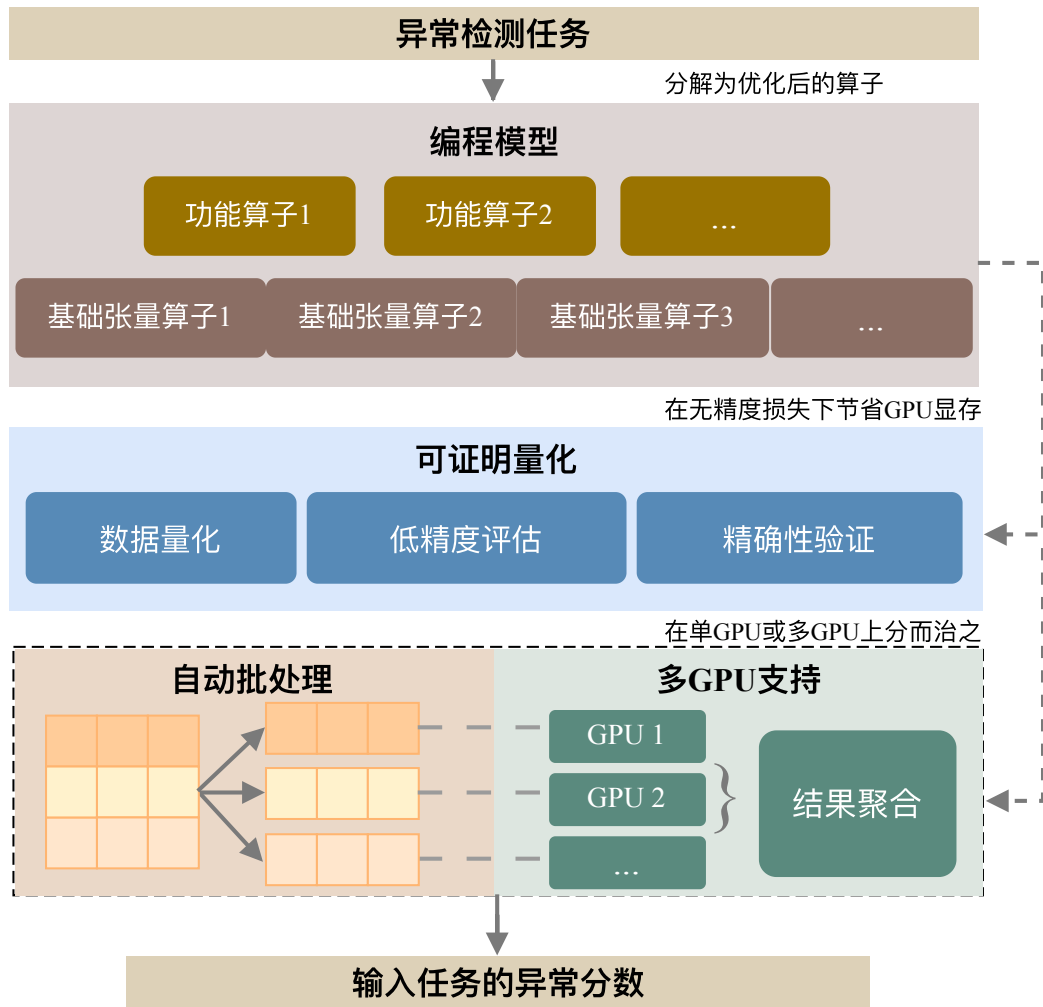


图 2-1 TOD 系统总览图（根据文献<sup>[6]</sup> 整理）

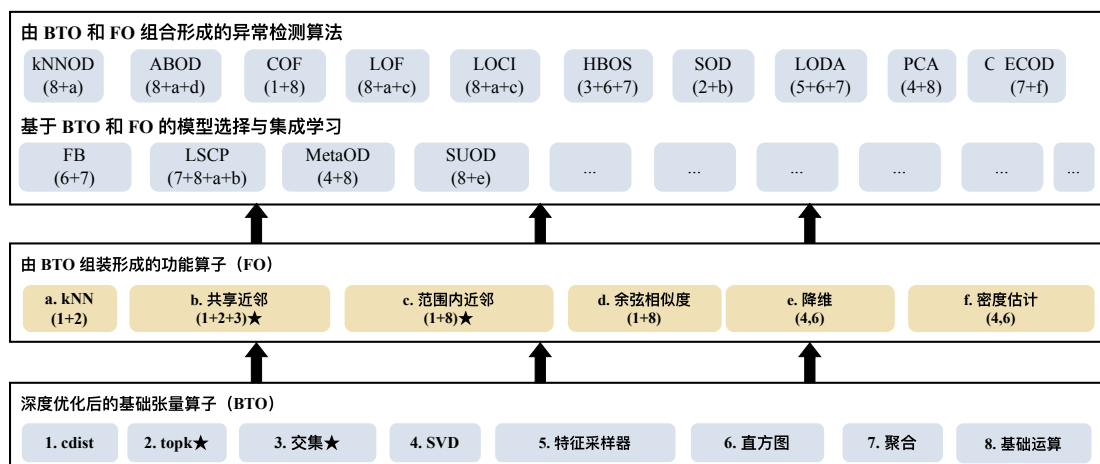


图 2-2 典型异常检测算法到张量算子的映射（根据文献<sup>[6]</sup> 整理）

## 2.3 FlashANNS 候选召回系统

在大规模异常检测系统中，若直接对全量样本执行精细异常评分，系统往往会因近邻搜索、候选组织和内存占用过大而难以满足吞吐要求。因此，引入高吞吐候选召回模块，先筛选潜在相关样本、再执行精细评分，是构建近实时检测系统的常见路线。FlashANNS 候选召回系统（面向外存驻留或超显存向量索引访问路径的 GPU 驱动近似最近邻检索系统）正是面向这一问题的代表性工作，它着力解决了两阶段链路中的候选召回前端。在 FlashTOD 中，FlashANNS 的作用是在大规模向量索引中快速返回规模受控的候选样本集合，为后续 PyTOD 异常评分压缩输入范围；它并不承担异常评分，也不直接输出异常分数。其关注重点不仅是近邻查找本身，还包括在外存驻留（out-of-core）路径下如何让近邻检索中的 I/O 与计算协同推进，从而缓解存储-计算瓶颈<sup>[7]</sup>。在大规模向量检索场景下，关键问题已从单纯索引结构扩展到硬件加速、磁盘-内存混合访问与数据访问优化等系统层面<sup>[5]</sup>。FlashANNS 的基本思想（如图 2-3 所示）可概括为异步 I/O—计算流水线：在保持 GPU 计算持续进行的同时，将外存访问与候选扩展过程重叠执行，以减少传统 ANN 图搜索中先读后算的严格串行开销。

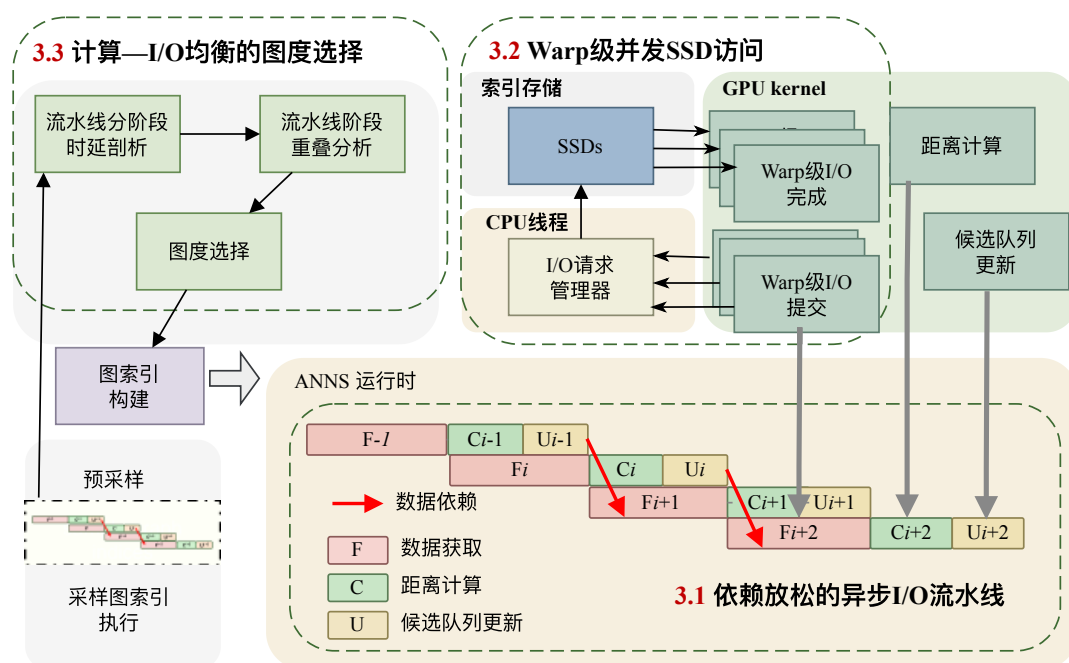


图 2-3 FlashANNS 异步 I/O—计算流水线原理（根据文献<sup>[7]</sup>整理）

FlashANNS 针对外存驻留 ANN 场景的价值主要体现在三个层面：它通过依赖放松的异步流水线，使部分计算不必等待全部 I/O 完成后再启动，从而提高 GPU 的使用率；它利用 warp 级并发和细粒度任务组织提升设备侧对候选扩展与结果筛

选的执行效率；更重要的是，它将系统瓶颈从单纯的计算速度扩展为存储路径与计算资源的协同平衡，强调 ANN 检索在大规模场景下是一个 I/O—计算联合优化问题，而非单纯算子优化问题<sup>[7]</sup>。这也决定了 FlashANNS 在本文中的角色边界：它承担候选召回前端，为后续更昂贵的异常评分缩小搜索空间，而不直接负责异常分数的生成。

在本文系统中，FlashANNS 被定位为候选召回前端，而不是异常评分模块。其输入来自前序预处理得到的统一向量表示，输出则是规模受控的候选集合，包括候选样本 ID、局部距离、局部排序信息或后续评分所需的数据引用。随后，PyTOD 在该候选集合上执行距离型、局部密度型或统计型异常评分，并生成异常分数与风险排序结果。由此形成的“候选召回—异常评分”两阶段结构，将大规模样本上的全量高代价评分转化为候选集范围内的精细评分，从而在检测效果与系统开销之间建立可调节的折中关系。

需要强调的是，FlashANNS 虽然能够降低后续评分阶段面对的搜索空间，但它并不直接输出异常分数，也不会自动解决候选结果到评分输入之间的接口组织问题。若候选 ID、局部距离和候选向量仍需回到 CPU 侧重新打包，再传入 GPU 端评分模块，则候选召回阶段获得的吞吐收益可能会被阶段边界上的数据搬运和同步开销抵消。因此，FlashANNS 在本文中的作用并非独立完成异常检测，其核心职责是为后续 PyTOD 张量化评分提供高吞吐、规模可控且尽量设备侧友好的候选输入。这一角色边界也为后续引入 RAPIDS、FAISS-GPU 前期接口验证以及 GPU 主导执行链路提供了技术动机。

从候选召回系统的研究脉络来看，传统 ANN 组件、GPU 内存内 ANN 检索和外存驻留 ANN 系统各自关注的瓶颈并不相同。就本文涉及的代表性候选召回路线而言，常见方案包括 HNSW、IVF/IVF-PQ，以及面向外存近似检索的 SPANN、DiskANN 和 FusionANNS 等。前者更关注索引结构与搜索精度，典型代表包括 HNSW 和基于 GPU 的 FAISS 路线；后者强调设备内向量搜索效率，而外存驻留系统则必须额外解决 SSD 访问、对齐 I/O、异步完成信号和大规模索引布局等问题<sup>[12,26]</sup>。类似 DiskANN 的单节点磁盘驻留 ANN 路线已经表明，SSD 与近似检索结合可以有效支撑十亿级向量搜索；本文之所以进一步关注 FlashANNS，则是因为这里面对的核心矛盾并非单卡内存足够时如何做到最快，而是在索引规模增大、候选池变大、设备数增加时，如何将外存访问与 GPU 计算更紧密地组织到同一条吞吐导向路径中<sup>[27]</sup>。表 2-2 将传统 ANN 组件、外存驻留 ANN 系统与 FlashANNS 的主要特征进行了对比，重点突出它们在 I/O—计算重叠、大规模外存适配性以及与本文系统关系上的差异。

FlashANNS 可以减轻评分阶段面对的全量搜索压力，但并不直接给出异常分数，也不会自动处理后续评分过程中的 GPU 常驻执行、张量组织、候选到评分的传递接口等问题。基于这一边界，本文将 FlashANNS 与 PyTOD 组合使用：前者负责高吞吐候选召回，后者负责张量化异常评分。这种明确的角色分工，使本文的两阶段系统既继承了 GPU 异常评分的计算优势，也继承了大规模 ANN 检索在外存条件下的吞吐优势。

表 2-2 候选召回系统比较

| 类别          | 代表方案                       | 主要存储假设                | I/O-计算重叠 | 主要特点                              |
|-------------|----------------------------|-----------------------|----------|-----------------------------------|
| 传统 ANN 组件   | FAISS、HNSW、IVF/IVF-PQ 等    | 以内存或显存内索引为主           | 通常不重点考虑  | 延迟低、工程成熟，但受内存/显存容量约束              |
| 外存驻留 ANN 系统 | SPANN、DiskANN、FusionANNS 等 | 面向 SSD/外存驻留索引         | 部分考虑     | 缓解内存容量限制，但仍可能存在 CPU 供数与 I/O 串行化问题 |
| FlashANNS   | FlashANNS                  | 面向 GPU + SSD 的外存驻留图索引 | 明确支持     | 通过异步 I/O-计算流水线实现高吞吐候选召回           |

## 2.4 GPU 数据处理、FAISS-GPU 初步验证与候选召回后端支撑

从算法框架看，FlashTOD 已具备候选召回—异常评分链路；但工程实现还需要进一步处理金融表格数据预处理、特征整理、中间结果表达、向量检索接口组织和 GPU 内存资源管理等问题。系统性能收益高度依赖数据驻留策略、查询处理模型和 CPU-GPU 协同组织方式，而不仅仅取决于单个算子的峰值速度<sup>[3-4]</sup>。本文进一步引入 RAPIDS，并利用 FAISS-GPU 进行前期接口验证，目的在于把这些高频且高开销的环节纳入统一 GPU 数据路径<sup>[9-12]</sup>。其中，RAPIDS GPU 数据处理生态主要用于 GPU 侧表格数据处理、候选输入组织、部分特征处理流程和内存资源管理支撑；FAISS-GPU 即 FAISS 的 GPU 检索实现，主要承担前期候选召回接口、设备侧输入输出路径和资源管理方式的工程验证职责。RAPIDS 在本文系统中的作用（如图 2-4所示）主要体现在“数据处理路径增强”上；FAISS-GPU（如图 2-5所示）则更多承担“设备侧向量检索与互操作能力验证”的职责。

RAPIDS 的核心价值在于为 GPU 上的表格处理和数据科学 workflows 提供统一支

持。**cuDF** 作为其重要组成部分，可以视为面向 GPU 的 **DataFrame** 引擎，适合处理金融场景中大量以表格形式存在的交易记录、账户属性、设备特征与时间统计结果。对于本研究而言，**RAPIDS** 并不承担异常检测本身，而主要承担候选召回前后的数据组织任务，包括字段清洗、特征拼接、归一化、窗口统计、中间结果重排和批量结果合并等。若这些操作长期停留在 CPU 侧，即便候选召回和异常评分本身已经 GPU 化，系统仍会因为频繁的 CPU↔GPU 往返而失去吞吐优势。因此，**RAPIDS** 在本文中的意义，体现在它将原本零散分布在链路边缘的数据处理步骤收拢到 GPU 路径中，为后续单卡 GPU 主导执行链路奠定基础。这一思路本质上也是通过统一设备侧数据组织来减少主机参与，从而降低链路边界上的额外代价<sup>[3]</sup>。

与此同时，**RAPIDS** 并不意味着整条链路就天然保持在 GPU 上，在兼容模式或未覆盖操作出现时，部分数据处理流程可能退回 CPU 执行；一旦这种回退发生，系统就会重新引入额外的数据搬运与同步等待。因此，本文更愿意将 **RAPIDS** 看作构建统一 GPU 数据路径的支撑件，只有在算子覆盖范围、运行路径和边界交换都被明确约束时，这种支撑作用才真正成立。第四章在讨论数据路径优化时，会将 GPU 保留路径与可能发生的 CPU fallback 一并作为工程边界加以说明。

引入 **FAISS-GPU** 的价值则体现在设备侧向量检索能力和 GPU 资源组织能力两个方面。作为成熟的 GPU ANN API，**FAISS-GPU** 支持在 GPU 上完成向量索引构建、近邻检索和部分结果筛选，其 **StandardGpuResources** 通过预分配临时工作区内存 (scratch memory) 与基于统一计算设备架构 (Compute Unified Device Architecture, CUDA) 的执行流 (stream) 组织方式，能够减少频繁 **cudaMalloc/cudaFree** 带来的同步与抖动<sup>[12-13]</sup>。此外，**FAISS-GPU** 还明确区分主机指针 (host pointer) 输入和设备指针 (device pointer) 输入两类路径：若输入已经位于同一 GPU 上，则可以避免额外拷贝；若输入位于主机端或位于不同设备上，则会引入相应的数据搬运开销。若数据分布方式和查询处理模型设计不当，PCIe 传输和设备内存利用不足会迅速抵消硬件加速收益<sup>[4]</sup>。对本文来说，这种区分把哪些开销来自算法和哪些开销来自接口组织清楚地暴露出来。因此，**FAISS-GPU** 在本文中的作用主要是提供候选召回接口、候选 ID 与距离返回格式、设备侧输入输出路径和资源管理方式的工程参照。

需要说明的是，**FAISS-GPU** 在本文中主要承担初步验证与接口参照作用。由于其 GPU ANN 检索 API 较为成熟，并能够清晰区分 host pointer 与 device pointer 输入路径，本文前期利用 **FAISS-GPU** 验证了查询向量输入、候选 ID 与距离返回、候选集合传递以及 **PyTOD** 候选评分之间的接口关系。随着系统目标从小规模功能验证转向大规模、外存驻留或超显存索引条件下的高吞吐候选召回，完整 **FlashTOD**

路径进一步以 FlashANNS 作为主要候选召回后端。因此，FAISS-GPU 不作为最终 FlashTOD 系统中的主要召回后端，而是作为候选召回链路的工程验证组件和设备侧接口参照。

从角色分工上看，RAPIDS 与 FAISS-GPU 分别服务于数据处理路径和前期候选召回接口验证。前者让金融表格型数据及中间结果组织尽量留在 GPU；后者为候选检索接口、设备侧输入输出和资源管理提供成熟参照。完整 FlashTOD 路径中的主要 ANN 候选召回后端仍是 FlashANNS，其外存驻留访问与异步 I/O—计算流水线能力支撑后续大规模系统实验。

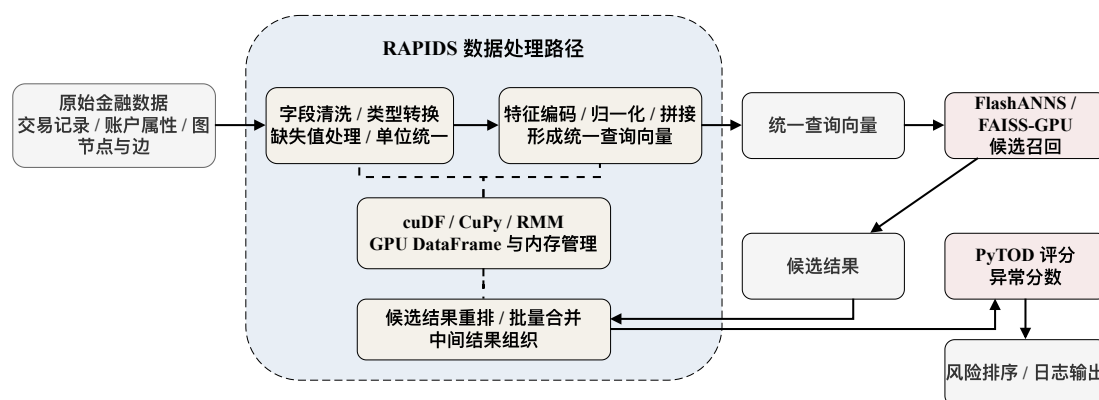


图 2-4 RAPIDS 在本文系统中的数据处理位置（本文绘制）

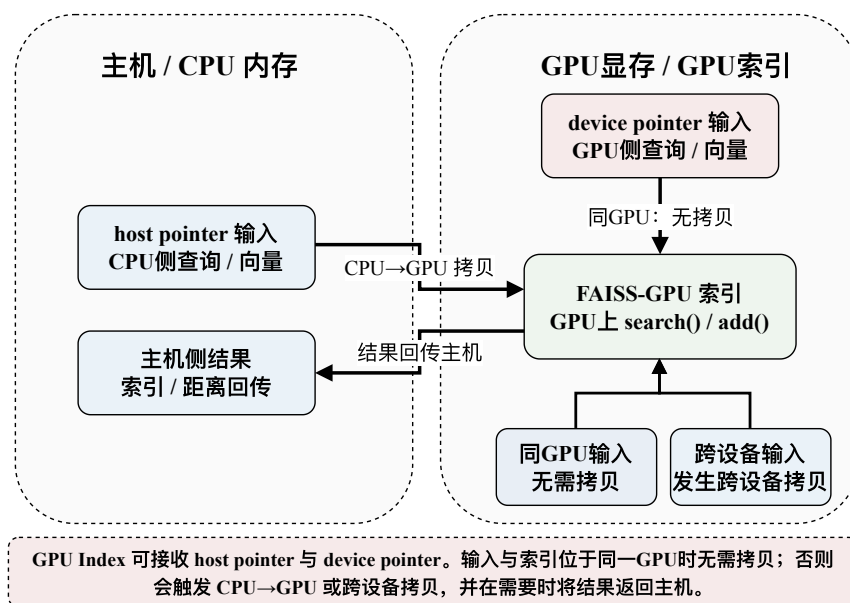


图 2-5 FAISS-GPU 的 CPU/GPU 互操作路径（根据 FAISS-GPU 文档与本文系统接口整理）

在本文系统中，RAPIDS/cuDF 与 RMM 主要把候选召回前后的表格处理、中

间结果组织和内存资源管理留在 GPU 路径内, FAISS-GPU 主要用于前期验证设备侧索引、近邻检索接口、候选 ID 与距离返回格式。完整路径中的主要召回后端仍以 FlashANNs 为主, 端到端收益取决于后续数据驻留、接口组织和边界控制。

## 2.5 多 GPU 调度与在线推理启示

当单卡系统的吞吐率接近上限, 或单卡显存无法容纳更大索引与更大批次时, 多 GPU 扩展便成为提升系统能力的自然方向。然而, 多 GPU 并不意味着性能的线性增加, 因为更多 GPU 同时也意味着更复杂的查询分发、索引组织、设备间通信、结果归并和共享 I/O 竞争<sup>[28-29]</sup>。由带宽不均衡引发的远程访问和 NUMA 问题会明显影响多 GPU 系统性能, 这与本文关注的拓扑与共享路径约束高度一致<sup>[8]</sup>。TOD 的多 GPU 支持表明, 批处理任务可以被分配到多个设备并行执行, 从而在算子级别提升总体吞吐<sup>[6]</sup>; 但对于本文所关注的“候选召回 + 异常评分”两阶段系统而言, 多 GPU 扩展要研究的是: 哪些工作可以在每张 GPU 上独立闭环完成, 哪些工作必须跨卡协同, 哪些环节又会把 CPU 重新拉回数据平面。

索引复制 (replicated) 模式倾向于在每张 GPU 上保留完整索引或完整功能链路, 使每个设备可以独立完成本地候选召回和异常评分; 其优点在于查询路径简单、设备间协同较少, 更符合吞吐优先的在线推理场景。分片 (sharded) 模式则通过多设备间分片索引或数据来扩展容量, 能够支持更大规模索引, 但也会引入额外的跨设备归并与同步开销。对本文而言, 这两种方案的取舍, 关键在于哪一种更符合金融近实时检测场景下的目标函数。单次请求更看重稳定吞吐和低尾延迟, 因此后续多 GPU 主线优先考虑索引复制架构; 当容量约束使得跨卡归并不可避免时, 再进一步讨论 sharded 及其归并代价<sup>[30]</sup>。

在归并路径上, CPU 聚合与 GPU 侧归并的差异尤为关键。若多个设备在本地生成候选或分数后, 统一回到 CPU 做聚合, 虽然实现简单, 但会把 CPU 重新拉回数据平面, 进而造成额外的 D2H/H2D 搬运和同步等待。相反, 如果将跨卡归并逻辑尽量下沉到 GPU, 并结合 NCCL 等设备间通信机制, 则可以把更多中间结果留在设备侧, 减少主机聚合带来的瓶颈。NCCL 官方文档指出, 其集体通信操作会异步入队到指定 CUDA stream 上执行, 但在同一 ncclGroupStart/End() 组内混用多个 stream 会形成所有 stream 之间的全局同步点<sup>[14]</sup>。这一点需要注意的是, GPU 侧归并虽然优于 CPU 聚合, 但只有在通信流和计算流组织得当时, 这种优势才能真正体现出来。

除了设备间通信, 存储路径与硬件拓扑同样会限制多 GPU 扩展上限。无论采用外存候选召回, 还是采用多设备共享存储路径, GPU—SSD—PCIe—NUMA 之

间的拓扑关系都可能使理论上的并行收益无法在实际系统中完全实现。共享 PCIe 交换机、共享 SSD 通道或非对称 NUMA 路径，都可能使部分设备在高并发下遭遇 I/O 争用，从而把瓶颈重新转移到存储子系统。GPUDirect Storage（下文简称 GDS）的概览文档指出，如果 GPU、网络接口卡（Network Interface Card, NIC）与存储设备位于更理想的 PCIe 拓扑关系下，可以减少 CPU 中转缓冲区（CPU bounce buffer）并提升带宽利用效率<sup>[31-32]</sup>。因此，多 GPU 扩展设计需要统筹考虑 GPU 负载均衡、拓扑感知 I/O 通道绑定、准入控制以及共享路径下的资源组织。这一判断也是第五章展开多 GPU 方案设计的重要前提。

基于以上比较，本章对多 GPU 扩展形成了三个判断：在吞吐优先的金融异常检测场景下，索引复制可作为优先考虑的主方案；当必须跨卡归并时，应优先把归并逻辑留在 GPU 侧而非回到 CPU；扩展上限不仅由设备数决定，还受到存储路径、PCIe/NUMA 拓扑与 I/O 资源组织方式的共同约束。第五章将围绕这些判断，展开多 GPU 扩展路径的具体设计。

## 2.6 本章小结

本章对相关工作进行综述，目的不仅在于为后续研究提供背景，更旨在明确后续研究需要考虑的具体问题：现有技术中，哪些可直接用于本文系统构建，哪些仅能提供思路参考，而哪些需要在系统层面做补充性组织。基于上述分析，可以形成如下判断：传统异常检测方法为评分设计提供了基础和业务解释，但其自身难以支撑大规模近实时处理；TOD/PyTOD 的工作表明，将异常评分统一映射到 GPU 执行框架是可行的，这使其适合作为本系统的评分模块；FlashANNS 的价值集中在高吞吐候选召回，而不承担异常评分本身；RAPIDS 支撑 GPU 侧数据处理路径，FAISS-GPU 在本文中主要用于前期 ANN 候选召回接口和设备侧互操作路径验证，完整候选召回后端则以后续接入的 FlashANNS 为主；多 GPU 相关研究进一步说明，索引复制、设备侧归并和拓扑感知 I/O 控制更符合本文后续的扩展方向。

在完成上述分析后，第三章的重点将从工作比较转向系统构建方案本身——以 PyTOD 为异常评分模块，以 FlashANNS 为候选召回模块，并结合 RAPIDS 数据处理路径、FAISS-GPU 初步验证经验、FlashANNS 候选召回后端、GPU 常驻执行链路以及多 GPU 数据并行与设备侧归并机制，构建一个面向金融大规模异常检测任务的统一系统框架。



## 第三章 FlashTOD 检测模型与数据准备

### 3.1 金融异常检测任务建模

金融大规模异常检测任务的本质，是在高维、多源、强时序的交易与行为数据中，快速识别少量偏离正常模式的高风险样本。与一般的二分类任务不同，这类任务不仅受到标签稀缺、样本极度不平衡等问题影响，还直接受制于系统吞吐、响应时延和设备资源约束。从工程实现角度看，任务建模不能只停留在“输入什么标签、输出什么分数”的概念层面，还必须明确原始金融数据如何进入系统、哪些对象进入候选召回、评分阶段对输入提出什么接口要求，以及输出结果如何支持后续排序、复核与追踪。因此，本节首先对金融异常检测任务进行统一建模，为后续 FlashTOD 设计建立清晰的输入、处理与输出边界。

从输入形式上看，本文将待检测对象抽象为一个由样本集合  $X = \{x_1, x_2, \dots, x_n\}$  组成的金融行为数据集，其中每个样本  $x_i$  对应一条交易记录、一个账户时间窗行为摘要或一个节点行为表示。每个样本由连续型数值特征、离散型类别特征和时间上下文特征组成，在预处理阶段被统一映射为固定维度向量表示  $v_i \in \mathbb{R}^d$ 。对于图结构金融数据，还需将节点特征与边关系经时间窗统计、邻域聚合或嵌入映射后转换为向量化表示，从而与表格型交易数据形成统一输入接口<sup>[33]</sup>。这一抽象为候选召回模块和异常评分模块提供一致的向量接口，使交易级异常、账户级异常与图节点级异常能够进入同一条执行链路。

从系统目标上看，本文的异常检测任务并非直接对全量数据执行精确评分，而是采用“候选召回—异常评分”两阶段处理框架。这样建模的直接原因在于，全量评分在大规模数据条件下会同时放大距离计算、邻域组织和中间结果存储开销，而候选召回可以先把评分对象压缩到规模可控的局部邻域，再由 PyTOD 异常评分模块完成更细粒度的异常判断。第一阶段利用高吞吐近似最近邻检索在大规模样本中快速找到与查询样本最相关的候选集合  $C_i$ ；第二阶段利用 PyTOD 在候选集合上执行张量化异常评分，生成异常分数  $s_i$ 。形式上，可将系统目标写为

$$C_i = \mathcal{R}(v_i, \mathcal{I}), \quad s_i = \mathcal{F}(v_i, C_i), \quad (3-1)$$

其中  $\mathcal{R}$  表示候选召回过程， $\mathcal{I}$  表示候选索引， $\mathcal{F}$  表示异常评分函数。由此可见，系统最终性能不再由单一评分公式决定，而是由候选召回质量、候选规模控制和评分阶段执行效率共同决定。这也是本文把两阶段结构作为系统建模基础，而不是

把它视为单纯实现细节的原因<sup>[6]</sup>。

从输出形式上看，系统可产生三类结果：其一是每个样本的异常分数，用于后续阈值判断或人工复核排序；其二是 **Top-k** 风险样本集合，用于构造重点审查名单；其三是与异常结果相关的候选邻域、关键特征统计及日志信息，用于系统追踪和后续解释分析。在金融风控场景中，这三类结果分别对应在线筛查、风险排序和审计追踪三种实际需求。这意味着，本文设计的 FlashTOD 不仅要完成候选召回和异常评分，还要让输出结果能够进入实验评估与工程监控链路。

图 3-1 对这一统一任务建模过程进行了整体展示。

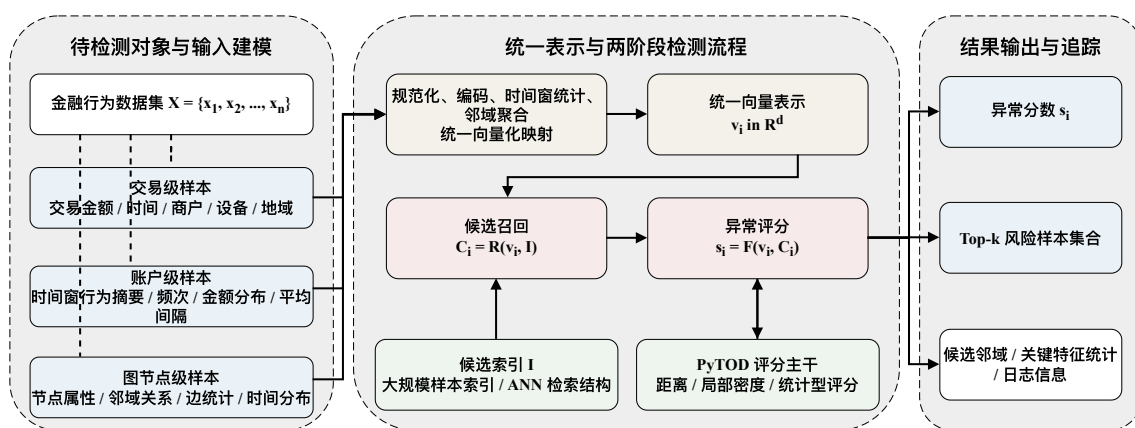


图 3-1 金融异常检测任务建模图（本文绘制）

在该统一建模下，单笔交易记录、账户时间窗摘要和图节点邻域信息不再对应三套割裂接口，而是统一映射为固定维度向量  $v_i \in \mathbb{R}^d$ 。候选召回与 PyTOD 评分据此共享输入，输出也自然收束为异常分数、**Top-k** 风险对象以及必要的候选邻域和日志信息。

### 3.1.1 FlashTOD 系统整体架构

如图 1-4 所示，FlashTOD 的总体结构可以划分为输入与特征构造、候选召回、异常评分、GPU 执行支撑、多 GPU 扩展和结果输出几个层次。系统重点不在于重新定义异常检测模型，而在于组织 PyTOD 与 FlashANNS 之间的数据流、候选接口和执行边界，使候选召回与异常评分能够在统一系统链路中协同工作。

在输入层，系统接收金融交易记录、账户行为摘要、设备信息以及关系特征等多源数据；在预处理层，这些异构输入经过缺失值处理、类别编码、时间窗统计、图关系聚合和归一化后，被统一映射为固定维度向量表示  $v_i \in \mathbb{R}^d$ 。这一统一向量既是候选召回模块的检索输入，也是后续 PyTOD 评分阶段的基础对象，从而避免为不同数据来源分别维护割裂的执行接口。

在候选召回层，完整 FlashTOD 路径以 FlashANNS 作为主要 ANN 候选召回后

端，面向大规模外存近似最近邻检索提供高吞吐候选召回路径；FAISS-GPU 则主要用于前期验证显存内或局部索引场景下的候选召回接口、设备侧输入输出路径以及候选 ID 与距离返回格式。候选召回层输出候选 ID、局部距离、排序信息以及后续评分所需的数据引用。在异常评分层，PyTOD 接收查询向量与候选集合，在候选范围内完成距离计算、局部密度或统计聚合等张量化异常评分，并生成异常分数。

在数据路径层，FlashTOD 尽量让查询表示、候选缓冲和评分中间张量保持 GPU 常驻，只在原始输入、必要日志和最终结果等边界通过页锁定内存（pinned memory）与主机交互，以压缩 CPU-GPU 数据往返和阶段同步开销。在多 GPU 扩展层，系统优先采用索引复制（replicated）架构，使每张 GPU 独立完成本地候选召回与异常评分闭环；当索引容量或部署条件限制完整复制时，再进入设备侧归并（GPU merge）或主机侧聚合（CPU aggregation）等兜底路径。最终输出层则面向风险排序、异常分数、前  $k$  个高风险对象（Top- $k$ ）、日志记录与反馈接口，为后续实验评估和工程监控提供统一结果。

## 3.2 实验数据集选取与构建

本文的数据集安排并不是把公开数据和合成数据简单并列，而是围绕全文实验口径逐层展开。真实或半真实金融数据承担现实场景下的有效性验证；1M 规模合成数据作为桥接层，用于把真实数据上的方法可用性与后续系统实验接到同一条规模链路上；10M 与 50M 规模合成数据主要对应单卡性能实验；10M、50M 与 100M 规模合成数据则构成多 GPU 扩展与压力测试的规模梯度<sup>[34-35]</sup>。这样安排的目的，是让真实口径、单卡性能口径和多 GPU 扩展口径之间能够平滑过渡，而不是让某一个数据集同时承担全部验证任务。

本文所称“真实或半真实金融数据”，主要指 IEEE-CIS、Credit Card Fraud Detection、DGraphFin 等公开金融相关数据集。这类数据通常来源于真实业务或真实任务背景，但已经经过脱敏、采样、匿名化、特征变换或竞赛化处理，仍保留金融异常检测中的高维稀疏、类别不平衡、行为偏离和关系结构等典型特征，却不能等同于金融机构生产系统中的在线原始流量。相应地，AMLSim / IBM AML 的 10M、50M、100M 数据主要用于构造可控规模下的压力测试和扩展性验证，适合观察系统吞吐、延迟、I/O 路径和多 GPU 扩展趋势，但不应直接解释为真实生产负载的等比例替代。

在真实口径的有效性验证中，本文主要使用三类公开金融数据。第一类为 IEEE-CIS 欺诈检测数据集（IEEE-CIS Fraud Detection），该数据集包含交易表与

身份表两部分，特征维度高、缺失值复杂，较接近现实支付风控场景，适合检验系统在高维结构化金融数据上的向量化表示、候选召回与评分链路是否能够稳定工作<sup>[36]</sup>。第二类为 **Credit Card Fraud Detection** 数据集，该数据集虽规模相对较小，但具有经典性和广泛使用基础，适合提供一组较稳定的对照验证口径<sup>[37-38]</sup>。第三类为 **DGraphFin** 金融图数据集（**DGraphFin**），其包含大规模节点与边关系以及金融行为特征，主要用于检验图金融数据在经过向量化映射后，能否进入本文统一的候选召回—异常评分链路<sup>[33,39-40]</sup>。在具体实验组织时，本文将真实数据实验控制在与 1M 合成数据相近的输入量级，以保证有效性验证与后续桥接实验之间的口径连续。

在 1M 规模桥接实验中，本文采用基于 **IBM** 反洗钱数据与模拟体系（**IBM AML / AMLSim**）构建的高保真合成金融交易数据。**IBM AML-Data** 明确以洗钱与异常金融交易为目标，提供带标签的合成金融交易数据集；**AMLSim** 则能够根据预定义行为规则、账户关系和交易模式生成大规模、可控分布的银行交易数据<sup>[34-35]</sup>。选择 1M 合成规模的原因，不在于它本身构成压力测试，而在于它能够在保留金融语境和结构一致性的同时，把真实集验证与更大规模系统实验接到同一数据生成体系上。

在更大规模的性能与扩展实验中，本文继续沿用 **IBM AML / AMLSim** 生成的高保真合成数据。10M 与 50M 主要用于单卡性能验证和中大规模趋势观察；多 GPU 路径则在 10M、50M 与 100M 三个层级上继续展开，其中 10M 用于索引复制主方案的起始扩展口径，50M 用于中间规模观察，100M 用于更强的容量与 I/O 压力场景。与简单随机生成数据相比，**IBM AML / AMLSim** 体系保留了更强的金融语境和结构一致性，因此可作为系统实验的统一规模底座。

不同数据集在全文中的角色由此也更加清晰：**IEEE-CIS**、**Credit Card Fraud Detection** 和 **DGraphFin** 对应真实或半真实数据口径下的有效性验证；1M 合成数据承担真实集与系统实验之间的桥接；10M、50M、100M 合成数据则分别进入单卡性能、多 GPU 扩展和超大规模压力测试。为便于说明不同数据集在全文中的基本属性，表 3-1 汇总了数据来源与实验规模口径。

**数据集名称口径说明**：需要说明的是，表中公开数据集名称、实验导出文件名和预处理后向量维度并不完全等同。部分实验文件来自公开数据集的预处理导出结果，部分文件来自 **AMLSim** 或金融合成数据生成流程。为避免将原始字段维度与预处理后向量维度混淆，本文在表中分别列出原始数据来源、实验文件名、统一向量维度和实验用途。其中，**fraud / ADBench 13\_fraud** 对应 **ADBench** 中的 **fraud** 导出文件，作为公开金融欺诈数据的预处理实验口径；**campaign / ADBench 5\_campaign**

表 3-1 实验数据集总体信息

| 数据集                         | 来源类型          | 数据形态                        | 实验规模口径             | 标签形式        | 在本文中的用途                              |
|-----------------------------|---------------|-----------------------------|--------------------|-------------|--------------------------------------|
| IEEE-CIS Fraud Detection    | 真实/半真实公开金融数据  | 结构化交易表 + 身份表, 高维表格特征、缺失值较复杂 | 约 1M 级输入口径         | 二分类欺诈标签     | 真实场景下的主有效性验证, 检验系统在高维金融表格数据上的可用性     |
| Credit Card Fraud Detection | 真实/半真实公开金融数据  | 结构化信用卡交易表格, 经典欺诈检测基准        | 按统一实验口径组织到约 1M 级输入 | 二分类欺诈标签     | 作为经典基准数据集, 用于对照验证两阶段框架的有效性           |
| DGraphFin                   | 真实/半真实公开金融图数据 | 图节点、边关系及金融行为特征              | 约 1M 级输入口径         | 节点/行为异常标签   | 验证图金融数据在向量化映射后进入统一候选召回—异常评分链路的可行性    |
| IBM AML / AMLSim (1M)       | 高保真合成金融交易数据   | 合成账户—交易网络与表格化交易记录           | 1M                 | 异常交易/可疑行为标签 | 真实集与系统实验之间的桥接规模, 用于统一后续单卡与多 GPU 实验口径 |
| IBM AML / AMLSim (10M)      | 高保真合成金融交易数据   | 合成账户—交易网络与表格化交易记录           | 10M                | 异常交易/可疑行为标签 | 单卡性能验证主规模, 同时作为多 GPU 索引复制扩展起始规模      |
| IBM AML / AMLSim (50M)      | 高保真合成金融交易数据   | 合成账户—交易网络与表格化交易记录           | 50M                | 异常交易/可疑行为标签 | 单卡中大规模性能验证与多 GPU 中间规模扩展观察            |
| IBM AML / AMLSim (100M)     | 高保真合成金融交易数据   | 合成账户—交易网络与表格化交易记录           | 100M               | 异常交易/可疑行为标签 | 多 GPU 归并、I/O 路径与超大规模压力测试             |

为 ADBench 公开异常检测数据; fraud\_handbook 来源于金融欺诈样例与合成链路; amlsim 对应 IBM AML / AMLSim 生成配置; finance\_seed\_200k 为合成种子或中间调试数据, 不作为最终主实验规模。IEEE-CIS 与 DGraphFin 在正文中用于说明真实或半真实金融数据来源和图金融口径, 但当前显存估算表只覆盖已经导出的向量文件, 因此未单独列入。

说明: 公开金融数据集的原始字段维度与进入 FlashTOD 后的统一向量维度并不等同。不同数据集在缺失值、类别编码、数值归一化和图关系聚合方式上存在差异, 本文实验在预处理后将其统一映射为固定维度向量  $v_i \in \mathbb{R}^d$ , 作为候选召回和异常评分的共同输入; 当前实现中各实验数据的向量维度及全量距离计算显存风险见表 3-2 的补充说明。IBM AML / AMLSim 高保真合成数据主要用于吞吐、延迟和扩展性压力测试, 不等同于真实金融机构生产流量。

这种安排将真实有效性验证与系统压力实验分开: IEEE-CIS、Credit Card Fraud Detection 和 DGraphFin 检验两阶段框架在真实或半真实金融数据上的可用性, 1M IBM AML / AMLSim 衔接真实口径和合成规模, 10M、50M、100M 则逐步进入单卡性能、多 GPU 扩展和压力测试。

表 3-2 数据集属性与实验用途说明

| 数据集                         | 数据来源与属性                    | 规模口径       | 原始特征说明                                 | 统一表示口径                                  | 本文用途                  |
|-----------------------------|----------------------------|------------|----------------------------------------|-----------------------------------------|-----------------------|
| IEEE-CIS Fraud Detection    | 公开金融欺诈任务数据，经竞赛化与特征脱敏处理     | 约 1M 级输入口径 | 交易表与身份表合并后约 434 列，去除 ID 与标签后的字段数随预处理变化 | 预处理后映射为固定维度向量 $v_i \in \mathbb{R}^d$    | 真实或半真实口径下的高维表格有效性验证   |
| Credit Card Fraud Detection | 公开信用卡交易欺诈数据，特征经匿名化变换       | 约 1M 级输入口径 | 30 个输入特征，包括匿名化主成分、交易时间与金额              | 预处理后映射为固定维度向量 $v_i \in \mathbb{R}^d$    | 经典金融欺诈基准上的对照验证        |
| DGraphFin                   | 公开金融图任务数据，包含节点、边关系与行为特征    | 约 1M 级输入口径 | 17 维节点原始属性，并结合边关系与邻域行为统计               | 节点及邻域信息聚合为固定维度向量 $v_i \in \mathbb{R}^d$ | 图金融数据向量化后进入统一链路的有效性验证 |
| IBM AML / AML-Sim (1M)      | 高保真合成金融交易数据，标签与异常比例随生成配置确定 | 1M         | 合成账户、交易网络与表格化交易记录                      | 预处理后映射为固定维度向量 $v_i \in \mathbb{R}^d$    | 真实口径与系统性能实验之间的桥接规模    |
| IBM AML / AML-Sim (10M)     | 高保真合成金融交易数据，主要放大样本规模       | 10M        | 合成账户、交易网络与表格化交易记录                      | 预处理后映射为固定维度向量 $v_i \in \mathbb{R}^d$    | 单卡性能验证与多 GPU 索引复制起始规模 |
| IBM AML / AML-Sim (50M)     | 高保真合成金融交易数据，强化中大规模路径压力     | 50M        | 合成账户、交易网络与表格化交易记录                      | 预处理后映射为固定维度向量 $v_i \in \mathbb{R}^d$    | 单卡中大规模趋势与多 GPU 中间规模观察 |
| IBM AML / AML-Sim (100M)    | 高保真合成金融交易数据，用于更强容量与 I/O 压力 | 100M       | 合成账户、交易网络与表格化交易记录                      | 预处理后映射为固定维度向量 $v_i \in \mathbb{R}^d$    | 多 GPU 归并、共享路径冲突与压力测试  |

表 3-2 补充：当前已导出实验文件的向量维度与全量距离计算显存估算

| 数据集或规模                           | 样本数 $n$ | 维度 $d$ | 输入矩阵     | 距离矩阵 $n^2$ | 显式差分 $n^2d$ | 说明               |
|----------------------------------|---------|--------|----------|------------|-------------|------------------|
| fraud / ADBench<br>13_fraud      | 284807  | 29     | 31.5 MiB | 302 GiB    | 8.56 TiB    | 真实集，全量距离矩阵已超单卡显存 |
| campaign / ADBench<br>5_campaign | 41188   | 62     | 9.74 MiB | 6.32 GiB   | 392 GiB     | 输入很小，但显式成对差分仍不可取 |
| fraud_handbook (1M)              | 1M      | 20     | 76.3 MiB | 3.64 TiB   | 72.8 TiB    | 合成桥接规模           |
| fraud_handbook (10M)             | 10M     | 20     | 763 MiB  | 364 TiB    | 7.11 PiB    | 单卡与多 GPU 起始性能规模  |
| fraud_handbook (50M)             | 50M     | 20     | 3.73 GiB | 8.88 PiB   | 178 PiB     | 中大规模压力场景         |
| fraud_handbook (100M)            | 100M    | 20     | 7.45 GiB | 35.5 PiB   | 711 PiB     | 超大规模压力场景         |
| amlsim (1M)                      | 1M      | 9      | 34.3 MiB | 3.64 TiB   | 32.7 TiB    | 合成桥接规模           |
| amlsim (10M)                     | 10M     | 9      | 343 MiB  | 364 TiB    | 3.20 PiB    | 中大规模性能场景         |
| amlsim (100M)                    | 100M    | 9      | 3.35 GiB | 35.5 PiB   | 320 PiB     | 超大规模压力场景         |
| finance_seed_200k                | 200k    | 20     | 15.3 MiB | 149 GiB    | 2.91 TiB    | 合成种子或中间数据        |

注：估算均按 float32 计算。输入矩阵按  $4nd$  字节估算；距离矩阵按  $4n^2$  字节估算，可视为直接保留全量样本两两距离的最低存储量级；显式差分按  $4n^2d$  字节估算，用于说明朴素成对差分中间张量的显存风险。实际 PyTOD/TOD 实现会通过批处理、算子组织或候选召回缩小计算范围，本文不将该估算解释为实际运行时固定占用。

### 3.3 数据预处理与特征构造

由于不同数据集在字段命名、数据类型、缺失值比例、标签分布以及时间粒度上存在较大差异,原始金融数据无法直接输入候选召回和异常评分模块。在FlashTOD中,数据预处理与特征构造的任务,不是为每个数据集分别堆叠一套特征工程,而是建立一套能够同时服务于候选召回和异常评分的统一向量接口。本节将进一步讨论,如何在尽量保留业务语义的前提下,将表格型金融数据和图结构金融数据整理为可检索、可评分、可批处理的向量表示  $v_i \in \mathbb{R}^d$ 。其中,  $d$  由最终预处理脚本确定,并在候选召回与异常评分实验中保持一致。

对于表格型金融数据,预处理过程主要包括缺失值处理、异常字段清洗、数值归一化、类别编码以及时间字段规范化。缺失值处理方面,针对数值型字段采用均值、中位数或业务合理缺省值进行填补,针对类别型字段则引入专门的缺失标识,以避免无效缺失模式被误当作正常类别。异常字段清洗包括删除重复样本、去除明显无意义标识列、统一金额和时间单位等。对于数值型特征,本文采用标准化或归一化处理,降低金额、频率、时差等不同尺度特征之间的量纲影响;对于类别型字段,则采用 one-hot、频次编码或目标统计前的安全编码方式,以便后续统一映射至数值空间<sup>[36-37]</sup>。经过这一轮处理后,表格型交易记录将进入统一向量接口,直接服务于候选召回与评分阶段。

对于时间信息,本文采用滑动时间窗和行为统计相结合的方式构造增强特征。以单个账户、设备或商户为中心,在固定时间窗口内统计交易频次、交易金额分布、平均时间间隔、夜间交易比例、跨地域交易切换次数等行为特征,并将其与原始交易字段拼接形成更完整的查询表示。这样处理的考虑在于,金融异常往往不是由单个字段瞬时决定,而是体现在与历史行为模式的偏离关系上。通过时间窗统计,系统可以在不引入复杂序列模型的前提下,把一部分时序上下文编码到统一向量中,使候选召回与异常评分共享同一组行为特征基础。

对于图金融数据,本文不直接在原始图结构上执行复杂图学习算法,而是先将图节点与边关系转换为节点级向量表示。具体做法包括:对节点原始属性进行规范化与编码;对邻域中边的数量、方向、金额统计和时间分布等进行聚合;将节点本身属性与邻域统计特征拼接形成统一向量。这样处理的重点在于接口统一: DGraphFin 等图金融数据经过映射后,可以与表格型交易数据一起进入同一条候选召回—异常评分链路,而不需要为图数据额外建立一套与系统主流程平行的执行结构<sup>[33]</sup>。

在最终特征构造阶段,本文将不同来源的特征统一组织为固定维度向量表示,并将其作为连接候选召回与异常评分的统一向量接口。该接口至少需要满足两项



要求：一是能够作为候选召回模块的统一查询输入，在前期验证阶段用于 FAISS-GPU ANN API 接口联调，在完整 FlashTOD 路径中则作为 FlashANNS 索引的查询输入；二是能够作为 PyTOD 异常评分模块的输入，参与距离计算、候选排序和异常评分。为保证这一接口在不同数据集上都可复用，本文在特征构造后统一执行向量维度检查、类型检查与异常值截断，尽量避免后续模块再针对数据源差异做额外适配。通过这一设计，原本异构的金融数据被整理成候选召回与异常评分共同接受的输入形式，为后续 FlashTOD 执行流程提供了稳定的数据接口。

本文将数值、时间、类别和图关系特征统一收束为固定维度向量，使不同来源金融数据能够进入同一候选召回与评分接口；字段到派生特征的具体组织见附录 B 补充表 B-1。

### 3.4 PyTOD 异常评分模块设计

在本文的两阶段异常检测框架中，PyTOD 承担的是候选集合上的异常评分模块角色。它接收的是由候选召回阶段输出的查询向量、候选张量及其索引映射信息，并在这一局部邻域内完成距离计算、近邻排序和异常性聚合，从而生成最终异常分数<sup>[6]</sup>。这一用法将 PyTOD 放在系统的“精排/评分”位置，从而把评分阶段的计算边界控制在候选预算范围内。

从执行接口上看，候选集合  $C_i$  一旦产生，查询样本  $v_i$  与候选集合中的向量可以直接进入 PyTOD 的计算链路，被组织为批量距离矩阵、局部近邻排序结果与聚合统计结果。PyTOD 评分阶段主要完成四类操作：计算查询样本与候选集合之间的距离；对候选集合按距离进行排序或筛选；对局部距离分布、密度差异或邻域结构执行聚合；根据预定义评分函数输出异常分数。这样组织后，PyTOD 能够成为一个围绕候选集工作的 GPU 评分内核。

在评分指标选择上，为了保证同一套执行路径能够适配不同数据集和业务场景下的异常性定义，本文保留基于距离、局部密度和邻域统计的通用评分接口。本文使用 PyTOD 作为统一评分骨架，同时保留可切换评分函数的能力，避免把固定评分公式直接写死在系统中。

本文对 PyTOD 的使用方式与其原本将大规模全样本直接用于处理异常检测的思路不同。TOD/PyTOD 更强调通过自动批处理和多 GPU 支持缓解全量异常检测的复杂度问题，而本文把它放在候选召回之后，专门负责较小候选集合上的精细评分。需要说明的是，这一调整在没有改变 PyTOD 评分能力的同时，重新限定了它在系统中的职责，使复杂度更多集中在“候选召回 + 数据路径优化”这一便于系统优化展开的层面。

图 3-2 给出了 PyTOD 异常评分模块的执行流程，表 3-3 对常用评分方式与张量算子映射进行了归纳。

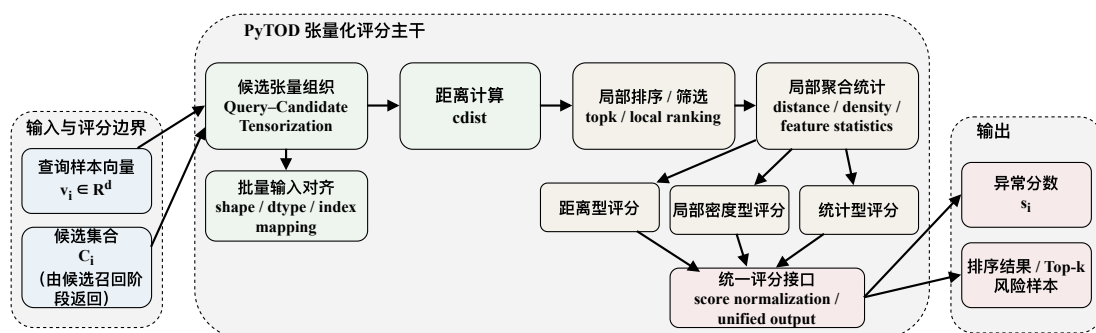


图 3-2 PyTOD 异常评分模块流程图（本文绘制）

表 3-3 异常评分指标与算子映射

| 评分类型    | 代表性评分思路                  | 候选集上的计算对象         | 简化算子链                                                          | 在本文中的定位          |
|---------|--------------------------|-------------------|----------------------------------------------------------------|------------------|
| 距离型评分   | 依据查询样本与候选邻域的距离分布刻画异常程度   | 查询向量与候选集合的局部距离关系  | $\text{cdist} \rightarrow \text{topk} \rightarrow \text{聚合}$   | 主评分方式，直观反映样本偏离程度 |
| 局部密度型评分 | 比较查询样本与邻域样本的局部密度差异       | 查询样本及其候选邻域的局部密度结构 | $\text{cdist} \rightarrow \text{topk} \rightarrow \text{密度比值}$ | 补充识别“相对异常”样本     |
| 统计型评分   | 基于候选邻域内特征分布、分位点或边缘偏离程度评分 | 查询样本在候选邻域中的特征偏离情况 | 统计/直方图 $\rightarrow$ 聚合                                        | 轻量级辅助评分或对照评分方式   |
| 融合评分接口  | 对不同评分结果进行统一输出，形成最终风险排序   | 距离、密度、统计等候选集评分结果  | 归一化 $\rightarrow$ 加权输出                                         | 体现统一评分模块的可扩展性    |

### 3.5 FlashANNS 候选召回模块设计

在 FlashTOD 中，FlashANNS 负责从大规模样本中快速生成与查询样本最相关的候选集合，为后续精细评分阶段提供规模可控且质量可接受的候选近邻集合<sup>[7]</sup>。这意味着，候选召回模块是评分阶段的前端输入组织模块：它决定了后续 PyTOD 看到什么样的局部邻域、在多大预算下完成评分，以及评分阶段需要承担多大计算负担。

在实现演进上，本文前期利用 FAISS-GPU 较完善的 ANN API 对候选召回接口进行初步验证，主要用于确认查询向量输入、候选 ID 与距离返回、候选集合传递和 PyTOD 候选评分之间的接口关系。随着系统目标转向大规模外存驻留或超显存索引场景，完整 FlashTOD 路径中的候选召回后端进一步切换为 FlashANNS，以

利用其异步 I/O——计算流水线提升大规模候选召回吞吐。

在索引构建阶段，本文以向量化后的样本表示  $v_i$  为输入，构建面向 ANN 检索的候选索引。对于规模较小且全部可驻留在显存内的实验场景，可直接使用 GPU 端索引完成近似搜索；对于规模更大或需要考虑外存路径的场景，则采用 FlashANNs 异步 I/O——计算流水线所支持的高吞吐检索路径。这里的设计重点不是把所有查询都推向最高召回精度，而是让索引结构和检索路径能够稳定服务于后续评分阶段：既要让查询以较低延迟定位到局部候选邻域，又要保留参数调节空间，使召回质量与系统吞吐之间能够按实验目标进行权衡。

在查询执行阶段，FlashANNs 对输入查询向量  $v_i$  快速检索并返回一个候选集合  $C_i$ 。在本文系统中，候选集合不仅包含候选 ID，还包含必要的距离值、局部排序信息或可支持后续评分的数据引用。之所以要把这些字段一起看作接口设计的一部分，是因为一旦候选结果还需要在 CPU 侧进行大规模整理、重排和格式转换，召回阶段获得的吞吐收益就会在模块边界被快速抵消。考虑以上原因，本文在设计候选召回模块时，特别强调输出格式与评分模块输入接口的一致性，即尽量让候选结果以可直接进入 GPU 评分链路的形式表达。

在参数组织方面，本文并不把候选集合规模、召回近邻数量和索引搜索参数理解为孤立的调参项，而是把它们视为系统预算约束。候选集合过小，可能导致真实异常邻域缺失，从而影响评分质量；候选集合过大，则会使后续 PyTOD 评分重新承受高复杂度负担。类似地，搜索深度过低会使召回不足，过高则会直接侵蚀系统吞吐。因此，本文对这些参数的控制逻辑并不是追求极限检索精度，而是追求对后续异常评分足够好的候选质量，使候选召回真正服务于系统总体目标，而不是成为独立优化指标的终点。

图 3-3 展示了候选召回模块的基本执行过程，表 3-4 总结了关键控制参数。

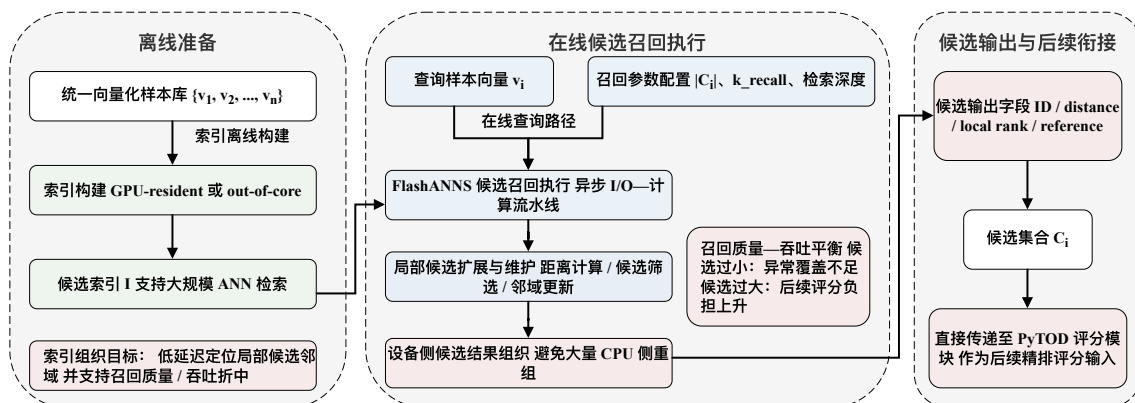


图 3-3 FlashANNs 候选召回执行流程图（本文绘制）

表 3-4 候选召回模块主要参数

| 参数名称        | 符号/表示                                      | 含义                      | 对系统的影响                  | 设置原则                    |
|-------------|--------------------------------------------|-------------------------|-------------------------|-------------------------|
| 候选集合规模      | $ C_i $                                    | 每个查询样本返回的候选样本数          | 过小会降低异常覆盖率，过大会增加后续评分负担  | 在召回质量与评分复杂度之间折中设置       |
| 近邻返回数量      | $k_{\text{recall}}$                        | 候选召回阶段实际返回的近邻数量         | 决定局部邻域完整性与后续评分的输入范围     | 与评分阶段使用的局部邻域规模保持协调      |
| 检索深度/访问强度参数 | nprobe、efSearch 等                          | 控制 ANN 检索阶段访问的索引范围或搜索强度 | 参数越高通常召回质量更好，但吞吐下降      | 以“对评分足够好”为目标，而非追求极限检索精度 |
| 索引驻留方式      | GPU 常驻 (GPU-resident) / 外存驻留 (out-of-core) | 候选索引驻留在显存内或通过外存路径访问     | 影响检索延迟、I/O 压力与系统扩展能力    | 小规模优先显存驻留，大规模场景采用外存驻留路径 |
| 候选输出字段      | ID、distance、rank、reference                 | 候选模块返回给评分模块的主要内容        | 决定后续 PyTOD 评分接口是否需要额外重组 | 尽量采用可直接进入 GPU 评分链路的输出格式 |
| 查询批大小       | batch_size                                 | 单次提交到候选召回模块的查询数量        | 影响设备利用率、时延与流水线连续性       | 与 GPU 资源、显存容量和时延约束共同确定  |
| 精排预算        | rerank_budget                              | 进入高精度评分阶段的候选上限          | 控制精排复杂度与最终检测效果之间的平衡     | 与候选集合规模联动设置，避免全量重评分     |

### 3.6 FlashTOD 执行流程

在完成数据预处理、向量化表示、候选召回模块和异常评分模块设计后，本文进一步将二者整合为统一的执行流程。该流程覆盖从查询输入到异常结果输出的完整链路，其目标是在保持模块职责清晰的同时，把各阶段的输入输出接口明确下来，并尽量减少中间结果的重复复制和格式转换。本文将 FlashTOD 的基础执行流程概括为输入向量化、候选召回、候选张量组织、PyTOD 评分和风险输出五个环节。

具体而言，系统首先接收原始交易记录、账户行为摘要或图节点样本，经由数据预处理与特征构造转换为统一向量表示。随后，向量表示作为查询输入送入候选召回模块：在前期初步验证阶段，该步骤可由 FAISS-GPU 的成熟 ANN API 完成，以验证候选召回与 PyTOD 评分之间的接口；在完整 FlashTOD 路径中，该步骤由 FlashANNs 作为主要 ANN 后端完成，以支持大规模外存驻留或超显存索引条件下的候选召回。候选集合在 GPU 侧完成必要的组织与重排，形成 PyTOD 可以直接接收的候选张量、距离张量及索引映射结构；PyTOD 在候选集合上执行张量化异常评分，输出异常分数及对应排序结果；最终，系统根据异常分数形成风险样本、日志记录和必要解释信息，并进入后续实验采样或部署监控链路。其中，统一向量接口负责连接异构金融特征与候选召回模块，候选张量组织则负责将候选 ID、距离和排序信息整理为 PyTOD 可直接使用的评分输入。

至此，第三章所完成的主要是功能级衔接：系统知道输入从哪里来、候选如何生成、评分如何执行、结果如何输出，但功能可行并不等于数据路径已经低开销。若候选召回结果仍需频繁回传 CPU 再被整理成评分输入，系统链路就会重新引入高昂的 CPU↔GPU 往返与阶段同步成本；若中间结果能够以 GPU 常驻张量或低开销设备对象表达，则整个“候选召回—异常评分”过程才更接近统一 GPU 执行链路。因此，第四章的重点不再停留在验证两阶段流程“能否工作”，转而继续压缩这条流程在单卡上的数据路径成本。

图 3-4 进一步展示查询向量、候选结果与评分中间张量在模块间的传递方式。

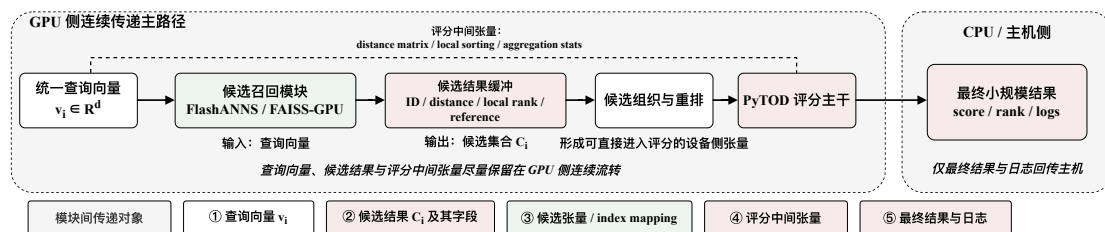


图 3-4 模块间中间结果传递示意图（本文绘制）

图中 FAISS-GPU 表示前期 ANN API 初步验证路径；完整 FlashTOD 候选召回后端以 FlashANNs 为主。

### 3.7 基础实验与初步验证

在进入系统级优化和多 GPU 扩展之前，有必要首先验证 FlashTOD 在功能上的正确性，并把真实口径实验与后续 1M 合成桥接实验接到同一条规模链路上。因此，本节的基础实验围绕三个更具体的问题展开：候选召回与异常评分链路是否能够正确联通；在候选集合上执行 PyTOD 评分是否会带来不可接受的检测效果损失；当前初始实现是否已经具备继续进入系统优化阶段的前提。

在功能正确性验证中，实验主要关注查询输入、候选集合生成、候选组织与异常评分输出之间是否存在接口不一致、结果异常或批处理错误。实验安排先以真实或半真实金融数据完成基础联调，再在 1M 合成规模下重复相同口径的检查，以确认候选召回与异常评分接口在跨数据来源条件下仍保持一致。随后，将候选集评分结果与小规模全量评分结果进行比较，以观察候选召回对异常评分口径的影响。实验重点考察精确率—召回率曲线下面积（Precision-Recall Area Under Curve, PR-AUC）、Top- $k$  召回率（Recall@ $k$ ）和 Top- $k$  重合度（Top- $k$  overlap）等指标在候选预算受限后的变化，以判断主要排序结果、Top- $k$  风险样本与整体检测效果是否仍保持在可接受范围内。如果候选集合过小导致异常模式无法被覆盖，则说明候选召回参数仍需调整；如果候选评分与全量评分在主要排序结果上保持较高一

致性，则说明两阶段框架具备继续开展系统优化的基础。

基础实验还将初步观察不同数据集下候选召回与异常评分之间的关系。结构化交易数据中的候选召回结果往往对金额、时间和设备行为更敏感，而图金融数据中的候选召回结果则更容易受到邻域统计特征影响。做这一层观察是为了在进入第四章和第五章之前，先确认统一接口和两阶段流程在不同金融数据形态下都能够稳定工作。图 3-5 展示了不同数据集上的候选召回—候选评分初步有效性结果。

本节实验以真实或半真实数据和 1M 合成桥接规模为入口，统一采用固定维度向量表示，并以候选集评分对照小规模全量评分。关注重点是候选预算变化对排序一致性、Top- $k$  风险样本重合度以及 PR-AUC、Recall@ $k$  的影响，从而判断两阶段流程是否足以支撑后续系统优化。

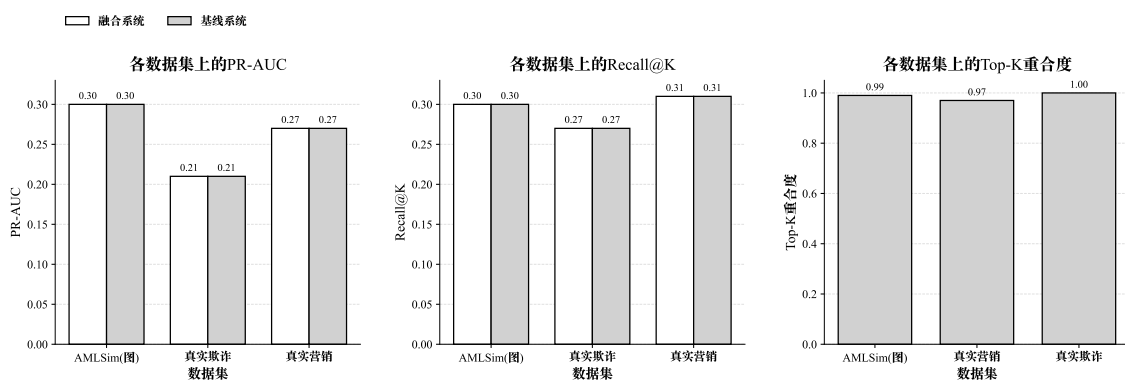


图 3-5 不同数据集上的候选召回—候选评分初步有效性验证结果（实验数据绘制）

### 3.8 本章小结

本章围绕 FlashTOD 的功能结构，完成了任务建模、数据集选取、预处理与特征构造、异常评分模块设计、候选召回模块设计以及执行流程构建。通过对金融异常检测任务的统一抽象，本文将表格型金融数据与图金融数据共同映射为统一向量接口，并建立了“候选召回—异常评分”的两阶段处理框架。在实验准备上，本章进一步明确了全文统一的数据规模口径：真实或半真实数据用于有效性验证，1M 合成数据承担桥接层，单卡性能实验沿 1M、10M、50M 合成规模展开，多 GPU 扩展则继续延伸到 100M 压力场景，从而为后续章节中的单卡优化和多 GPU 验证提供一致的数据基础。

本章解决了系统对象如何建立、模块接口如何对齐、基础实验如何起步的问题，但并未真正解决数据路径成本。到本章为止，PyTOD 与 FlashANNS 已经不再是两个孤立模块，而是被放置在一条候选召回—异常评分执行链路中统一考察；然

而，这条链路目前主要满足功能可行和接口联通，尚未从系统层面压缩 CPU↔GPU 数据往返与阶段同步开销。因此，第四章将进一步进入单卡优化，重点讨论 GPU 主导执行链路、异步 I/O—计算流水线以及数据路径压缩问题。

## 第四章 单卡场景下的系统设计与数据路径优化

### 4.1 单卡系统需求与总体架构

在完成第三章的任务建模、数据准备和 FlashTOD 执行流程设计之后，本节将研究重点进一步转向系统实现层面，即在单 GPU 条件下，如何通过合理的数据路径组织与执行链路压缩，使异常检测系统在保证检测效果的前提下获得更高吞吐率和更低延迟。单卡场景在本文中具有两层作用：一方面，它是多 GPU 扩展的前提，只有单卡链路本身足够高效，多卡扩展才有现实意义；另一方面，它更容易把系统瓶颈完整地暴露出来，尤其是 CPU↔GPU 数据交换、候选结果组织、I/O 等待与算子同步之间的连锁影响。

从系统需求出发，单卡异常检测系统需要同时满足三个目标：在有限显存条件下稳定处理大规模输入，并通过合适的批处理组织支撑候选召回与异常评分连续执行；尽量减少 CPU 对数据平面的直接介入，使查询特征、候选结果与评分中间张量更多停留在 GPU 一侧；在存在外存访问或多阶段流水线的条件下，尽可能重叠 I/O 与 GPU 计算，压缩串行等待时间。由此可见，单卡优化的重点是围绕 GPU 构建一条中间结果传递成本更低的端到端执行链路。

基于这一目标，本文将单卡系统划分为五个核心模块：数据接入与预处理模块、查询向量组织模块、候选召回模块、异常评分模块以及结果输出与日志模块。它们分别承担原始金融记录接入与字段规范化、统一设备侧查询表示构造、候选集合高吞吐返回、候选集上的张量化评分以及最终结果回传与运行时记录等职责。这样的模块划分把热路径和边界路径区分开来：前四个模块构成候选召回到异常评分的核心执行链路，最后一个模块只在必要边界与主机交互。系统总体设计调整之后，能够使系统围绕 GPU 主导路径重新组织它们的衔接关系，避免让这些模块继续沿用“CPU 预处理—GPU 检索—CPU 整理—GPU 评分”的分段模式。

从执行结构上看，本章所研究的单卡系统并不把 FlashANNS 和 PyTOD 当作两个彼此独立的程序，而是把它们放在同一条执行链路的前后两段：前者负责高吞吐候选召回，后者负责精细异常评分，二者之间通过设备侧中间结果表达和统一接口进行衔接。后续各节将围绕这一思路展开，先分析原始链路中的主要瓶颈，再给出 GPU 主导的端到端执行链路；其中，FAISS-GPU 主要用于前期候选召回接口和设备侧路径验证，完整候选召回后端以 FlashANNS 为主，单卡优化则围绕 FlashANNS 候选输出、PyTOD 评分输入、页锁定内存、异步 stream 和双缓冲等机



制展开。

图 4-1 展示了单卡系统五个核心模块的连接关系，为后续按数据路径逐层展开分析提供了整体视角。

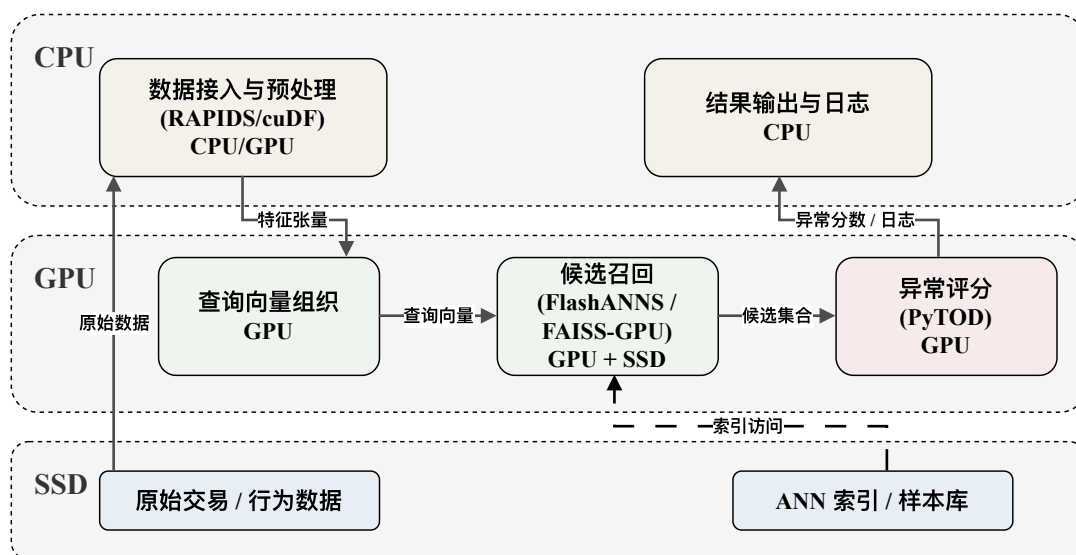


图 4-1 单卡异常检测系统总体架构图（本文绘制）

图中 FAISS-GPU 表示前期候选召回接口和设备侧检索路径的初步验证组件，完整 FlashTOD 单卡路径中的候选召回后端以 FlashANNs 为主。

## 4.2 单卡场景下的瓶颈分析

尽管第三章已经完成了 FlashTOD 执行流程的构建，但从系统执行效率看，原始链路仍存在明显瓶颈。问题在于这些阶段的衔接方式仍然保留了典型的“CPU 主导分段执行”特征：查询特征多在 CPU 侧整理，候选结果在 GPU 上产生后又返回 CPU 重组，后续异常评分再重新依赖 GPU。这样一来，原本分散在不同阶段边界上的开销会在 FlashTOD 链路中叠加出现：传输代价增加，同步点变多，等待时间也被层层放大。

首先，主机与设备之间的频繁边界交换会削弱候选召回和异常评分的单点加速收益。GPU 索引既可接收 host 指针，也可接收 device 指针；当输入本来就在与索引相同的 GPU 上时，不发生拷贝且速度最快，否则会发生 CPU→GPU 或跨设备拷贝，结果也会在完成后返回相应位置。这意味着，如果查询特征在 CPU 侧完成拼接和标准化，再逐批传输到 GPU，那么即便 GPU 检索本身很快，整条链路仍要持续承担入口传输成本。若候选结果又在 GPU 侧生成后回到 CPU 被重新整理，再送入 PyTOD 评分，则一次查询实际上会跨越两轮主机—设备边界。这样形成的

折返路径在小规模实验中未必突出，但在百万级输入和高并发场景下会迅速累积，成为系统瓶颈。

其次，候选中间结果的表达方式会继续放大链路成本。候选召回模块返回的通常不仅是候选样本 ID，还可能包含局部距离、索引位置或局部排序信息。若这些结果优先被组织成 CPU 侧更容易解释的数据结构，例如 Python 对象列表、pandas DataFrame 或主机侧稀疏结构，则后续 PyTOD 在 GPU 上评分前还需要重新完成设备侧重建。这样带来的问题不仅是一次额外的数据复制，更在于它打断了 GPU 对中间张量的连续访问与批量组织能力。因此，在单卡系统中，候选结果若不能以设备侧友好的形式表达和传递，评分阶段就很难形成低开销的连续执行链路。

再次，当候选召回需要访问外存或已经存在明显 I/O 压力时，SSD-I/O 与 GPU 计算之间的串行等待会把前两类问题进一步放大。传统外存驻留 (out-of-core) ANN 系统的主要问题之一在于 SSD 访问与 GPU 计算无法有效重叠，系统被迫在“读数据—等数据—算数据”的节奏中推进，从而使高性能 GPU 长时间等待 I/O 完成。而在本文的 FlashTOD 链路中，如果检索内部已经开始重叠 I/O 与计算，但候选召回结束后仍要回到 CPU 完成数据组织和评分前准备，那么被检索阶段隐藏掉的一部分等待会在后续边界重新出现。也正因为这些开销是串联叠加的，单卡优化不能只依赖某一个模块的局部加速，而必须从端到端执行链路出发识别瓶颈。

最后，内存分配与资源组织方式也会在单卡环境下引入隐式开销。StandardGpuResources 会预先保留 scratch memory，以避免频繁 cudaMalloc/cudaFree 导致的同步和性能恶化；页锁定主机内存与内存池机制也有助于稳定传输并减少分配抖动。这说明，即便查询路径和候选结果传递已经更多地留在 GPU 侧，如果系统仍在热路径中频繁申请和释放临时缓冲区，也会因为资源管理本身而造成明显损失。

基于上述分析，本文将单卡场景下的瓶颈概括为四类：查询与候选结果跨边界传递过于频繁、候选中间结果依赖主机侧重组、SSD-I/O 与 GPU 计算之间存在串行等待，以及内存与资源管理带来的隐式同步。这四类问题并不是彼此孤立存在的，它们共同说明单卡系统要获得稳定收益，关键在于压缩整条数据路径。

图 4-2 以并列方式对比了原始 CPU↔GPU 往返链路和优化后的 GPU 主导连续链路，直观说明多次 H2D/D2H 传输和阶段同步如何成为主要瓶颈；表 4-1 则对这些瓶颈的类型与后续优化方向进行了分类。

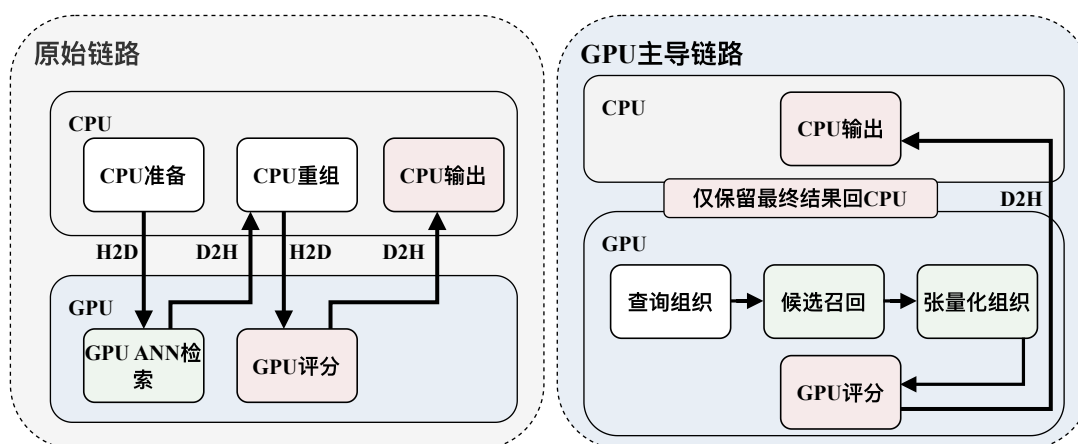


图 4-2 原始链路与 GPU 主导链路对比图（本文绘制）

表 4-1 单卡主要瓶颈与对应优化项

| 瓶颈类型          | 发生位置                         | 主要影响                         | 对应优化方案                                                | 涉及组件                                                     | 预期收益                                       |
|---------------|------------------------------|------------------------------|-------------------------------------------------------|----------------------------------------------------------|--------------------------------------------|
| 高频 CPU↔GPU 往返 | 查询准备、候选输出与评分结果边界             | 放大 H2D/D2H 次数，增加同步等待与尾延迟     | GPU 主导执行链路，边界处结合页锁定内存做必要交换                            | RAPIDS/cuDF<br>PyTOD<br>RMM 页锁定内存                        | 降低跨设备往返频次，压缩尾延迟并提高端到端吞吐                    |
| 候选重组开销        | 候选召回到异常评分的接口边界               | 主机侧 pack_candidates、序列化重建成本高 | 采用设备常驻候选缓冲（device-resident candidate buffer）和统一张量交接接口 | FlashANNS<br>PyTOD<br>FAISS-GPU<br>初步接口验证                | 缩短候选打包阶段，避免 CPU 重新成为数据平面瓶颈                 |
| I/O 串行等待      | SSD 读页、ANN 图扩展与局部距离计算之间      | GPU 空转等待数据，召回吞吐受限            | 异步 I/O 重叠、双缓冲和流级并行推进                                  | FlashANNS<br>NVMe SSD<br>CUDA streams                    | 隐藏部分读页等待，提高候选召回有效吞吐与 GPU 忙碌度               |
| 资源分配抖动        | 临时缓冲区、scratch memory 与主机交换缓冲 | 频繁分配释放导致延迟抖动和显存碎片            | 内存池复用、scratch 预分配与微批双缓冲                               | FAISS<br>GpuResources<br>RMM memory pool<br>CUDA streams | 稳定阶段延迟，降低分配开销（allocation overhead），改善资源利用率 |

### 4.3 GPU 主导的端到端执行链路设计

针对上一节所分析的瓶颈，本文设计面向单卡场景的 GPU 主导端到端执行链路。这一路径的核心做法是：在 FlashANNs 提供的异步 I/O—计算流水线基础上，把查询特征组织、候选检索结果传递与异常评分中间张量计算尽量收拢到 GPU 一侧，仅在必要边界通过页锁定内存等机制完成主机交互，从而同时压缩 SSD-I/O 与 GPU 计算之间的串行等待，以及主机—设备边界上的额外搬运。

从总体原则上看，GPU 主导端到端执行链路包含三层含义：查询特征的最终组织尽可能在 GPU 侧完成；候选召回结果以设备侧可直接消费的形式交给异常评分模块；评分阶段产生的中间张量继续在设备侧推进，只有最终分数、排序结果或必要日志信息再回传 CPU。对应到系统行为上，这一路径的真正收益并不是某一次传输被完全消除，而是把原先反复出现的“主机—设备—主机—设备”折返模式压缩为少数必要边界交互。

#### 4.3.1 查询特征在 GPU 侧的驻留与组织

在原始实现中，金融交易数据通常先以表格形式在 CPU 侧完成清洗、归一化和拼接，再整体打包为查询向量传入 GPU。这样组织虽然直观，但会让所有查询都在入口处承担一次完整的 H2D 成本。本文在这一位置的设计决策，是把部分预处理与特征组织前移到 GPU 数据处理后端，使关键查询向量尽可能在设备侧形成最终表示。RAPIDS 的 GPU DataFrame 组件 cuDF，以及与其协同的 GPU 数组计算库 CuPy，提供了批量列运算、张量表达和统计聚合能力，使这一做法在工程上具备可行性<sup>[10]</sup>。预期收益也较为直接：查询入口不再承担整批向量的重复搬运，后续候选召回阶段能够更快接收到可检索的设备侧表示。

#### 4.3.2 候选检索结果在 GPU 侧的保持与传递

候选召回阶段结束后，系统面临的关键问题是：候选结果应以何种形式进入异常评分模块。本文在这里的设计决策，是尽量保持候选结果为设备常驻（device-resident）对象，即候选 ID、局部距离和必要索引信息不先回 CPU 再重建，而是在 GPU 上直接组织为 PyTOD 可消费的中间张量。FAISS-GPU 文档指出，同 GPU 上的输入与输出可以避免额外拷贝，且 GpuResources 可通过预分配 scratch memory 降低中间缓冲区分配开销<sup>[13]</sup>。沿着这一接口设计，前期验证阶段中 FAISS-GPU 的候选 ID 与距离输出可用于确认 PyTOD 评分输入接口；在完整 FlashTOD 路径中，FlashANNs 输出的候选结果则以设备侧对象形式进入 PyTOD 评分阶段，预期收益主要体现在候选打包和阶段切换成本的下降。

### 4.3.3 异常评分中间张量的 GPU 常驻执行

PyTOD 评分阶段本身具备张量化执行基础，因此在本文设计中，其主要中间表示包括候选距离矩阵、局部排序结果、聚合统计量和最终异常分数。本文在这一位置的选择很明确：中间张量不在计算过程中回到 CPU，而是在 GPU 上连续推进，直到得到最终小规模输出后再按需回传主机。这样做的原因在于，一旦这些对象中途离开 GPU，不仅会增加 D2H 成本，还会破坏 PyTOD 在设备侧连续执行和批量算子复用的优势。预期收益因此不仅表现为传输减少，也表现为评分阶段的局部性和算子复用被更好地保留。

### 4.3.4 必要条件下的主机侧交互与页锁定内存机制

GPU 主导执行并不意味着 CPU 在单卡系统中完全消失。原始输入读取、部分日志写回、结果落盘以及特定部署场景下的主机接口，仍然要求系统与 CPU 保持交互。因此，这一小节对应的设计决策不是“取消主机交互”，而是把交互严格限制在必要边界，并通过页锁定内存等机制把这些边界开销控制在较低水平。RMM 文档指出，PinnedHostMemoryResource 可提供页锁定主机内存，从而加快 Host↔Device 数据传输<sup>[11]</sup>。这类机制的收益主要不是提高某个算子的峰值速度，而是降低必要交互对整条热路径的干扰。

因此，单卡热路径只保留必要的主机边界：查询向量、规范化特征、候选 ID/distance/rank 以及 PyTOD 的距离矩阵、排序和聚合张量尽量在 GPU 侧连续传递；原始输入、最终异常分数和运行日志则以小规模方式回到 CPU。这样既保留系统可观察性，也避免候选到评分之间重新形成主机侧折返。

## 4.4 异步 I/O—计算流水线融合

在上一节确立 GPU 主导执行链路之后，本节进一步处理另一类关键问题：如何把候选召回阶段内部的 I/O 等待压缩到最低，并使这种收益真正传导到系统整体。FlashANNS 的设计表明，传统外存驻留检索系统存在明显的串行等待特征；而通过依赖放松异步流水线（dependency-relaxed asynchronous pipeline），可以使 I/O 和 GPU 计算在更大程度上重叠，从而提升候选召回吞吐<sup>[7]</sup>。本文在单卡系统中继续把它与 GPU 主导执行链路衔接起来，使检索内部收益不会在后续阶段被新的边界开销抵消，避免只形成局部模块接入。

具体而言，系统在候选召回阶段不再将 SSD-I/O 视为必须先完成的前置步骤，而是把检索中的部分访问与 GPU 侧局部计算、候选维护和评分前准备并行组织。对检索内部而言，这意味着查询在等待部分图节点或向量块从 SSD 到达时，GPU

仍可继续推进已到达部分的距离计算和候选维护；对本文系统整体而言，这意味着候选召回阶段的等待不再以完整停顿的形式暴露在评分入口前，而是更多地被吸收到设备侧持续运行的流水线中。

这一融合的收益主要表现在两个层面。对 FlashANNS 内部而言，候选召回阶段的有效吞吐被提高，GPU 不再频繁因为等页到达而空转；对本文系统整体而言，PyTOD 评分模块不需要周期性等待检索阶段完全结束才能获得候选输入，能够更连续地获得候选输入。更重要的是，由于候选结果继续保留在 GPU 一侧，中间结果不必回到 CPU 侧整理，FlashANNS 内部获得的 I/O—计算重叠收益也不会后续阶段被新的边界阻塞抵消。

从实现角度看，这一融合要求系统在单卡场景下对流、缓冲区和中间结果表达进行统一组织，而 FlashANNS 对 warp 级并发 SSD 访问和双缓冲思想的使用，也支持这种边访问边计算的路径可以落到具体执行机制上的<sup>[7]</sup>。

图 4-3 展示了 FlashANNS 异步流水线与本文链路的融合方式。

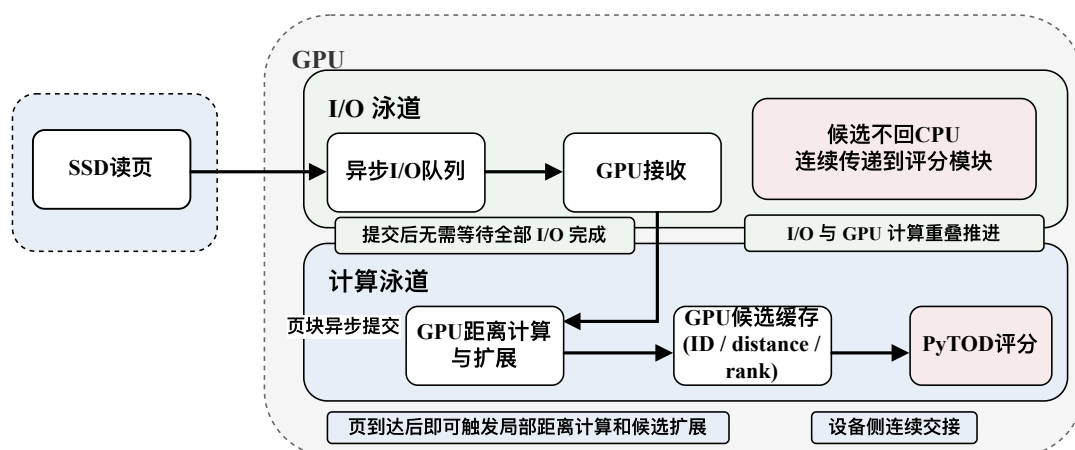


图 4-3 FlashANNS 异步 I/O—计算流水线与本文执行链路融合（本文根据文献<sup>[7]</sup>和系统实现整理）

这一链路把 SSD 读页等待、页到达后的局部计算和批次切换尽量交叠起来：异步 I/O 队列削弱等待，warp 级并发访问与双缓冲候选缓存维持设备侧连续推进，候选结果则直接进入评分模块，避免重回 CPU 打包。

## 4.5 GPU 数据处理与候选组织优化

在完成 GPU 主导执行链路和 FlashANNS 异步流水线融合之后，单卡系统仍需要进一步优化 GPU 数据处理与候选组织方式，以减少表格型中间结果处理和设备资源管理所带来的额外开销。为此，本文在原始 PyTorch/CPU 数据路径基础上引入 RAPIDS GPU 数据处理生态，对金融数据预处理、特征拼接、中间结果组织

和批量统计环节进行后端重构<sup>[10-11]</sup>。

后端优化围绕热路径展开：字段筛选、类型转换和窗口统计尽量前移到 RAPIDS/cuDF；候选组织、批量拼接和统计聚合转为 GPU DataFrame 或 CuPy 张量表达；临时内存与必要的主机交换缓冲由 RMM 内存池和页锁定主机端（pinned host）资源管理；与 PyTOD、检索后端交接时，则优先采用设备侧连续缓冲，避免“DataFrame—Python 对象—Tensor”的反复转换路径<sup>[11]</sup>。

由此，RAPIDS 在本文中并不是替代检测算法，而是用于稳定热路径的数据和内存后端。后端重构的直接目标，是缩短 query\_prep、pack\_candidates 以及检索资源初始化中的对象转换和资源抖动。

## 4.6 候选召回后端的初步验证与完整路径接入

除了数据处理后端的重构外，候选召回后端本身也需要进一步工程化衔接。虽然 FlashANNS 提供了高吞吐候选召回机制，但在实际系统实现中，检索后端不仅要关注检索内部的吞吐，还要关心查询输入、候选输出以及评分模块之间的接口组织是否会重新引入边界成本。在候选召回后端的实现演进中，本文首先利用 FAISS-GPU 较成熟的 GPU ANN API 完成候选召回链路的初步验证。该阶段主要用于确认 GPU 侧查询输入、候选 ID 与距离输出、设备指针路径和 PyTOD 候选评分接口之间的衔接关系。

随后，完整 FlashTOD 路径进一步以 FlashANNS 作为主要候选召回后端，以适应大规模外存驻留或超显存索引条件下的高吞吐召回需求。因此，FAISS-GPU 在本文中更接近初步验证和接口参照组件，而 FlashANNS 才是最终候选召回后端。本文在后端接入上的优化主要体现在三个方面：统一检索输入接口，使查询向量能够以设备侧对象直接进入候选召回后端；统一候选输出接口，使前期验证阶段 FAISS-GPU 返回的候选结果，以及完整路径中 FlashANNS 返回的候选结果，都能以 GPU 侧张量形式被 PyTOD 消费；统一资源管理策略，通过 FAISS 的 GpuResources 与系统侧缓冲区管理配合，为后续缓冲区申请、设备指针路径和延迟抖动分析提供工程参照<sup>[13]</sup>。这些优化的共同目标，是让候选召回后端更顺畅地嵌入整条热路径，避免单独追求某个后端组件的峰值性能。

## 4.7 单卡执行链路的工程增强

在完成数据路径、异步流水线和后端优化后，系统仍然需要若干工程增强机制来保证单卡链路在高负载下保持稳定。这些增强能够补足其稳定性与可观察性，包括异步 CUDA stream 组织、双缓冲与微批执行、显存碎片控制以及日志和性能

采样机制。

在流组织方面，本文通过将查询准备、候选检索、评分计算和必要的主机边界交互映射到不同 CUDA stream，使数据传输、检索和评分能够在更大程度上并行推进。在缓冲方面，本文采用双缓冲与微批组织方式：当当前批次进入候选召回与评分阶段时，下一批次的输入组织和部分准备可以并行进行，从而进一步压缩流水线空泡。在显存管理方面，本文通过限制中间张量生命周期、统一缓冲区尺寸以及使用内存池机制减少碎片和反复分配开销。在监控与可观察性方面，本文建立统一日志字段与性能采样机制，记录查询输入规模、候选集合规模、各阶段耗时、GPU 利用率和 Host↔Device 交互次数，为后续章节分析多 GPU 扩展提供统一观测基础。

## 4.8 单卡实验与性能分析

为了验证前述单卡系统设计与数据路径优化的有效性，本文将单卡实验口径统一组织为“真实集 → 1M 合成数据 → 10M 合成数据 → 50M 合成数据”。其中，真实或半真实数据主要用于确认链路可运行和检测效果不失真，1M、10M、50M 合成数据主要用于性能、吞吐和压力分析；二者不直接比较绝对吞吐。1M 合成数据承担由真实集过渡到系统实验的桥接角色，10M 与 50M 合成数据对应单卡性能分析的主体规模。在这一框架下，本节重点比较四类方案：原始分段执行链路、仅启用候选召回加速的链路、启用 GPU 主导执行链路但不做后端重构的链路，以及本文完整的单卡优化方案。这样的对比口径能够区分不同收益来源：候选召回本身带来的收益、执行链路重构带来的收益，以及后端与工程增强叠加后的整体收益。换言之，这四类方案构成递进式消融实验，用于观察候选召回、数据路径压缩和后端重构在端到端性能中的累积贡献。实验关注的指标包括每秒查询数（Queries Per Second, QPS）、平均延迟与第 99 百分位延迟（p99 延迟）、CPU↔GPU 传输次数与体积、GPU 利用率、I/O 利用率以及各阶段耗时分解。

实验结果显示，单纯依赖候选召回加速虽然能够降低部分检索耗时，但若候选结果仍需回到 CPU 侧整理再进入评分，端到端吞吐提升并不充分。相比之下，当查询特征、候选结果与评分中间张量更多地留在 GPU 一侧后，主机—设备边界交互次数明显减少，系统吞吐率和延迟分位数都表现出更稳定的改善。进一步地，当 FlashANNs 的异步 I/O—计算流水线与设备侧连续执行链路结合后，候选召回阶段的等待可以被部分隐藏，评分阶段也更容易连续接收输入，反映在更高的 GPU 利用率上。

从阶段耗时分解来看，优化前系统的主要开销集中在查询组织、候选结果主



机侧重组和阶段同步等待；优化后，系统性能不再首先被路径开销拖住，其决定因素开始更多落在 GPU 侧核心阶段，主要耗时更多地集中到设备侧检索与评分本身。

考虑到本节结果展示需要兼顾版面与代表性，图 4-4 与图 4-5 选取 1M 与 10M 两个具有代表性的合成规模展示阶段耗时分解，图 4-6 汇总比较四种方案在 QPS、p99 延迟、PCIe 传输次数与 GPU 利用率上的综合表现。

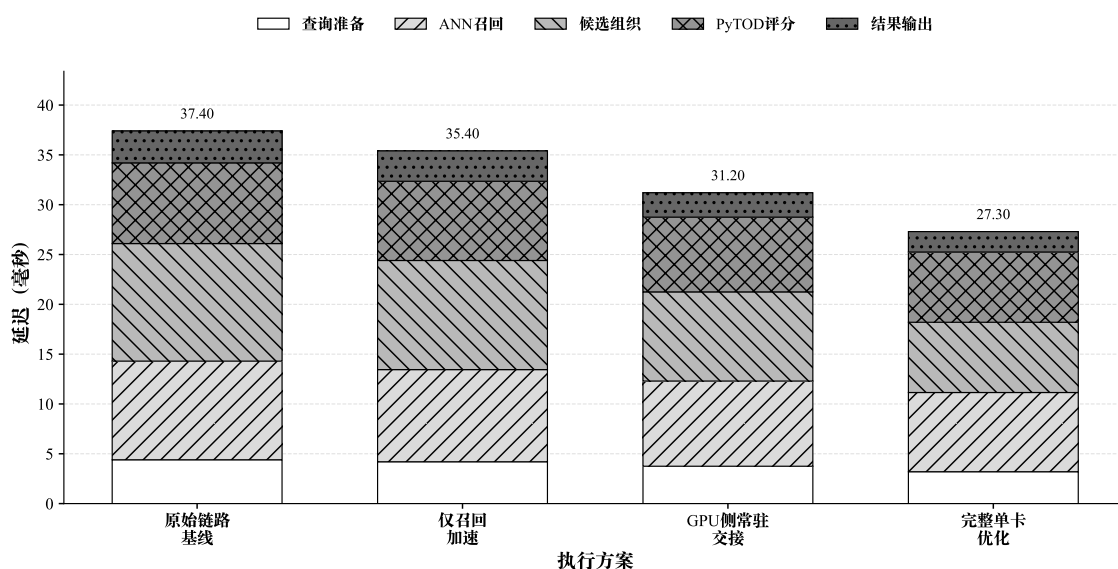


图 4-4 1M 数据规模下单卡各阶段耗时分解（实验数据绘制）

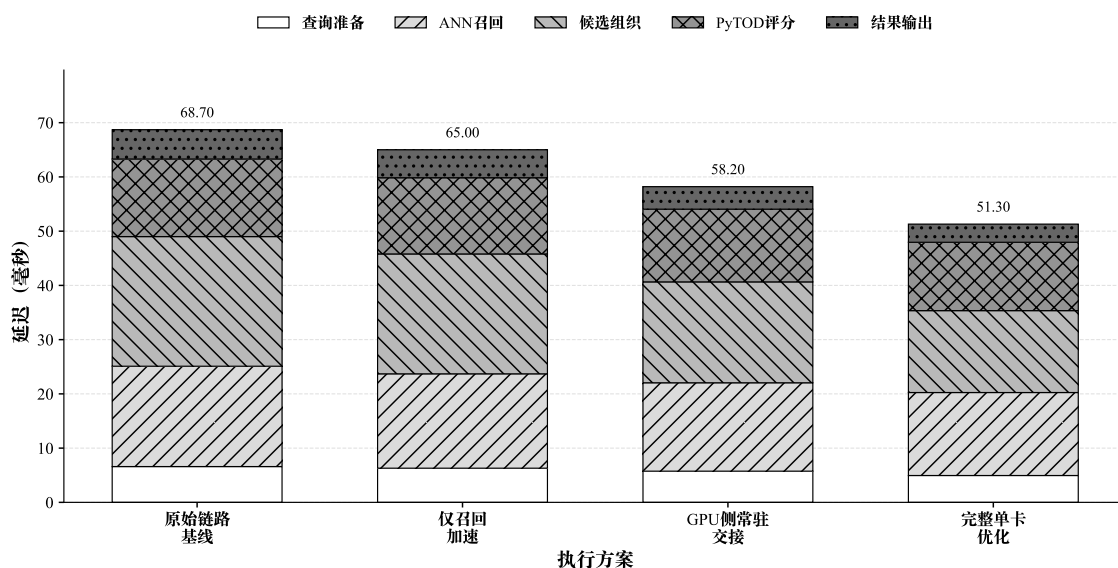


图 4-5 10M 数据规模下单卡各阶段耗时分解（实验数据绘制）

图 4-6 展示的是递进式消融结果，而不是完全独立的单因素消融。其目的在于从原始分段链路出发，逐步加入候选召回加速、GPU 主导执行链路和完整后端优

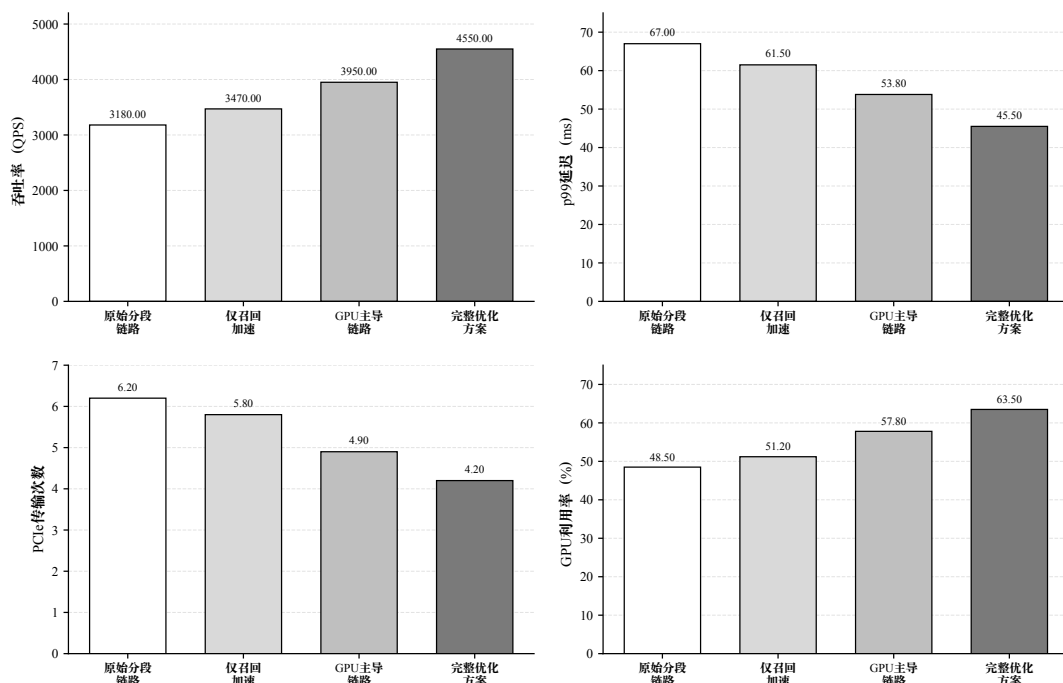


图 4-6 单卡执行链路递进式消融结果（实验数据绘制；该图为递进式消融，非完全独立的单因素消融）

化，以观察主要系统优化项对端到端吞吐、尾延迟、PCIe 传输次数和 GPU 利用率的累积影响。由于设备侧候选缓冲、异步 I/O—计算流水线、后端资源管理和评分张量组织之间存在耦合关系，本文不将其解释为严格单因素实验，而是将其作为单卡链路优化收益来源的递进式验证。

## 4.9 本章小结

本章围绕单 GPU 场景下的系统实现问题，分析了原始 FlashTOD 链路中存在的边界交换、候选重组、I/O 等待以及资源管理抖动等瓶颈，并在此基础上设计了 GPU 主导的端到端执行链路。本章进一步通过 RAPIDS 重构数据处理路径，基于 FAISS-GPU 完成候选召回接口的初步验证，并在完整链路中以 FlashANNs 作为主要候选召回后端，同时围绕设备侧候选组织、异步 I/O—计算流水线和 PyTOD 评分接口进行工程优化。

单卡实验的结果说明，本章的主要收益并不来自某一个局部算子的极限加速，而来自候选召回到异常评分整条数据路径的压缩，以及设备侧连续执行能力的增强。经过这一轮优化后，系统的主要压力已经更多集中到 GPU 侧核心阶段，这也使下一章有必要转向多 GPU 场景：当单卡链路已经被尽量收紧之后，系统吞吐的进一步提升才真正转化为多 GPU 扩展问题。

## 第五章 多 GPU 并行扩展与归并优化

### 5.1 多 GPU 扩展需求分析

第四章已经从单 GPU 视角完成了系统级优化，并初步说明在 GPU 主导端到端执行链路、异步 I/O—计算流水线和后端重构协同作用下，单卡场景中的主要损耗已不再集中在主机—设备边界，而更多落在设备侧检索与评分过程本身。然而，这并不意味着单卡已经足以覆盖后续所有实验和部署需求。随着输入规模继续增大、查询并发进一步提升、候选索引逼近单卡显存上限，单卡系统会同时遭遇三类上限：计算上限、容量上限和调度上限。

第一类需求来自计算上限。当候选召回和异常评分都已稳定驻留在 GPU 侧时，单卡吞吐最终会受限于单设备的计算资源。此时，即便继续压缩局部路径开销，系统整体 QPS 仍然难以随业务峰值线性提升。第二类需求来自显存与索引容量上限。在更大索引或更大规模合成扩展数据集条件下，单 GPU 可能无法同时容纳完整索引、副本缓存以及评分中间张量，系统必须在缩小索引规模、降低候选预算和增加设备数量之间进行权衡<sup>[30,41]</sup>。第三类需求则来自调度压力。金融近实时异常检测并不只追求平均吞吐，系统还需要在高峰时段保持稳定的延迟分位数和资源利用率；一旦请求在单设备上集中堆积，即使平均性能尚可，尾延迟也会迅速恶化。

因此，对本文系统而言，多 GPU 扩展的目标并不是抽象地提高并行度，而是在保持检测效果、延迟分位数和实现复杂度可控的前提下，解决三个问题：如何把更多查询稳定分摊到多张 GPU 上，如何在容量受限时继续维持索引可用性，以及如何避免调度与共享路径重新成为系统上限<sup>[30]</sup>。这些问题一旦进入多卡场景，局部结果如何归并、CPU 是否重新回到关键路径、共享存储路径是否发生拥塞，都会直接影响扩展收益是否能够兑现。

基于这一判断，本文后续优先采用索引复制（replicated）架构主方案，使每张 GPU 尽可能独立完成候选召回与异常评分；只有在索引容量或业务约束明确表明无法复制完整索引时，才进入设备侧跨卡归并与设备间通信的兜底路径。换言之，本章的关注点是在金融近实时异常检测场景下找到更符合吞吐优先和工程可控性的扩展方式，避免发散到构造复杂的多 GPU 检索系统。

## 5.2 多 GPU 扩展策略设计原则

多 GPU 扩展方案的设计必须建立在系统目标与瓶颈分析的基础上。结合第四章单卡结论与第二章相关工作梳理, 本文将多 GPU 扩展方案归纳为四项基本原则: 查询级并行优于单查询跨卡并行; 数据并行与索引复制优先于直接采用索引分片; CPU 保持在控制面; 当跨卡归并不可避免时, 优先采用 GPU 侧归并。这四项原则并非并列罗列, 其中前两项决定主方案形态, 后两项负责约束兜底路径和运行边界。

首先, 查询级并行优于单查询跨卡并行。若一个查询必须在多张 GPU 上分别完成局部搜索并在末端归并, 系统会额外承担局部结果表达、跨卡通信和同步等待等成本; 而当不同查询被均衡分发到不同 GPU 时, 每张设备就更容易复用单卡阶段已经压缩好的“候选召回—异常评分”闭环。cuVS 多 GPU 文档将索引复制 (replicated) 模式描述为通过在各 GPU 间分发查询以获得更高查询吞吐 (query throughput), 这一点与本文的吞吐目标一致<sup>[30]</sup>。

在此前提下, 第二项原则进一步限定了主方案选择: 数据并行与索引复制优先于一开始即采用索引分片。cuVS 的分布模式定义表明, 索引复制 (replicated) 更偏向吞吐优先, 分片 (sharded) 更偏向容量扩展<sup>[30,42]</sup>。因此, 只要索引仍可复制, 完整副本在每张 GPU 上独立服务本地查询, 通常比从一开始就引入跨卡依赖更符合本文的系统目标。

第三项原则主要约束控制边界, 即 CPU 保持在控制面而不重新回到结果处理主路径。cuVS 文档提示, 多 GPU ANN 的部分实现要求数据集、查询与输出数组位于主机内存 (host memory), 这意味着若直接沿用主机中心式 (host-centric) 默认路径, 则 CPU 与主机内存会重新进入关键路径<sup>[41]</sup>。基于这一认识, 本文在多 GPU 方案中将 CPU 定位为控制面调度者: 它负责请求队列管理、负载均衡、有界批处理 (bounded batching)、运行时监控和最终小规模结果汇总, 但不承担大规模候选集合或评分中间张量的重新组织。

最后一项原则只在主方案不能直接成立时才发挥作用, 即当必须跨卡归并时, 归并逻辑尽量下沉到 GPU 侧, 并通过 NCCL 等设备间通信机制完成。NCCL 文档指出其通信操作会异步入队到 CUDA stream 上执行, 但也警告在同一 ncclGroupStart/End() 中混用多个 stream 会形成全局同步点<sup>[14]</sup>。这意味着 GPU 侧归并虽然优于 CPU 聚合, 却仍然带来额外通信组织成本, 因此它在本文中是明确的兜底路径, 并不作为第二主线考虑。

表 5-1 对 FlashTOD 多 GPU 部署中的索引复制主方案、设备侧归并兜底路径以及主机侧聚合对比路径的适用边界进行了归纳。

表 5-1 FlashTOD 多 GPU 部署适用边界

| 条件                | replicated 主方案    | 设备侧归并                   | 主机侧聚合                    | 主要风险                         |
|-------------------|-------------------|-------------------------|--------------------------|------------------------------|
| 索引可完整复制，吞吐优先      | 每卡保留完整索引，按查询或微批分发 | 通常不需要启用，仅保留为容量受限兜底      | 仅用于调试或小规模对照              | 显存占用高，索引规模继续增大后可能失效          |
| 单卡无法容纳完整索引，需要容量扩展 | 不再直接适用            | 索引按卡分片，局部结果在 GPU 侧组织和归并 | 可作为实现简单的对照路径             | 跨卡通信、同步点和归并复杂度上升             |
| 原型验证、调试或离线分析      | 可用于简化路径验证         | 可与主机侧聚合形成对照             | 各 GPU 输出局部结果后回传 CPU 排序合并 | CPU 重新进入数据平面，D2H/H2D 与排序开销放大 |

该表用于说明不同多 GPU 路径的适用条件，而不是把三类方案视为并列的最终选择。本文当前结论限定在单机多 GPU、固定候选预算和固定硬件拓扑条件下：replicated 路径是吞吐优先主方案，设备侧归并是容量受限时的兜底路径，主机侧聚合主要用于形成可解释的对照基线。

### 5.3 数据并行与索引复制的多 GPU 执行架构

基于前述设计原则，本文将多 GPU 场景下的主扩展方案定义为数据并行与索引复制（replicated）架构。索引复制架构适用于索引仍可在单张 GPU 显存内复制、查询吞吐优先且希望复用完整单卡链路的场景；该路径主要提升吞吐，不提升单卡可容纳的索引容量。这意味着，只要完整索引仍能复制到每张 GPU，本文就更倾向于用显存换取更简单的数据路径和更稳定的查询闭环<sup>[30]</sup>。在该方案中，完整索引被复制到每张 GPU 上，每张 GPU 都具备完整的候选召回与异常评分能力；查询按批次或微批次被分发到不同 GPU，各设备在本地独立执行“查询向量组织—候选召回—异常评分—局部结果输出”全链路流程；CPU 主要承担请求级调度、批次分配和最终小规模结果汇总职责。

在执行流程上，数据并行与索引复制架构可以形式化表示为：给定总查询集合  $Q = \{q_1, q_2, \dots, q_m\}$ ，调度器将其划分为若干子批次  $\{B_1, B_2, \dots, B_g\}$ ，并将这些子批次分配给  $g$  张 GPU。每张 GPU  $G_j$  本地维护一份完整索引  $I_j = I$ ，对其收到的批次  $B_j$  独立执行

$$C_j = R(B_j, I_j), \quad S_j = F(B_j, C_j), \quad (5-1)$$

其中  $R$  表示候选召回， $F$  表示异常评分。由于每张 GPU 都能本地完成完整闭环，因此系统不需要在查询处理中途引入设备间依赖。

从系统工程角度看，索引复制虽然会增加显存占用，但换来了更低的执行复杂度 and 更清晰的数据路径。其优势主要体现在三个方面：查询之间天然独立，便于按请求或微批次做吞吐扩展；每张 GPU 都能沿用第四章已经优化好的本地执行链路，不需要为多卡场景重新拆解候选召回与评分接口；系统只需在极小规模结果层面做收口，而不需要在处理过程中持续维护跨卡依赖。综合以上原因考虑，索引复制在本文中始终被定义为主方案，而不是若干方案中的普通选项。

图 5-1 给出了索引复制主方案的总体结构，直观展示了 CPU 控制面与 GPU 本地闭环在多卡场景下的分工关系。

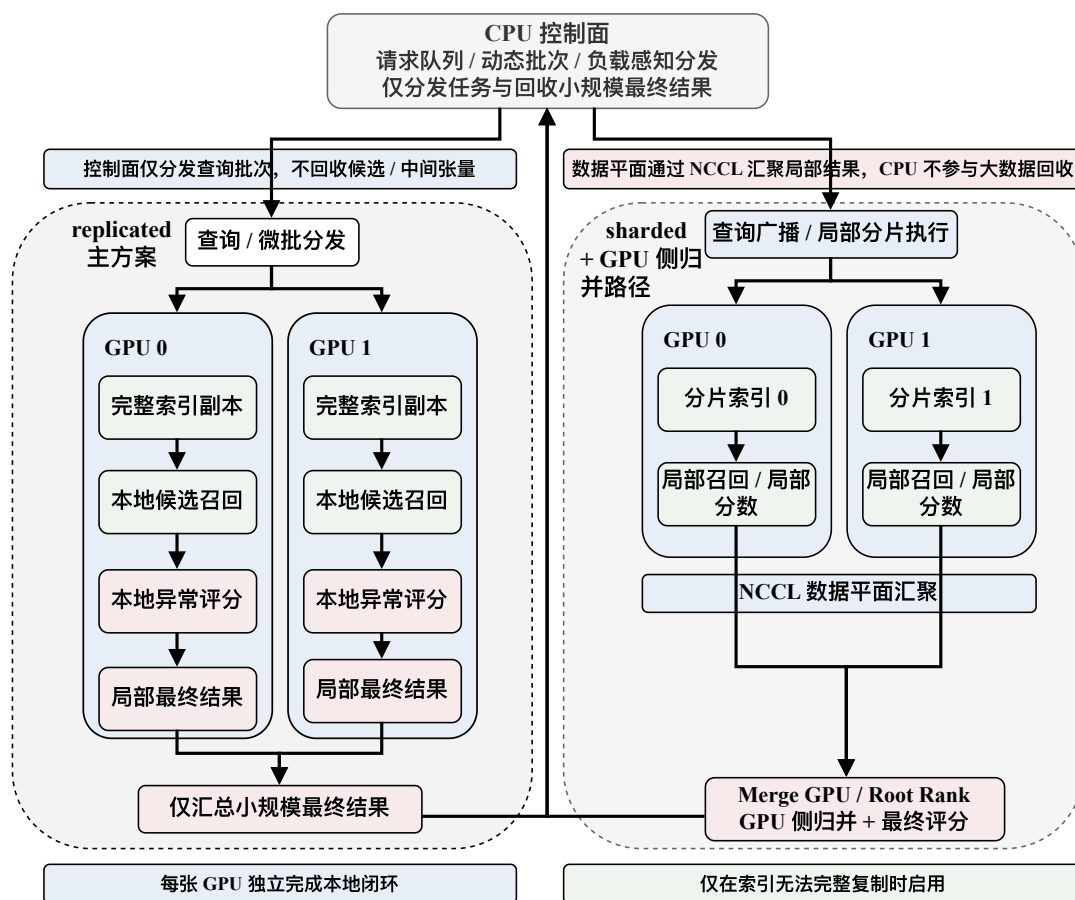


图 5-1 数据并行与索引复制多 GPU 架构（本文绘制）

## 5.4 CPU 控制面调度与负载均衡

在索引复制（replicated）架构下，CPU 的职责需要被严格限定。本节所讨论的控制面工作主要包括请求队列维护、微批形成、按 GPU 负载分发批次、控制 bounded batching 策略，以及在各 GPU 完成本地闭环之后接收最终小规模输出。

请求队列组织方面，系统采用统一入口队列接收查询请求，并根据队列长度、GPU 空闲度和微批策略动态形成批次。低时延推理系统和可预测深度学习服务系统的研究都表明，面向吞吐与尾延迟的批处理策略，需要在设备利用率和排队时间之间做持续折中；cuVS 动态批处理文档则为本文的多 GPU 实现提供了更接近工程接口层的参考<sup>[43-45]</sup>。对本文而言，微批并不是单纯为了“批更大”，而是为了在吞吐和尾延迟之间建立一个可调的控制旋钮：批次过小会浪费设备并行能力，批次过大又会推高排队等待。

负载均衡方面，本文在索引复制架构下优先采用查询级动态调度而非静态平均分配。cuVS 在索引复制模式下提供了 LOAD\_BALANCER 与 ROUND\_ROBIN 两类搜索模式，其中前者用于保持各 GPU 负载均衡<sup>[42]</sup>。这类调度能够防止某张 GPU 因瞬时队列堆积而拉高整机的第 99 百分位延迟（p99 延迟），避免只在形式上平均分配请求。

最终结果汇总方面，CPU 仅接收每张 GPU 已完成的最终小规模结果，如异常分数、Top-k 排序结果和必要日志元信息。局部候选集合、评分中间张量以及归并前的设备侧结构都不应回流到 CPU。

图 5-2 展示了 CPU 控制面的调度与回收流程，控制面与数据平面的职责边界将在正文中结合各组件的关键参数分别说明。

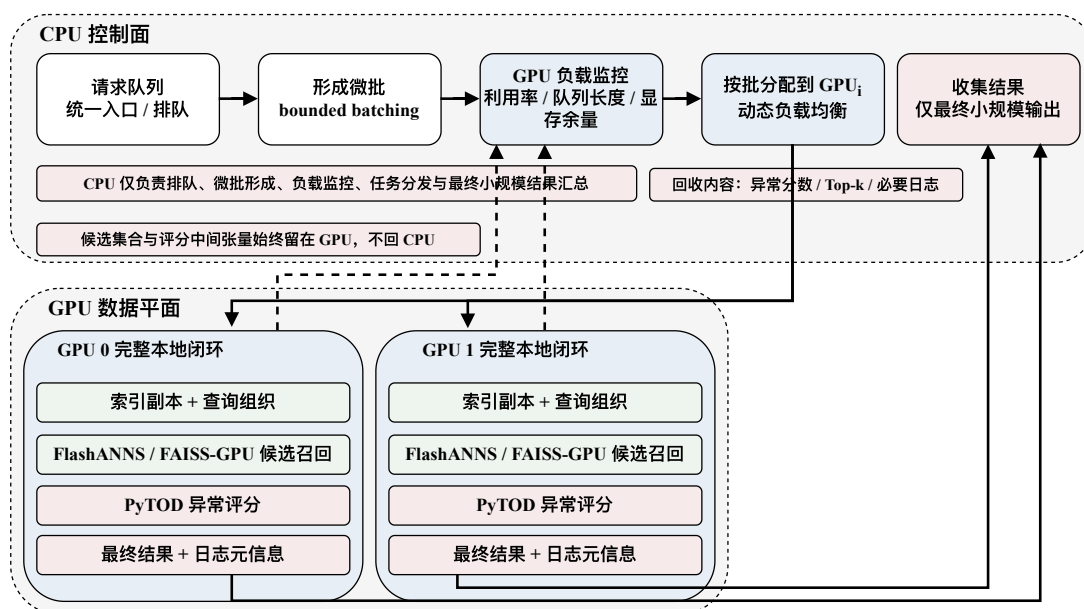


图 5-2 CPU 控制面调度与微批示意（本文绘制）

图中 FAISS-GPU 表示前期候选召回接口验证路径；多 GPU 完整路径下的本地候选召回以后续接入的 FlashANNs 为主。

## 5.5 跨卡归并场景下的 GPU 侧归并与 NCCL 通信

当索引容量超过单卡显存可容纳范围，或业务约束要求单一查询必须访问分布在多张 GPU 上的局部索引时，索引复制主方案就不再直接适用，系统必须进入跨卡归并场景。本文把这一场景明确界定为兜底路径：只有在主方案因容量或部署条件失效时，才引入分片索引与跨卡结果合并。因为一旦进入归并路径，系统就必须额外承担局部结果表达、设备间通信和流协调等成本。

在此条件下，本文仍坚持一个约束：局部结果尽量在 GPU 侧组织与合并，CPU 只接收最终极小规模结果。cuVS 在分片模式下给出了 MERGE\_ON\_ROOT\_RANK 与 TREE\_MERGE 两类归并方式，为本文设计设备侧归并路径提供了参考<sup>[42]</sup>。与主机侧聚合路径相比，设备侧归并路径的主要收益在于减少大规模结果回传和主机侧排序开销，使跨卡合并仍尽可能延续设备侧处理逻辑；但它并不会像索引复制主方案那样天然简单，因为额外的通信与同步组织无法完全省去。

在设备间通信实现方面，本文采用 NCCL 作为主要通信机制，并将通信流与局部计算流明确分离。NCCL 文档指出其通信操作异步入队到指定 CUDA stream，但在同一 ncclGroupStart/End() 组内混用多个 stream 会形成全局同步点<sup>[14]</sup>。因此，GPU 侧归并的收益建立在一个前提上：通信必须被控制在必要范围内，并与本地计算流合理解耦。若通信组织不当，兜底方案很容易因同步点过多而侵蚀掉原本希望保留的扩展收益。

## 5.6 拓扑感知 I/O 通道绑定与准入控制

在多 GPU 异常检测系统中，扩展上限不仅由 GPU 数量决定，还受制于存储路径和 PCIe/NUMA 拓扑。单卡阶段被压缩掉的路径开销，在多卡环境下可能以另一种形式重新出现：如果多张 GPU 共享同一 SSD 通道或 PCIe 路径，局部检索和候选读取虽然仍在设备侧执行，但共享链路上的排队会直接反映到吞吐下降和 p99 延迟上升。GDS 概览文档指出，GDS 通过在 GPU 显存与存储设备之间建立直接内存访问（Direct Memory Access, DMA）路径来避免 CPU bounce buffer，并强调拓扑路径会影响传输效率<sup>[31]</sup>。基于这一认识，本文引入拓扑感知 I/O 通道（I/O lane）绑定与准入控制机制：运行前分析 GPU、SSD、PCIe 根复合体（PCIe root complex）、PCIe 交换机（PCIe switch）与 NUMA 节点之间的关系；运行时尽量将 GPU 与 SSD 通道进行绑定，并对每条 I/O 通道的在途请求（in-flight requests）数做有界控制，以避免过度并发导致尾延迟失控。

图 5-3 以左右对比方式展示了拓扑感知 I/O 通道在合理绑定与共享冲突两种条件下的 GPU、SSD、PCIe 与 NUMA 关系，为后续扩展结果中的路径差异提供解



释依据。

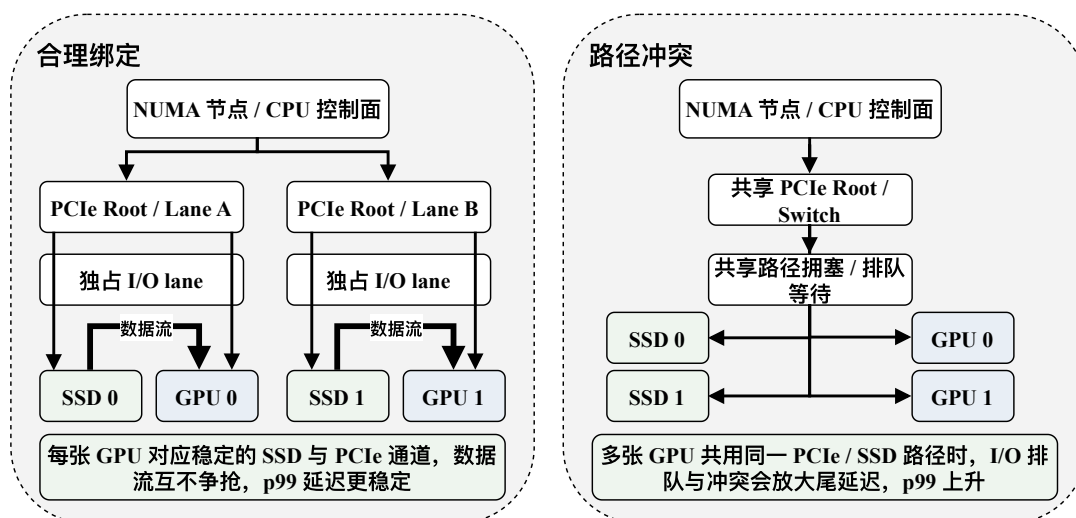


图 5-3 拓扑感知 I/O 通道示意（本文绘制）

## 5.7 多 GPU 运行时支撑

多 GPU 场景下，系统还需要运行时支撑机制保证运行稳定性与实验可重复性，包括多进程/多 GPU 运行时组织、统一监控指标、异常恢复与请求重试、以及资源隔离和日志管理。本节为索引复制主方案与设备侧归并兜底路径提供统一的运行时支撑，使后续实验能够在可比口径下观测吞吐、延迟、CPU/GPU 利用率以及 I/O 路径占用。

## 5.8 多 GPU 扩展实验与结果分析

多 GPU 实验沿“真实集 → 1M 合成数据 → 10M 合成数据 → 50M 合成数据 → 100M 合成数据”逐级组织，并围绕索引复制、设备侧归并、主机侧聚合与共享路径冲突 (shared-path conflict) 四类路径展开；对应配置列入附录 B 补充表 B-2。图 5-4 汇总展示索引复制路径的扩展曲线与共享路径下的 p99 延迟变化，图 5-5 展示不同归并路径下的延迟变化，图 5-6 用于比较 I/O 通道配置对扩展性的影响，表 5-2 对总体结果进行了汇总。

从结果上看，索引复制方案在 1/2/4 GPU 条件下保持了较为稳定的 QPS 增长和相对平稳的延迟变化，说明在索引可复制的前提下，查询级并行和本地闭环执行更符合本文的吞吐优先目标。与之对应，本地评分 + 设备侧归并和本地评分 + 主机侧聚合的对比主要用于回答兜底路径如何选的问题：两者都建立在索引分片的前提下，但在本文实验口径下，设备侧归并的平均延迟、p99 延迟和 CPU 利用

率低于主机侧聚合，说明一旦必须跨卡归并，把局部结果留在设备侧处理更有利于控制路径开销。

另一方面，拓扑感知绑定（Topology-aware binding）与共享路径冲突（Shared-path conflict）的对比表明，即便计算资源足够，当多张 GPU 共享同一 PCIe/SSD 路径时，QPS 仍会明显下降，p99 延迟也会迅速抬升。这意味着，多 GPU 扩展的实际表现并不只由归并方式决定，还受到共享路径组织方式的直接限制。

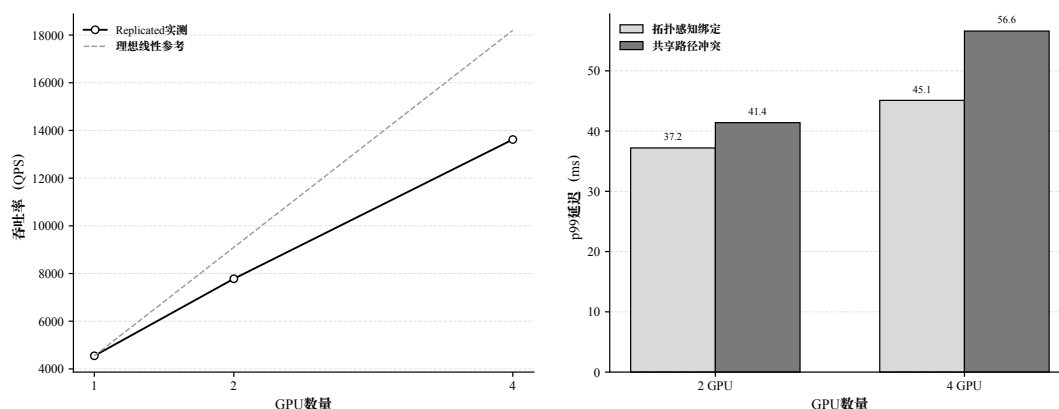


图 5-4 多 GPU 扩展趋势与共享路径瓶颈分析（实验数据绘制）

图 5-4 左图给出了索引复制路径下 QPS 随 GPU 数量增加的变化，并加入理想线性扩展参考线；该参考线仅由 1 GPU QPS 线性外推得到，是理论参考而非实测结果。实际曲线与理想线性之间存在差距，说明 CPU 控制面调度、查询分发、设备侧资源竞争和 I/O 路径都会削弱扩展效率。右图比较了拓扑感知绑定与共享路径冲突下的 p99 延迟，结果显示共享 PCIe/SSD 路径会放大尾延迟，因此拓扑感知 I/O 通道绑定并非附属优化，而是在多 GPU 场景下维持稳定扩展的重要条件。对于索引无法完整复制的容量受限场景，仍需结合图 5-5 比较设备侧归并与主机侧聚合：前者把局部结果尽量留在设备侧，后者需要回到主机侧排序与聚合，因此会引入更高的路径开销。

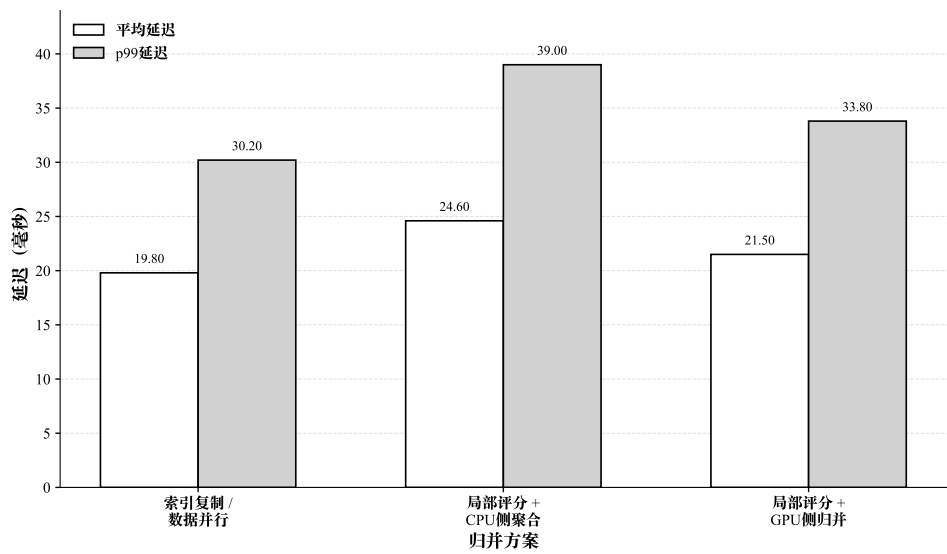


图 5-5 平均延迟与 p99 延迟变化（实验数据绘制）

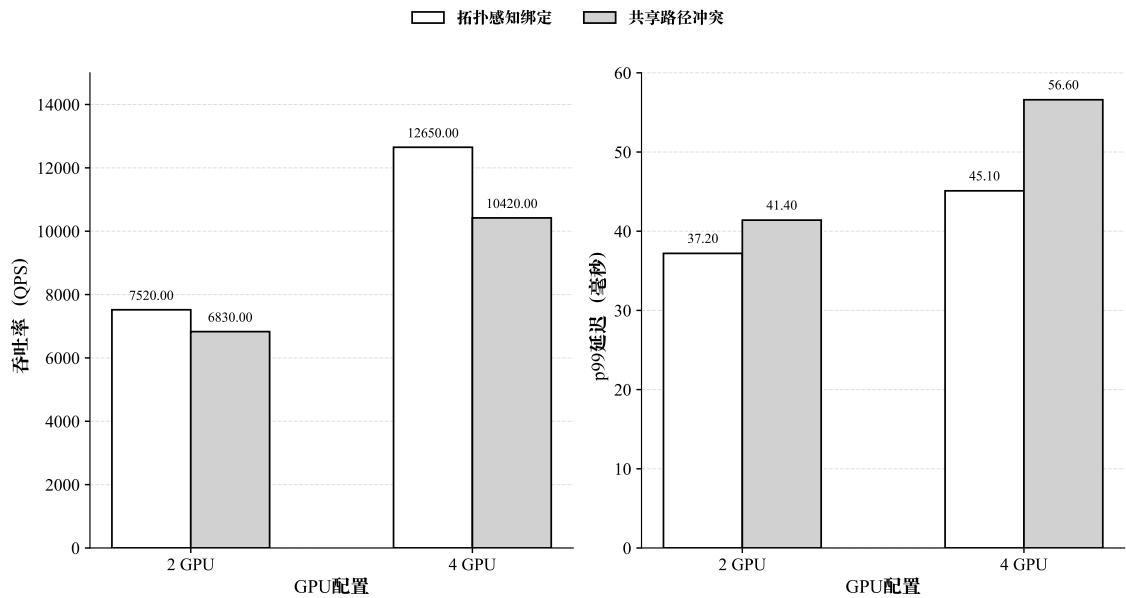


图 5-6 I/O 通道配置对扩展性的影响（实验数据绘制）

表 5-2 多 GPU 扩展结果汇总

| GPU 数 | 方案                     | 数据规模 | QPS      | avg latency (ms) | p99 latency (ms) | CPU 利用率 (%) | GPU 利用率 (%) |
|-------|------------------------|------|----------|------------------|------------------|-------------|-------------|
| 1     | Replicated             | 10M  | 4550.00  | 22.80            | 34.80            | 47.00       | 63.50       |
| 2     | Replicated             | 10M  | 7780.00  | 21.00            | 31.90            | 42.00       | 68.80       |
| 4     | Replicated             | 10M  | 13620.00 | 19.80            | 30.20            | 37.80       | 73.50       |
| 4     | 本地评分 + 设备侧归并           | 100M | 11380.00 | 21.50            | 33.80            | 41.50       | 69.80       |
| 4     | 本地评分 + 主机侧聚合           | 100M | 9580.00  | 24.60            | 39.00            | 48.50       | 62.50       |
| 2     | Topology-aware binding | 100M | 7520.00  | 23.40            | 37.20            | 43.80       | 67.50       |
| 4     | Topology-aware binding | 100M | 12650.00 | 22.10            | 45.10            | 39.80       | 72.20       |
| 2     | Shared-path conflict   | 100M | 6830.00  | 25.10            | 41.40            | 45.80       | 61.80       |
| 4     | Shared-path conflict   | 100M | 10420.00 | 28.90            | 56.60            | 44.50       | 56.50       |

注：结果展示遵循与附录 B 中补充表 B-2 相同的多 GPU 统一规模口径。受版面与结果表达重点限制，表中列出的是 10M 索引复制扩展结果和 100M 压力场景结果，用于回答主方案、兜底路径与共享路径冲突三类设计问题。

## 5.9 本章小结

本章的工作主要形成了三项进入后续综合实验的结构性结论。第一，在索引可复制的前提下，索引复制主方案能够最直接地复用第四章已经优化好的单卡执行链路，因此适合吞吐优先场景。第二，当容量约束迫使系统进入分片路径时，设备侧归并相比主机侧聚合更能保留设备侧处理优势，但其收益仍然受通信与同步组织方式影响。第三，多 GPU 扩展的上限不仅取决于计算资源，还取决于 I/O 路径与拓扑关系能否支撑多卡并发。

这些结论在本章内完成了初步验证，但其适用边界还需要在下一章与单卡优化结果、不同数据规模表现和整体检测效果一起统一考察。因此，第六章将不再分别讨论单卡与多卡的局部机制，而是把本章得到的主方案、兜底路径和路径冲突基线放回到综合实验框架中，继续验证其在整体系统层面的有效性与边界。

## 第六章 综合实验设计与结果分析

### 6.1 实验环境与软硬件配置

本章的作用是把前文提出的系统路线放到统一实验框架中检验：FlashTOD 是否在真实或半真实金融数据上保持可用，单卡优化是否确实压缩了数据路径，多 GPU 扩展是否把吞吐提升建立在合理的结构选择之上，以及 I/O 路径与归并方式在何种条件下开始成为新的限制因素。围绕这些问题，本节先交代实验所依赖的硬件条件、软件栈和实验分类口径。

硬件环境方面，实验平台由多张同型号 GPU、主机 CPU、内存、SSD 以及对应 PCIe 拓扑组成。单卡实验使用 1 张 GPU 运行完整的候选召回与异常评分链路，主要观察第四章提出的数据路径优化、异步流组织、RAPIDS 后端重构、FAISS-GPU 前期接口验证经验和 FlashANNS 主召回后端接入在单设备上的综合表现；多卡实验使用 2 张及以上 GPU，主要对应第五章中的索引复制（replicated）扩展、控制面调度、设备侧归并和 I/O 通道控制。由于本文后续还需要讨论拓扑差异对扩展结果的影响，GPU 型号、显存容量、CPU 型号、主机内存、SSD 容量、PCIe 代际和 NUMA 关系都需要作为实验前提显式给出。

软件环境方面，实验系统由 PyTOD 评分实现、FlashANNS 候选召回模块、FAISS-GPU 初步 ANN 接口验证组件、RAPIDS 数据处理与资源管理组件、多 GPU 通信和调度运行时共同组成。FAISS-GPU 主要用于验证候选召回接口、设备侧输入输出路径和候选结果格式，完整 FlashTOD 路径中的大规模候选召回以后续接入的 FlashANNS 为主。为了保证结果可复核，本文固定 Python、PyTorch、PyTOD、FAISS-GPU、CUDA、NCCL 以及操作系统版本；CuPy、cuDF、cuML、RMM 等 GPU 数据处理与资源管理组件的版本也一并记录为统一软件环境快照。

在实验分类上，本文将综合实验划分为三组：方法有效性验证实验，对应真实或半真实数据及 1M 合成桥接口径；单卡系统性能实验，规模口径统一按“真实集 → 1M 合成数据 → 10M 合成数据 → 50M 合成数据”组织；多 GPU 扩展实验，规模口径统一按“真实集 → 1M 合成数据 → 10M 合成数据 → 50M 合成数据 → 100M 合成数据”组织，用于比较索引复制主方案、设备侧归并兜底路径以及 I/O 路径组织方式。三类实验共享一套基础代码框架，但在数据规模、硬件使用方式和评价重点上各有分工。

表 6-1 给出了实验平台与软件环境配置快照，图 6-1 则补充展示了实验平台中

CPU、GPU、SSD 与 PCIe/NUMA 的连接关系，为后续性能差异解释提供硬件背景。

表 6-1 实验平台与软件环境配置

| 项目            | 规格                                                                      | 说明                                          |
|---------------|-------------------------------------------------------------------------|---------------------------------------------|
| <b>硬件平台配置</b> |                                                                         |                                             |
| CPU 平台        | 2 × Intel Xeon Gold 6348 @ 2.60GHz                                      | 56 核 112 线程，双 NUMA                          |
| 主存容量          | 503 GiB                                                                 | 系统总内存                                       |
| GPU 平台        | 8 × NVIDIA GeForce RTX 3090                                             | 24 GB/卡，总显存 192 GB                          |
| PCIe 能力       | PCIe Gen4 ×16                                                           | 8 张 GPU 均为 Gen4 ×16                         |
| 本地存储          | 3 × NVMe SSD                                                            | 894.3 GB × 2, 3.5 TB × 1, 总约 5.29 TB        |
| NUMA 拓扑       | 2 个 NUMA 节点                                                             | NUMA 0 对应 GPU0–GPU3, NUMA 1 对应 GPU4–GPU7    |
| GPU 互连特征      | 同侧 PIX/PXB, 跨侧 SYS                                                      | 用于解释多 GPU 与 I/O 通道实验                        |
| 网络接口          | 2 × Mellanox NIC                                                        | mlx5_0, mlx5_1                              |
| <b>软件环境配置</b> |                                                                         |                                             |
| 操作系统          | Ubuntu 20.04.4 LTS, Linux 5.13.0-30-generic                             | 实验运行环境                                      |
| Python / CUDA | Python 3.11.14; CUDA Runtime 12.4                                       | 基础运行时环境                                     |
| 深度学习框架        | PyTorch 2.5.0+cu124; NCCL 2.21.5                                        | GPU 张量计算与通信环境                               |
| 驱动环境          | NVIDIA Driver 550.120                                                   | 与 CUDA 12.4 对应                              |
| 异常检测框架        | PyTOD (yzhao062/pytod)                                                  | GitHub 主仓库默认分支 main                         |
| ANN 检索组件      | FlashANNS / ann_search (Tinker/ann_search)                              | GitHub 主仓库默认分支 main                         |
| 其他核心库         | FAISS 1.13.2; PyOD 2.0.6; NumPy 2.2.6; SciPy 1.13.1; scikit-learn 1.6.1 | 实验所用主要 Python 依赖; FAISS 用于初步 ANN 接口验证和对照实现  |
| RAPIDS 相关     | CuPy 13.3.0; cuDF 25.02.00; cuML 25.02.00; RMM 25.02.00                 | 用于 GPU 侧表格处理、中间结果组织、数组计算和内存资源管理，不单独归因整体性能收益 |

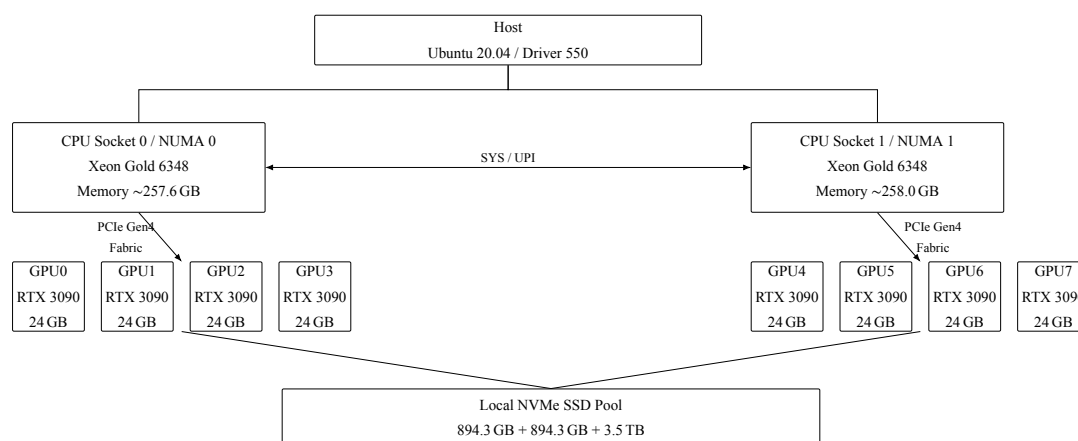


图 6-1 实验平台拓扑示意图（本文根据实验平台配置绘制）

## 6.2 数据集与任务设置

本章沿用第三章的数据安排，但实验目的已经从建立系统对象转向统一验证系统结论，不同规模数据在这里承担的角色也更明确。真实或半真实金融数据对应全文的现实口径，1M 合成数据承担桥接层，单卡实验沿“真实集 → 1M 合成数据 → 10M 合成数据 → 50M 合成数据”逐层展开，多 GPU 实验则继续延伸到 100M 压力场景。

需要说明的是，本章中的真实或半真实金融数据并不等同于金融机构生产系统中的在线原始流量，而是指经过公开发布、脱敏、匿名化、采样或特征变换后的金融相关任务数据。它们适合用于检验异常检测链路在真实业务语境下的可用性；10M、50M、100M 等 AMLSim / IBM AML 合成规模则主要服务于系统压力、吞吐扩展和 I/O 路径验证，相关结论应理解为在本文实验平台和生成口径下的工程趋势。

在有效性验证部分，本文继续使用 IEEE-CIS Fraud Detection、Credit Card Fraud Detection 和 DGraphFin 这三类具有代表性的公开金融数据，分别覆盖结构化支付风控、经典信用卡欺诈以及图金融数据场景。具体数据来源和字段特征已在第三章中给出，这里不再重复展开。

在系统性能与扩展实验部分，本文采用第三章已经介绍过的高保真合成金融交易数据。通过规模分层让不同规模分别承担不同验证任务，其中，1M 数据用于连接真实数据验证与系统实验，使检测效果和端到端实现处于同一实验口径；10M 数据是单卡综合性能与多 GPU 扩展的共同起点；50M 数据主要用于观察规模继续增大后单卡链路与多 GPU 扩展的中间趋势；100M 数据则主要服务于多 GPU 扩展、跨卡归并和共享路径冲突实验。

在任务组织上，本章实验可分为四类：候选召回与异常评分有效性任务、单卡

性能任务、多 GPU 索引复制扩展任务以及跨卡归并与 I/O 路径任务。这些任务共享同一套核心实现，但比较对象和观测重点并不相同。有效性任务更关注检测结果与候选排序质量；单卡性能任务更关注数据路径压缩后的吞吐、延迟和利用率；多 GPU 任务则进一步考察索引复制主方案、设备侧归并兜底路径以及共享路径冲突对扩展收益的影响。

在数据使用方式上，有效性验证任务按训练集、验证集和测试集划分，以保证 PR-AUC、Recall@ $k$  等指标可比；性能与扩展实验则采用固定索引集与固定查询集，将查询批量持续注入系统，以更准确地衡量 QPS、延迟分位数与资源利用率。这样区分能够避免把“检测效果评估”和“系统压测”混在同一套数据口径下讨论。

图 6-2 展示了规模层级与实验类型的映射。

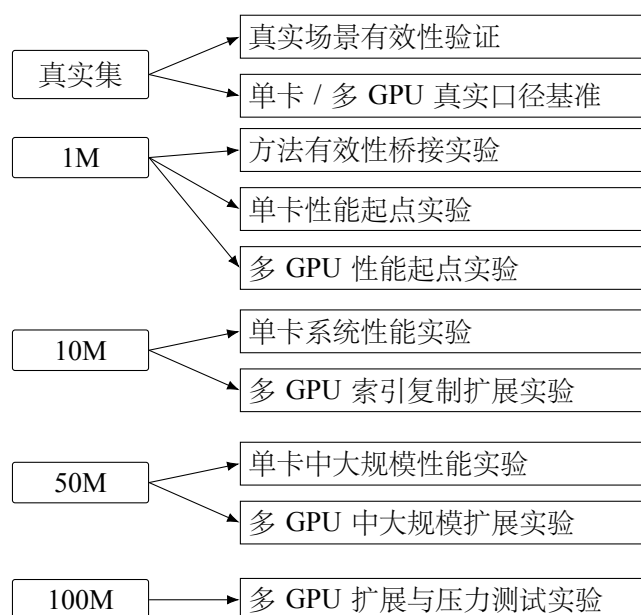


图 6-2 数据规模层级与实验类型关系图（本文绘制）

本章据此区分效果评估和系统压测：真实集与 1M 桥接实验沿用固定随机种子的训练/验证/测试划分，用于比较 PR-AUC、Recall@ $k$  与 Top- $k$  overlap；10M、50M、100M 性能实验则固定索引集和查询集，持续注入请求以观察 QPS、p99 延迟和资源利用率。

### 6.3 评价指标与比较原则

本章采用的指标体系服务于全文的四条核心判断：FlashTOD 是否在效果上可接受，单卡优化是否主要压缩了数据路径，多 GPU 索引复制是否真正提升了吞吐，以及 I/O 路径是否决定扩展上限。围绕这四类判断，指标被组织为四组：异常检测效果指标、候选检索质量指标、系统吞吐与延迟指标以及资源利用率指标。



其中,受试者工作特征曲线下面积(Receiver Operating Characteristic Area Under Curve, ROC-AUC)、PR-AUC 与 Recall@ $k$  用于回答“候选召回 + 张量化评分”是否仍保持了合理的检测能力; Recall@ $k$  与 Top- $k$  overlap 用于观察候选召回是否改变了最终排序口径; QPS、平均延迟与 p95/p99 延迟直接对应系统在单卡和多卡场景中的端到端收益; GPU 利用率、CPU 占用率、显存使用率、I/O 带宽利用率以及 CPU↔GPU 传输开销占比,则用于解释这些收益来自哪里。

比较原则同样围绕这一逻辑展开。首先,同一类实验尽量共享相同数据集、向量表示和硬件配置,以避免数据口径与平台差异掩盖系统设计差异。其次,在比较不同召回配置时,控制候选集合规模或召回质量处于同一数量级,以保证检测效果与吞吐变化具有可解释性。再次,在多 GPU 实验中固定软件环境与相近查询到达方式,使索引复制、设备侧归并、主机侧聚合和共享路径冲突基线之间的差异尽量来自扩展策略本身,避免外部扰动影响。

为了回应不同层次的实验问题,本文将对对比方案组织为从效果上界、模块基线到完整系统方案的递进关系,如表 6-2 所示。PyTOD 全量评分主要用于提供小规模条件下的检测效果上界和全量评分基线;仅召回方案用于判断候选召回本身是否足以完成异常检测;初步融合方案用于刻画 FAISS-GPU ANN API 与 PyTOD 简单串接时的系统表现;完整融合方案 FlashTOD 则对应以 FlashANNS 为候选召回后端,并经过数据路径组织和后端优化后的完整实现。多 GPU 场景下,主机侧聚合与设备侧归并分别作为两类跨卡归并对照,用于分析跨卡结果组织带来的额外代价。

其中,初步融合方案对应 FAISS-GPU ANN API 与 PyTOD 的简单串接,用于验证候选召回与候选集评分链路的功能可行性;完整融合方案 FlashTOD 则对应以 FlashANNS 为候选召回后端,并结合 GPU 主导数据路径和设备侧候选组织后的完整实现。

需要说明的是,表 6-2 中的基线主要用于刻画 FlashTOD 内部不同执行路径之间的差异,包括全量评分、仅召回、简单串接、完整链路以及不同多 GPU 归并路径。本文尚未在相同硬件、相同候选预算和相同 PyTOD 评分主干下系统覆盖 HNSW、IVF/IVF-PQ、DiskANN、PyOD CPU 实现等外部后端或系统级基线,因此后续结果不解释为对所有候选召回系统或异常检测框架的全面优势比较。

在统计口径上,当前实验结果基于固定硬件平台、固定索引集、固定查询集、固定候选预算和固定批大小下的稳定运行配置。由于送审前实验时间有限,本文尚未对所有配置给出完整误差棒、标准差和显著性检验,因此正文图表主要展示代表性结果和趋势性比较,相关结论也限定在本文设定的数据规模、硬件平台和

表 6-2 对比方案与基线设置

| 方案                 | 候选召回                                | PyTOD 评分       | 数据路径                                 | 对比目的                        |
|--------------------|-------------------------------------|----------------|--------------------------------------|-----------------------------|
| PyTOD 全量评分         | 不使用                                 | 全量样本评分         | 以 PyTOD 原始评分链路为主                     | 作为小规模条件下的检测效果上界和全量评分基线      |
| 仅召回方案              | FAISS-GPU<br>ANN API / 初步<br>ANN 召回 | 不进入完整评分<br>模块  | 只观察候选集合与排序结果                         | 验证仅依靠召回是否不足以替代异常评分          |
| 初步融合方案             | FAISS-GPU<br>ANN API                | 候选集上评分         | 成熟 API 简单串接, 候选组织和阶段边界未充分优化          | 验证 ANN 候选召回 + PyTOD 评分链路可行性 |
| 完整融合方案<br>FlashTOD | FlashANNS 候选<br>召回                  | 候选集上评分         | FlashANNS 后端 + GPU<br>主导链路 + 设备侧候选组织 | 验证本文完整系统设计的端到端收益            |
| 主机侧聚合              | 多 GPU 本地<br>FlashANNS 召回            | 多 GPU 本地评<br>分 | 跨卡结果回到 CPU 侧排序与聚合                    | 作为主机侧归并路径的对照                |
| 设备侧归并              | 多 GPU 本地<br>FlashANNS 召回            | 多 GPU 本地评<br>分 | 结果尽量留在 GPU 侧并借助通信库归并                 | 评估设备侧归并相对主机侧聚合的收益和代价        |

实现路径内。后续若进一步完善实验, 应在附录或实验日志中补充各配置的重复运行记录、标准差范围和必要的统计检验; 在此之前, 本文不将当前差异解释为统计显著结论。

本章后续补充或重绘的图优先使用现有图表、绘图脚本和 LaTeX 表格中已经报告过的实现数据; 对于仅作为理论参照的曲线, 正文会明确说明其不属于实测结果。若后续引入保守预期数据或占位数据, 应在送审前尽量以重复实验日志替换。

图 6-3 给出了指标体系结构。

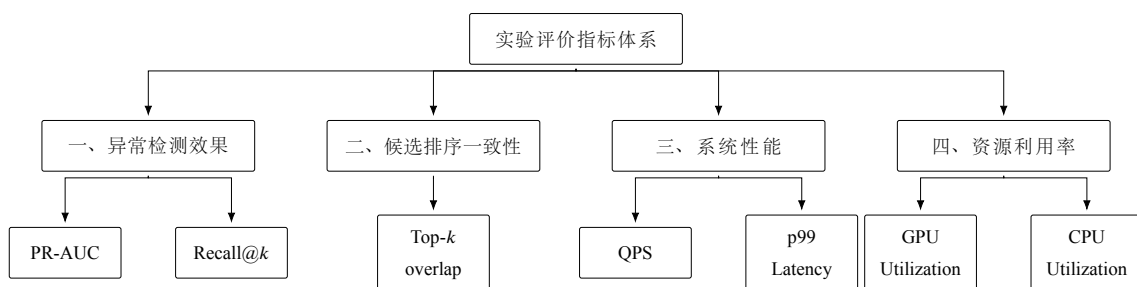


图 6-3 指标体系结构图 (本文绘制)

比较时, 同类实验固定数据源、标签口径、有效性划分、索引集和查询集; 不同方案共享候选预算、批大小、硬件平台以及 CUDA、PyTorch、NCCL 等基础软件栈。其中, FAISS-GPU 主要用于初步 ANN 召回和接口验证方案, 完整 FlashTOD 方案以 FlashANNS 作为候选召回后端; RAPIDS 主要用于 GPU 侧数据资源和资源

管理支撑。单卡与多 GPU 仅在相同规模下比较，并统一统计口径；部分配置尚未覆盖完整重复运行和显著性检验，相关差异主要用于趋势性分析。

## 6.4 核心结果展示

本节围绕四个需要被验证的问题展开：FlashTOD 是否总体有效；单卡优化和多 GPU 扩展分别带来了什么收益；多 GPU 主方案与兜底路径的边界如何理解；规模继续增大后系统是否仍保持可用。各小节先给出比较对象和待回答的问题，再引出对应图表。

### 6.4.1 FlashTOD 与基线系统的总体对比

这一组结果回答“FlashTOD 是否具备系统比较价值”。比较对象包括 PyTOD 全量评分、仅召回方案、初步融合方案以及完整融合方案。若完整融合在 PR-AUC 或 Recall@ $k$  上与全量评分保持接近，同时在 QPS 上表现更好，就可以说明“候选召回压缩评分范围 + 候选集上张量化评分”这一结构在本文实验口径下具有实际意义。

### 6.4.2 单卡优化与多 GPU 扩展性能对比

第二组结果回答“单卡优化与多 GPU 扩展各自改变了什么”。比较对象包括原始分段链路、完整单卡优化、2 GPU 索引复制扩展以及 4 GPU 索引复制扩展。这里希望区分两类收益来源：单卡优化主要压缩候选召回到异常评分之间的数据路径和尾延迟，多 GPU 扩展则在此前基础上继续抬高系统吞吐上限，并缓解 CPU 压力。

### 6.4.3 多 GPU 扩展结果对比

第三组结果回答“第五章中的结构选择是否成立”。比较对象不再只是 GPU 数量不同的索引复制方案，还包括在容量受限场景下的设备侧归并与主机侧聚合。需要验证的是：索引复制是否可以作为优先主方案；当主方案不能直接成立时，设备侧归并是否比主机侧聚合带来更低的路径开销；这些差异是否反映在延迟分位数、CPU 利用率和吞吐变化上。

### 6.4.4 不同数据规模下的系统表现对比

第四组结果回答“规模继续增大之后，系统还能否保持前述判断”。比较对象覆盖 1M、10M、50M、100M 四个规模层级，观察 QPS、第 99 百分位延迟（p99 延迟）和 PR-AUC 如何随规模变化。需要说明的是，PyTOD 全量评分可以作为 1M

量级下的效果上界参考，但并不适合作为 10M 以上规模的统一对照，因此这一组跨规模结果采用“初步融合方案”和“完整融合方案”作为可扩展口径下的比较对象。这里更关注的是，在外存驻留路径和多 GPU 扩展逐渐进入关键位置后，系统性能与检测效果是否同时出现可解释的退化：吞吐是否按预期下降，p99 延迟是否继续抬升，PR-AUC 和候选覆盖类指标是否在 10M 之后出现更明显回落，以及 I/O 路径是否开始主导上限。

为便于综合展示，表 6-3 汇总了核心结果，图 6-4 从检测效果和吞吐率两个维度同时比较 FlashTOD 与基线方案，图 6-5 与图 6-6 分别补充展示吞吐能力和效果-性能折中关系，图 6-7 汇总单卡优化与多 GPU 扩展的收益差异，而图 6-8 则展示跨规模趋势。

表 6-3 核心结果汇总

| 结果主题           | 代表方案            | 核心指标                              | 关键结果                     | 支撑图表               |
|----------------|-----------------|-----------------------------------|--------------------------|--------------------|
| FlashTOD 总体有效性 | 完整融合            | PR-AUC, QPS                       | PR-AUC 接近全量评分，QPS 提升     | 图 6-4-图 6-6        |
| 单卡链路优化效果       | 单卡优化            | QPS, p99 Latency, CPU Utilization | p99 延迟和 CPU 负载下降，端到端吞吐提升 | 图 4-4-图 4-6, 图 6-7 |
| 多 GPU 索引复制扩展性  | 2/4 GPU 索引复制    | QPS, CPU Utilization              | QPS 随卡数提升，CPU 利用率未同步放大   | 图 5-4, 图 6-7       |
| 跨卡归并策略差异       | 设备侧归并 vs. 主机侧聚合 | avg/p99 Latency                   | 设备侧归并的平均延迟和 p99 延迟更低     | 图 5-5, 表 5-1       |
| 跨规模可行性         | 完整融合            | QPS, p99 Latency, PR-AUC          | 规模增大后性能退化可解释，完整融合仍优于初步融合 | 图 6-8              |

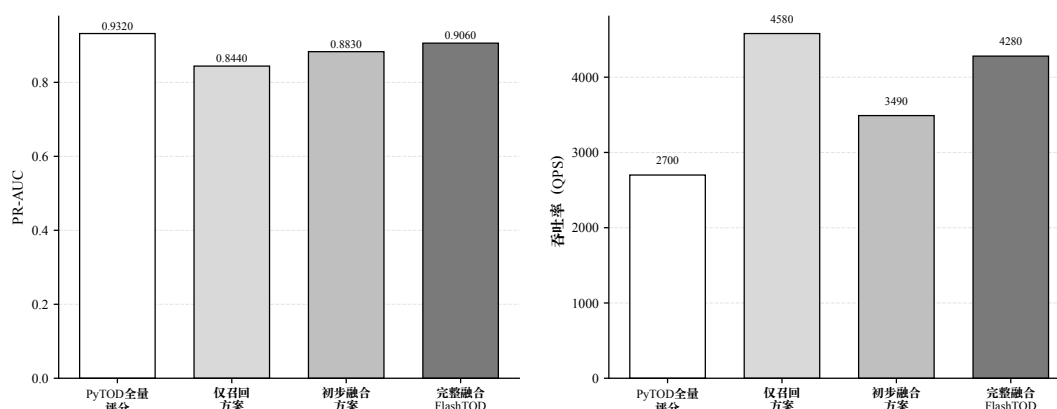


图 6-4 不同基线方案下检测效果与吞吐率对比（实验数据绘制）

图 6-4 用于同时观察不同系统配置在检测效果和吞吐率上的差异。PyTOD 全

量评分提供小规模条件下的效果参考，但其全量评分成本较高，并不适合作为大规模吞吐实验的统一对照；仅召回方案虽然吞吐较高，但 PR-AUC 低于带有异常评分的方案，说明候选召回本身不能替代评分模块；初步融合方案介于仅召回和完整方案之间，而完整融合方案 FlashTOD 在 PR-AUC 与 QPS 之间取得更均衡的表现，体现了候选召回、张量化评分和数据路径组织共同作用后的系统收益。

其中，初步融合方案对应 FAISS-GPU ANN API 与 PyTOD 的简单串接，用于验证候选召回与候选集评分链路的功能可行性；完整融合方案 FlashTOD 则对应以 FlashANNs 为候选召回后端、并结合 GPU 主导数据路径和设备侧候选组织后的完整实现。

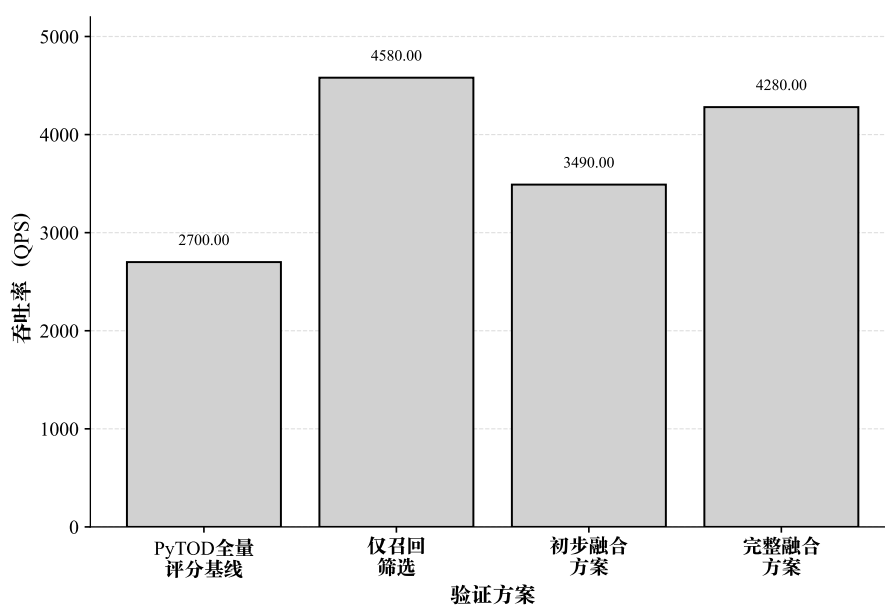


图 6-5 FlashTOD 与基线系统吞吐对比（实验数据绘制）

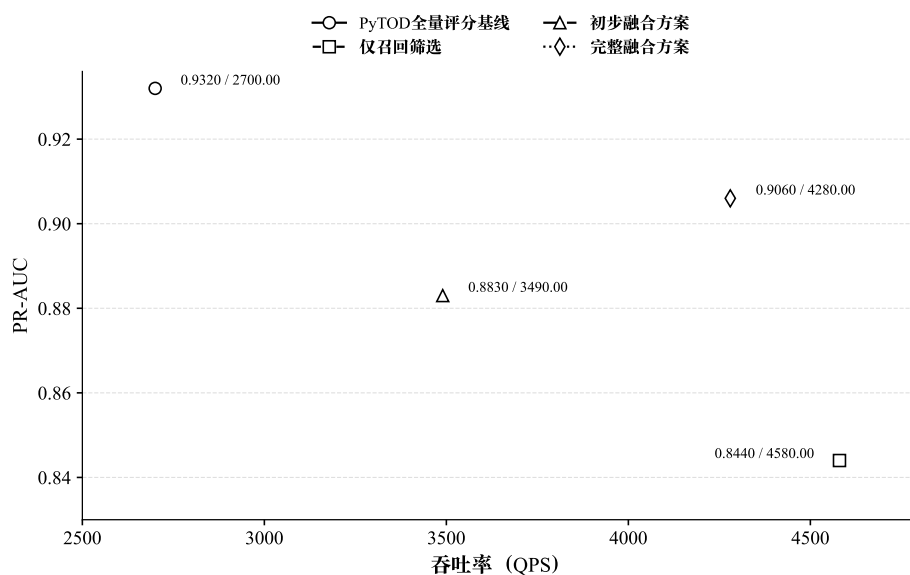


图 6-6 FlashTOD 效果与性能折中 (实验数据绘制)

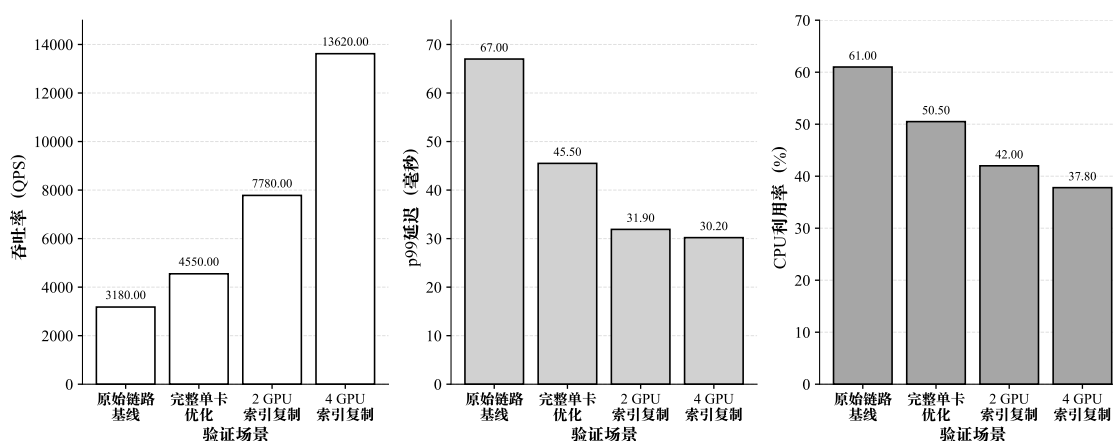


图 6-7 单卡优化与多 GPU 扩展收益对比 (实验数据绘制)

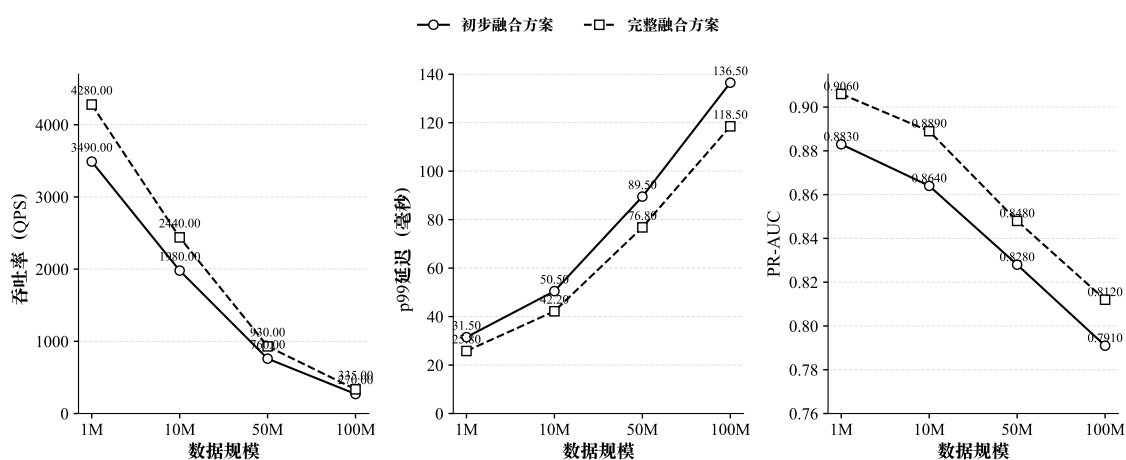


图 6-8 不同规模下系统表现趋势 (实验数据绘制)

## 6.5 结果讨论

从综合结果看，本文形成的四条主结论之间并不是彼此独立的。FlashTOD 之所以成立，并不仅仅因为它“比基线快”，而在于它在效果和吞吐之间建立了更稳定的平衡关系：候选召回没有明显破坏异常评分模块的判别能力，同时又把后续评分范围控制在系统可承受的预算内。对金融异常检测系统来说，这一点比单纯追求某个离线指标更重要，因为它决定了系统有机会进入真实高并发场景。

为了更清楚地解释不同优化项的收益来源，可以将端到端时间分解为

$$T_{\text{total}} = T_{\text{prep}} + T_{\text{retrieve}} + T_{\text{pack}} + T_{\text{score}} + T_{\text{merge}} + T_{\text{io}}.$$

其中， $T_{\text{prep}}$  表示查询准备时间， $T_{\text{retrieve}}$  表示候选召回时间， $T_{\text{pack}}$  表示候选组织与张量化时间， $T_{\text{score}}$  表示 PyTOD 评分时间， $T_{\text{merge}}$  表示多 GPU 归并时间， $T_{\text{io}}$  表示外存访问与主机—设备传输开销。按这一分解看，单卡优化主要降低  $T_{\text{pack}}$  和  $T_{\text{io}}$ ，异步 I/O 流水线主要压缩  $T_{\text{retrieve}}$  中的等待成分；多 GPU 索引复制则通过查询级并行降低单位查询分摊的检索和评分成本，而设备侧归并或主机侧聚合会额外引入  $T_{\text{merge}}$ 。

单卡优化的意义也不只是数值上的提速。第四章和本章结果共同表明，系统收益主要来自数据路径被压缩之后，候选召回与异常评分之间的边界成本下降，CPU↔GPU 传输次数与阶段同步等待被压到更低水平。这意味着，原先限制系统的并非某一个算子绝对不够快，而是模块之间的折返路径过长。当这一点被解决之后，GPU 侧检索和评分才真正成为决定性能的主要阶段。

多 GPU 结果则进一步说明，第五章给出的结构判断是必要的，而不是形式上的方案区分。索引复制能够成为主方案，并不是因为它在所有条件下都最节省资源，而是因为它最符合吞吐优先场景下的路径组织方式：查询分发简单，单卡链路可以直接复用，CPU 不需要重新承担大规模中间结果处理。当容量约束使索引复制不再可行时，在本文 100M 规模多 GPU 归并实验中，设备侧归并在平均延迟和 p99 延迟上低于主机侧聚合；但它仍会引入额外跨卡通信和设备侧归并开销，因此只应被视为容量受限场景下代价更高的兜底路径。

同时，多 GPU QPS 随 GPU 数量增加而提升，并不意味着系统获得了理想线性加速。原因在于 CPU 控制面仍需承担请求分发和最终小规模结果汇总，多 GPU 并发访问共享 PCIe/SSD 路径时会放大 I/O 竞争，设备侧归并或主机侧聚合也会分别带来跨卡通信和主机侧排序开销。因此，本文更稳妥地将相关结果解释为“具有吞吐扩展能力”，而不是把索引复制或归并方案描述为理想线性扩展。

最后，I/O 路径对扩展上限的影响在综合实验中表现得比单卡阶段更直接。随着规模继续增大，系统瓶颈会逐步从计算资源扩展到共享存储路径与拓扑组织上。

这一点也解释了为何本章在扩展收益之外，还必须单独比较拓扑感知绑定与共享路径冲突两组结果。

## 6.6 局限性分析

尽管本文在 FlashTOD、单卡数据路径优化与多 GPU 扩展方面给出了较完整的实验链路，但结论仍需放在明确边界内理解。

首先，超大规模实验主要依赖 IBM AML / AMLSim 等高保真合成数据。这类数据能够较好支撑系统吞吐、路径压力和扩展性验证，但仍不能完全替代真实金融机构中的在线负载和数据噪声结构。因此，本章关于 10M、50M 和 100M 规模的结论，主要用于工程可行性分析，而不是对所有真实业务场景的直接外推。

其次，PyTOD 全量评分在大规模数据上的计算和中间结果存储成本较高，因此本文只能在较小规模或桥接口径下将其作为效果上界和全量评分基线。对于 10M 以上规模，正文更多采用初步融合方案、完整融合方案以及多 GPU 归并路径进行比较，这能够回答系统扩展问题，但不能完全替代所有大规模全量评分基线。

再次，索引复制 (replicated) 架构的优势建立在索引仍可复制的条件上。一旦索引规模继续增大，或候选预算和中间张量占用进一步上升，系统就必须转向分片 (sharded) 路径，并接受更高的通信与归并复杂度。这意味着本文对多 GPU 主方案的判断带有明确的适用前提，而不是无条件成立。

此外，外存驻留路径与拓扑感知 I/O 优化对部署条件依赖较强，其收益与 GPU-SSD-PCIe-NUMA 拓扑、驱动与 GDS 支持高度相关。同一套方法在不同硬件节点上的收益幅度可能存在明显波动，因此本章关于 I/O 路径的结论更强调方向正确，而不意味着具体数值在所有平台上都可等比例复现。

同时，本文正文图表尚未系统覆盖所有配置下的重复运行误差范围和显著性检验。现有结果能够支撑趋势性比较和工程可行性分析，但若进一步面向生产级部署评估，还需要补充更多随机种子、更多查询到达模式和更完整的统计重复实验。

**统计稳定性与重复运行限制。**当前实验结果主要来自固定硬件、固定查询集、固定候选预算和固定批处理参数下的稳定运行记录。受实验周期限制，正文尚未对每一种规模、每一类归并路径和每一个候选预算组合提供完整重复轮次、均值、标准差、误差棒或显著性检验。因此，本文对实验结果的解释聚焦于趋势性差异和工程可行性，不将当前图表中的数值差异直接解释为统计显著结论。

**外部基线覆盖范围限制。**本文基线主要围绕 FlashTOD 内部路径展开，用于比较全量评分、仅召回、FAISS-GPU 初步串接、FlashANNS 完整链路以及不同多 GPU 归



并策略之间的差异。尚未在相同数据、相同候选预算和相同评分主干下系统纳入 HNSW、IVF/IVF-PQ、DiskANN、PyOD CPU 实现等外部方案，因此本文不声称对全部 ANN 后端或异常检测系统形成全面优势。后续工作可按表 6-4 所示口径补充外部基线。

表 6-4 后续外部基线补充计划

| 后续基线       | 主要比较对象         | 需要固定的实验口径                     | 预期回答的问题                      |
|------------|----------------|-------------------------------|------------------------------|
| HNSW       | FlashANNS 候选召回 | 数据规模、候选预算、Recall@k、PyTOD 评分主干 | 内存图索引在相同评分链路下的候选召回代价         |
| IVF/IVF-PQ | FlashANNS 候选召回 | 聚类参数、候选预算、召回质量、批大小            | 量化或聚类索引与外存候选召回的效果-性能折中       |
| DiskANN    | FlashANNS 外存路径 | SSD 路径、I/O 并发、候选预算、查询集        | 外存 ANN 后端在相同数据路径约束下的吞吐与尾延迟差异 |
| PyOD CPU   | PyTOD 评分模块     | 数据划分、评分函数、样本规模、候选集合           | CPU 传统实现与 GPU 张量化评分主干的可部署性边界 |

最后，本文的实验范围仍然限制在单机多 GPU 场景，多机分布式环境中的跨机通信、分布式索引管理和更复杂的调度问题尚未进入验证范围。这一限制并不影响本文对单机大规模异常检测系统的结论，但确实限定了结论的外推边界。

由此，本文结论应理解为单机多 GPU 条件下的工程验证：索引复制主方案仍受显存与索引规模约束，外存驻留和拓扑感知 I/O 的收益依赖具体硬件路径与 GDS 支持，超大规模真实金融负载验证仍有限。多机分布式场景中的跨机通信、分布式索引和集群调度尚未纳入本文范围，需要在后续工作中继续验证。

## 6.7 本章小结

本章的综合实验把全文前几章分散建立的判断收束到同一验证框架中。在本文设定的数据规模、硬件平台和候选预算条件下，FlashTOD 在保持合理检测效果的同时，体现出面向大规模金融异常检测的系统组织价值；单卡优化的主要收益来自候选召回到异常评分之间的数据路径压缩；多 GPU 条件下，索引复制可作为吞吐优先场景的主方案，而设备侧归并与拓扑感知 I/O 控制则主要用于容量受限或路径受限场景下维持扩展效果。

## 第七章 全文总结与展望

### 7.1 全文总结

本文围绕金融大规模数据异常检测系统的实现与优化展开研究。面对传统异常检测系统在大规模场景下暴露出的异常评分复杂度高、候选召回吞吐不足、主机—设备边界交换频繁以及多 GPU 扩展效率受限等问题，论文把研究重点放在系统组织方式上，而不是新的异常检测理论模型上。围绕这一边界，全文尝试将候选召回、异常评分、数据路径优化与多 GPU 扩展放入同一框架中考察，并据此验证其在金融近实时异常检测场景中的工程可行性。

在系统架构层面，论文以 PyTOD 为异常评分模块、以 FlashANNS 为候选召回前端，构建了 FlashTOD。这样组织的目的，是把候选召回和异常评分纳入同一条执行链路，使系统能够先在大规模样本中缩小评分范围，再在候选集合上执行张量化异常评分，从而缓解全量评分带来的时间与空间压力。

围绕这一执行链路，论文进一步完成了数据准备与任务建模工作。针对金融交易数据、账户行为数据和图金融数据的差异，本文对输入形式、向量化表示、候选召回目标和异常评分目标进行了统一建模，并形成了按规模逐层推进的数据组织方案：真实或半真实数据用于有效性验证，1M 合成数据承担桥接层，单卡实验沿 1M、10M、50M 合成规模展开，多 GPU 扩展继续延伸到 100M 压力场景。借助这些数据组织方式，全文把真实口径、单卡性能口径和多 GPU 扩展口径纳入同一实验框架。

在系统实现层面，单 GPU 场景的工作重点放在数据路径压缩上。论文围绕 GPU 主导的端到端执行链路，对查询特征组织、候选结果传递和评分中间张量的驻留方式进行了重新安排，并通过页锁定内存、RAPIDS 后端重构和 FAISS-GPU 前期接口验证降低阶段边界上的重复搬运与同步等待；完整路径中的候选召回后端则以 FlashANNS 为主。在这一基础上，多 GPU 部分继续沿用吞吐优先的思路，选择数据并行与索引复制作为主扩展方案，使每张 GPU 尽可能独立完成本地候选召回与异常评分；当索引复制不可行时，再以设备侧局部结果组织、NCCL 通信和设备侧最终归并作为兜底路径，并结合拓扑感知的 I/O 通道绑定与准入控制减轻共享路径冲突。

本文前期利用 FAISS-GPU 较完善的 ANN API 完成候选召回接口和 PyTOD 评分衔接验证，并在完整 FlashTOD 路径中以 FlashANNS 作为主要候选召回后

端。该处理使 FAISS-GPU 的作用限定在工程验证和接口参照层面，避免将其与 FlashANNS 的大规模外存候选召回职责混同。

综合实验部分则把前述系统设计放回统一验证框架中。在本文设定的数据规模、硬件平台和候选预算条件下，结果显示 FlashTOD 在真实数据口径和 1M 合成桥接口径上能够保持合理的检测效果；在更大规模的高保真合成数据上，单卡优化的主要收益来自候选召回到异常评分之间的数据路径压缩，而多 GPU 数据并行与索引复制方案表现出一定的吞吐扩展能力。随着规模继续增大，I/O 路径与拓扑关系对扩展上限的影响也变得更加明显。

需要强调的是，本文实验主要验证 FlashTOD 相对于内部递进式基线和不同归并路径的工程收益，不声称对所有候选召回后端或异常检测系统形成全面优势。

综合来看，本文的主要研究结论与技术贡献可以归纳为三点：

1. FlashTOD 将高吞吐候选召回与异常评分组织到同一执行链路中，使系统从全量高复杂度评分转向候选缩减后的精排评分路径，并在本文实验口径下取得更稳妥的效果与效率平衡。
2. 单 GPU 场景下，系统性能的主要限制并不来自个别算子的绝对速度，而更多来自数据路径与阶段同步成本；通过 GPU 主导执行链路、RAPIDS 后端重构、FAISS-GPU 初步验证经验和 FlashANNS 主召回后端接入，可以更稳定地改善端到端吞吐与延迟表现。
3. 多 GPU 场景中，数据并行与索引复制可作为吞吐优先条件下的主扩展方案；当索引无法复制时，GPU 侧归并相比 CPU 聚合更有利于减少主机侧路径开销，而扩展收益能否真正保留下来，还取决于存储拓扑与 I/O 路径组织方式。

## 7.2 未来工作展望

本文的工作主要集中在单机环境下金融大规模异常检测系统的实现与优化。沿着这一主线继续向前推进，仍有若干方向值得进一步深入。它们并非对现有工作的简单延伸，而是源于本文已经触及但尚未展开的研究边界。

此外，后续可进一步补充 FAISS-GPU、HNSW、IVF/IVF-PQ、DiskANN 与 PyOD CPU 实现等外部基线，在相同数据、相同候选预算和相同 PyTOD 评分主干下开展系统性对比，从而更细致地刻画候选召回后端选择对 FlashTOD 执行链路的影响。

### 7.2.1 面向更真实金融场景的数据验证

本文已经在真实或半真实数据上验证了方法可用性，也借助高保真合成数据完成了大规模系统实验，但更接近生产环境的金融业务数据仍然有限。在严格遵

守隐私与合规要求的前提下，后续若能接入更复杂分布、更强业务噪声和更明显概念漂移条件下的数据，将有助于进一步检验 FlashTOD 在真实业务约束下的稳定性，也能更准确地评估系统结论向生产场景外推的边界。

### 7.2.2 面向多机场景的分布式扩展

本文重点解决的是单机多 GPU 条件下的系统路径组织，多机场景中的跨节点通信、远程结果归并和分布式索引管理尚未进入论文范围。如果继续沿用控制面与数据平面分离的思路，把它扩展到多节点环境，就有机会进一步回答更大规模集群中的查询分发、跨节点索引一致性以及层次化归并问题，这将决定本文方案能否从单机扩展走向更大规模部署。

### 7.2.3 面向更深层存储路径的优化

本文已经讨论了外存驻留路径、拓扑感知 I/O 控制和 GDS 相关约束，但对更深层的存储访问优化仍然展开得不够。后续若进一步引入 GPUDirect Storage、RDMA 等技术，并把查询调度与存储访问策略做更细粒度的联动，就有可能继续减少主机参与和拷贝代价，从而在更大规模数据和更复杂路径条件下保留更多系统收益。

### 7.2.4 面向图异常检测与时序异常检测的统一扩展

本文已经把图金融数据纳入统一向量接口，但现有系统主干仍主要围绕表格型异常检测与候选召回—评分链路展开。后续若在保持总体架构稳定的前提下，引入面向图异常检测和时序异常检测的表示方式，例如图嵌入、图神经网络、时序编码或基于事件流的特征聚合机制，就有机会把当前系统从可兼容多类输入进一步推进到对不同异常类型具备更强适配性的统一平台。

### 7.2.5 面向国产 GPU 平台的验证与部署

本文当前实验与实现主要建立在现有 NVIDIA GPU 软件栈之上，而国产 GPU 平台在算子支持、向量检索实现、设备间通信机制和存储接口适配上往往存在不同约束。围绕这些差异进一步验证和移植本文方案，不仅能够检验系统设计中哪些部分具有较强的平台无关性，也有助于探索等价或近似等价的工程实现路径，从而提升方案在不同计算平台上的实际部署价值。

## 参考文献

- [1] 宋凌云, 马卓源, 李战怀, 等. 面向金融风险预测的时序图神经网络综述 [J]. 软件学报, 2024, 35(8):3897-3922.
- [2] Aggarwal C C. Outlier analysis[M]. 2nd ed. Springer, 2017.
- [3] 裴威, 李战怀, 潘巍. GPU 数据库核心技术综述 [J]. 软件学报, 2021, 32(3):859-885.
- [4] 张延松, 刘专, 韩瑞琛, 等. GPU 数据库 OLAP 优化技术研究 [J]. 软件学报, 2023, 34(11): 5205-5229.
- [5] 宋子文, 王斌, 张喜瑞, 等. 向量数据库中近似最近邻搜索关键技术综述 [J]. 软件学报, 2026, 37(3):971-1005.
- [6] Zhao Y, Chen G, Jia Z. Tod: Gpu-accelerated outlier detection via tensor operations[J]. Proceedings of the VLDB Endowment, 2022.
- [7] Xiao Y, Sun M, Song Z, et al. Breaking the storage-compute bottleneck in billion-scale anns: A gpu-driven asynchronous i/o framework[Z]. 2025.
- [8] 李晨, 刘畅, 葛一漩, 等. 多 GPU 系统非一致存储访问优化: 研究进展与展望 [J]. 电子学报, 2024, 52(5):1783-1800.
- [9] Paul J, Lu S, He B. Database systems on gpus[J/OL]. Foundations and Trends in Databases, 2021, 11(1):1-108. DOI: 10.1561/19000000076.
- [10] RAPIDS Developers. cudf documentation: How it works / cudf.pandas[EB/OL]. 2026. [https://docs.rapids.ai/api/cudf/stable/cudf\\_pandas/how-it-works/](https://docs.rapids.ai/api/cudf/stable/cudf_pandas/how-it-works/).
- [11] RAPIDS Developers. Rmm documentation: Pinnedhostmemoryresource[EB/OL]. 2026. <https://docs.rapids.ai/api/rmm/stable/>.
- [12] Johnson J, Douze M, Jégou H. Billion-scale similarity search with gpus[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2019.
- [13] FAISS Contributors. Faiss on the gpu[EB/OL]. 2024. <https://github.com/facebookresearch/faiss/wiki/Faiss-on-the-GPU>.
- [14] NVIDIA. Nccl documentation: Cuda stream semantics[EB/OL]. 2024. <https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/streams.html>.
- [15] Chandola V, Banerjee A, Kumar V. Anomaly detection: A survey[J/OL]. ACM Computing Surveys, 2009, 41(3):15:1-15:58. DOI: 10.1145/1541880.1541882.
- [16] Markou M, Singh S. Novelty detection: A review—part 1: Statistical approaches[J/OL]. Signal Processing, 2003, 83(12):2481-2497. DOI: 10.1016/j.sigpro.2003.07.018.
- [17] Phua C, Lee V, Smith-Miles K, et al. A comprehensive survey of data mining-based fraud detection research[J/OL]. arXiv preprint arXiv:1009.6119, 2010. DOI: 10.48550/arXiv.1009.6119.

- [18] Akoglu L, Tong H, Koutra D. Graph based anomaly detection and description: A survey[J/OL]. Data Mining and Knowledge Discovery, 2015, 29(3):626-688. DOI: 10.1007/s10618-014-0365-y.
- [19] 陈波冯, 李靖东, 卢兴见, 等. 基于深度学习的图异常检测技术综述 [J]. 计算机研究与发展, 2021, 58(7):1436-1455.
- [20] Angiulli F, Pizzuti C. Fast outlier detection in high dimensional spaces[C]//Proceedings of the 6th European Conference on Principles of Data Mining and Knowledge Discovery (PKDD). 2002.
- [21] 李康和, 黄震华. 基于噪声过滤与特征增强的图神经网络欺诈检测方法 [J]. 电子学报, 2023, 51(11):3053-3060.
- [22] 于浩淼, 刘炜, 孟流畅, 等. RWK-GNN: 基于特征增强与子核分解的非平衡图欺诈检测算法 [J]. 电子学报, 2024, 52(10):3382-3391.
- [23] Breunig M M, Kriegel H P, Ng R T, et al. Lof: Identifying density-based local outliers[C]//Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data. 2000.
- [24] Angiulli F, Basta S, Lodi S, et al. Fast outlier detection using a gpu[C/OL]//2013 International Conference on High Performance Computing & Simulation (HPCS). 2013:143-150. DOI: 10.1109/HPCSim.2013.6641405.
- [25] Azmandian F, Yilmazer A, Dy J G, et al. Harnessing the power of gpus to speed up feature selection for outlier detection[J/OL]. Journal of Computer Science and Technology, 2014, 29(3):408-422. DOI: 10.1007/s11390-014-1439-4.
- [26] Malkov Y A, Yashunin D A. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2020.
- [27] Subramanya S J, Devvrit, Kadekodi R, et al. Diskann: Fast accurate billion-point nearest neighbor search on a single node[C]//Advances in Neural Information Processing Systems 33 (NeurIPS 2019). 2019.
- [28] Blumofe R D, Leiserson C E. Scheduling multithreaded computations by work stealing[J]. Journal of the ACM, 1999.
- [29] Moritz P, Nishihara R, Wang S, et al. Ray: A distributed framework for emerging AI applications[C]//Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation. 2018.
- [30] NVIDIA. cuvs documentation: Multi-gpu nearest neighbors[EB/OL]. 2026. [https://docs.rapids.ai/api/cuvs/stable/python\\_api/neighbors\\_multi\\_gpu/](https://docs.rapids.ai/api/cuvs/stable/python_api/neighbors_multi_gpu/).
- [31] NVIDIA. Gpudirect storage documentation: Overview guide[EB/OL]. 2024. <https://docs.nvidia.com/gpudirect-storage/overview-guide/>.
- [32] NVIDIA. Gpudirect storage documentation: Design guide[EB/OL]. 2024. <https://docs.nvidia.com/gpudirect-storage/design-guide/>.
- [33] Liu Z, Xiao Z, Li L, et al. Dgraph: A large-scale financial dataset for graph anomaly detection[EB/OL]. 2022. <https://arxiv.org/abs/2207.03579>.

- 
- [34] IBM Research. Ibm aml-data: Synthetic data for anti-money laundering research[EB/OL]. 2022. <https://github.com/IBM/AML-Data>.
  - [35] IBM Research. Amlsim: A synthetic financial transaction generator for anti-money laundering research[EB/OL]. 2021. <https://github.com/IBM/AMLSim>.
  - [36] IEEE-CIS. Fraud detection (ieee-cis) dataset[EB/OL]. 2019. <https://www.kaggle.com/c/ieee-fraud-detection>.
  - [37] Dal Pozzolo A, Caelen O, Johnson R A, et al. Calibrating probability with undersampling for unbalanced classification[C/OL]//2015 IEEE Symposium Series on Computational Intelligence (SSCI). 2015. DOI: 10.1109/SSCI.2015.33.
  - [38] Machine Learning Group – ULB, Kaggle. Credit card fraud detection (kaggle dataset)[EB/OL]. 2013. <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>.
  - [39] DGraph Team. Dgraphfin dataset (official website)[EB/OL]. 2022. <https://dgraph.xinye.com/dataset>.
  - [40] DGraph Team. Dgraphfin\_baseline (github repository)[EB/OL]. 2022. [https://github.com/DGraphXinye/DGraphFin\\_baseline](https://github.com/DGraphXinye/DGraphFin_baseline).
  - [41] NVIDIA. cuvs documentation: Multi-gpu ivf-pq / ivf-flat (host memory requirement)[EB/OL]. 2026. <https://docs.rapids.ai/api/cuvs/stable/>.
  - [42] NVIDIA. cuvs documentation: Distribution mode / search mode / merge mode (multi-gpu ann)[EB/OL]. 2026. <https://docs.rapids.ai/api/cuvs/stable/>.
  - [43] Crankshaw D, Wang X, Zhou G, et al. Clipper: A low-latency online prediction serving system[C]//Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation. 2017.
  - [44] Gu J, Lee Y, Zhang C, et al. Clockwork: Predictable and efficient deep learning serving for cloud[C]//Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation. 2020.
  - [45] NVIDIA. cuvs documentation: Dynamic batching[EB/OL]. 2026. <https://docs.rapids.ai/api/cuvs/stable/>.

## 附录 A 实验脚本与日志归档说明

本附录结合现有实验脚本与历史运行记录，对论文中各验证阶段所对应的脚本入口、日志归档方式和参数组织口径作统一说明。目的不是列出全部内部文件，而是将正文中需要复核的单卡、多 GPU、真实集有效性和论文出图阶段，整理为可复用的一套归档规范。

### A.1 实验阶段与材料对应关系

| 论文验证阶段           | 代表性脚本或日志前缀                                                      | 主要用途                                                   |
|------------------|-----------------------------------------------------------------|--------------------------------------------------------|
| 真实集路径检查与冒烟验证     | freeze_last_tune_<ts>/01_*                                      | 对真实集输入路径、基础运行链路和候选召回—评分接口进行快速联调，对应正文中的真实集有效性验证起点。      |
| 真实集校准与配置筛选       | freeze_last_tune_<ts>/02_*                                      | 保存校准轮次的标准输出、排序摘要和配置对比结果，用于冻结论文定稿前的候选规模与评分配置。           |
| 单卡链路对比与阶段耗时分析    | bench_flash_<scale>_<scene>_gpu<id>.log、compare_<topic>_<ts>.md | 对应第四章单卡路径优化实验，重点记录原始链路、召回加速、GPU 主导链路和完整优化方案之间的耗时与吞吐差异。 |
| 多 GPU 扩展、归并与冲突对比 | bench_flash_<scale>_replay_*、*_shard_*、gpu_to_pic_<gpu>id>.csv  | 对应第五章索引复制主方案、设备侧归并兜底路径、主机侧聚合对比基线以及共享路径冲突实验。            |
| 论文图表整理与结果汇总      | latest_<topic>_paper/、chart_ready_*.csv、session_metadata.txt    | 将多轮实验结果整理为论文图表输入数据，保留轮次汇总、出图口径与会话元信息。                  |

### A.2 核心实验日志与复现材料归档

表 A-1 给出正文核心实验与归档材料之间的对应关系。该表只说明复现材料的组织口径，不引入新的实验数值；若后续补充重复运行统计，应在相同目录结构下继续追加原始日志、汇总 CSV 和绘图输入文件。



表 A-1 核心实验日志与复现材料归档说明

| 实验类别     | 代表性日志或数据前缀                                                               | 对应正文位置                  | 归档说明                                                                   |
|----------|--------------------------------------------------------------------------|-------------------------|------------------------------------------------------------------------|
| 有效性验证    | freeze_last_tune_<ts>/、<br>baseline_effect_*.csv                         | 第三章基础实验、第六章<br>基线效果对比   | 保留真实或半真实数据、1M 桥接规模下的候选召回、候选评分和绘图输入。                                    |
| 单卡递进式消融  | bench_flash_<scale>_<scene>_gpu<id>.log、<br>single_gpu_progressive_*.csv | 第四章单卡实验、第六章<br>综合对比     | 保留原始分段链路、候选召回加速、GPU 主导链路和完整优化方案的运行记录。                                  |
| 多 GPU 扩展 | bench_flash_<scale>_replica_*.csv、<br>multigpu_scaling_*.csv             | 第五章多 GPU 扩展、第六章<br>扩展趋势 | 保留 replicated、GPU merge、CPU aggregation 和 shared-path conflict 相关运行记录。 |
| 绘图与结果汇总  | plot_assets/、figures/exp/、<br>data/*.csv                                 | 第四章至第六章结果图              | 保留论文图表使用的数据源、出图脚本和导出 PDF/SVG。                                          |

### A.3 统一文件命名风格

- 会话目录统一采用“任务名 + 时间戳”形式，例如 freeze\_last\_tune\_<YYYYmmdd\_HHMMSS>，用于标识一轮冻结前复核或专题实验。
- 单次运行日志统一采用 bench\_<route>\_<scale>\_<scene>\_gpu<id>.log，其中 route 表示单卡或多卡执行路径，scale 表示数据规模，scene 表示验证场景或配置标签。
- 对比结果统一采用 compare\_<topic>\_<timestamp>.md 与 compare\_<topic>\_<timestamp>.csv 成对保存，其中 Markdown 文件用于保留人工可读结论，CSV 文件用于后续绘图与表格汇总。
- 过程检查类文件统一采用 0N\_<step>.prompt.txt、0N\_<step>.stdout.log 和 checklist.log 的组合形式，用于保留命令入口、标准输出和阶段性检查记录。
- 出图目录统一保留 all\_rounds.csv、chart\_ready\_\*.csv 和 session\_meta.txt 三类文件，分别对应原始轮次结果、绘图输入和会话说明。

### A.4 统一参数设置风格

- 数据与划分参数统一使用 real-root、real-datasets、synthetic-root、synthetic-scales、input-npz、out-dir、seed 等名称，明确区分真实集入口、合成集规模和数据切分位置。
- 单卡热路径参数统一使用 device、batch-size、score-chunk-size、warmup、repeats、max-run-seconds，分别控制设备绑定、批大小、评分块大小、预热次数和单轮运行上限。

- 候选召回与索引参数统一使用 `flash-search-k`、`flash-rerank-exact`、`flash-indices-only`、`faiss-index`、`faiss-nlist`、`faiss-nprobe`、`faiss-pq-m`、`faiss-pq-bits` 等名称，避免在不同脚本中混用语义相近的别名。
- 多 GPU 相关参数统一使用 `gpu-list`、`flash-multi-gpu-route`、`flash-service-topology`、`flash-service-protocol`、`flash-multi-gpu-devices`、`flash-multi-gpu-gather-device`、`flash-shards`、`flash-service-batch-size`，明确区分索引复制主路径、分片兜底路径和归并位置。
- 监控与分析参数统一使用 `gpu-log-path`、`gpu-log-interval-ms`、`chain-profile`、`trace-output-dir` 等名称；Shell 包装脚本中对应使用全大写环境变量，如 `BATCH_SIZE`、`FLASH_SEARCH_K`，冻结配置则统一使用 `FREEZE_*` 前缀。

## A.5 RAPIDS 相关组件使用位置说明

RAPIDS 相关组件在本文中作为 GPU 数据处理与内存资源管理支撑，不单独作为异常检测算法或候选召回算法，也不将整体性能收益单独归因于 RAPIDS。表 A-2 对相关组件在本文系统中的使用位置作统一说明。

表 A-2 RAPIDS 相关组件使用位置说明

| 组件   | 使用位置           | 对应脚本/阶段口径                              | 作用                                                  | 是否单独归因性能 |
|------|----------------|----------------------------------------|-----------------------------------------------------|----------|
| cuDF | 数据预处理与候选前后表格组织 | 预处理脚本 / <code>query_prep</code> 阶段     | GPU <code>DataFrame</code> 形式的字段筛选、类型转换、窗口统计和批量结果组织 | 否        |
| CuPy | 候选数组与中间张量表达    | 候选组织 / <code>pack_candidates</code> 阶段 | 用于设备侧数组计算、候选 ID/距离缓冲和张量接口衔接                         | 否        |
| RMM  | 内存池与页锁定主机内存资源  | 资源初始化 / 运行时配置阶段                        | 管理临时内存、pinned host 资源和内存分配抖动                        | 否        |
| cuML | 环境组件与后续可扩展接口   | 软件环境快照 / 非核心热路径                        | 作为 RAPIDS 生态组成记录，本文不将其作为核心评分或召回路径依赖                 | 否        |

## 附录 B 图表补充说明

本附录对正文图表中反复出现的变量命名、单位和比较口径作统一补充，便于阅读时快速核对不同章节之间的含义是否一致。

### B.1 图表口径说明

- 图中的 QPS、平均时延和 p99 时延均指对应实验场景下的端到端统计结果；若未特别说明，时延单位统一为 ms。
- 单卡结果图主要用于比较数据路径优化前后的链路差异，多 GPU 结果图主要用于比较索引复制主方案、设备侧归并兜底路径以及共享路径冲突条件下的扩展差异。
- 真实集结果用于有效性验证，1M 及以上合成数据结果用于系统性能与扩展分析，二者不直接比较绝对吞吐数值。
- 正文结果图中的方案名称均对应具体验证场景或系统路径，不再使用含义不明确的编号标签。

### B.2 特征构造字段与派生特征补充说明

表 B-1 特征构造明细

| 原始字段/信息源  | 派生特征                                                | 特征类型       | 预处理方式               | 在系统中的作用                                                      |
|-----------|-----------------------------------------------------|------------|---------------------|--------------------------------------------------------------|
| 交易金额      | 标准化金额、对数金额、<br>时间窗金额均值/波动统计                         | 连续型        | 缺失填补、异常值截断、标准化/归一化  | 刻画交易规模及金额异常波动                                                |
| 交易时间戳     | 小时段、星期信息、夜间交易标记、平均交易间隔                              | 时间型/统计型    | 时间规范化、周期化表达、滑动时间窗统计 | 捕捉异常时段行为与时序突变                                                |
| 账户标识及账户历史 | 账户时间窗交易次数、平均金额、活跃度统计                                | 类别索引 + 统计型 | 安全编码、滑动窗口聚合         | 表征账户级长期与短期行为模式                                               |
| 设备标识      | 设备使用频次、设备共享度、账户—设备切换统计                              | 类别型/统计型    | 缺失标识、频次编码、窗口统计      | 识别设备复用、盗刷与团伙风险                                               |
| 商户标识/商户类别 | 商户类别编码、商户频次、商户局部风险统计                                | 类别型/统计型    | one-hot、频次编码或安全统计编码 | 增强交易场景相似性与商户异常识别能力                                           |
| 地理位置字段    | 地域编码、跨地域切换次数、地理跳变标记                                 | 类别型/统计型    | 地域粒度统一、编码、窗口切换统计    | 捕捉异地交易、短时跨区域异常                                               |
| 支付渠道/终端类型 | 渠道编码、终端类别向量、渠道频次统计                                  | 类别型        | one-hot 或频次编码       | 丰富候选召回的相似度基础，提升异常区分度                                         |
| 图节点原始属性   | 节点属性规范化向量                                           | 混合型        | 数值归一化、类别编码          | 形成图节点输入的基础表示                                                 |
| 图边关系与邻域信息 | 入边/出边计数、邻域金额统计、方向比例、时间分布                            | 关系统计型      | 邻域聚合、时间窗统计、向量拼接     | 将图结构信息映射为可检索、可评分的节点向量                                        |
| 融合后统一表示   | 固定维度最终向量 ( $v_i \in \mathbb{R}^d$ , $d$ 由最终预处理脚本确定) | 向量型        | 维度检查、类型检查、异常值截断     | 完整路径作为 FlashANNs 候选召回输入，前期可用于 FAISS-GPU 接口验证，并直接供 PyTOD 评分使用 |

### B.3 多 GPU 实验配置补充表

表 B-2 多 GPU 实验配置补充表

(a) 固定参数与实验口径

| GPU 数 | 方案                   | 数据规模 | 批大小  | 候选数 | $k$ | 实验目的            |
|-------|----------------------|------|------|-----|-----|-----------------|
| 1     | Replicated           | 10M  | 4096 | 128 | 50  | 单卡吞吐扩展基线        |
| 2     | 索引复制                 | 10M  | 4096 | 128 | 50  | 2 GPU 索引复制扩展基线  |
| 4     | 索引复制                 | 10M  | 4096 | 128 | 50  | 四卡索引复制扩展基线      |
| 4     | 本地评分 + 设备侧归并         | 100M | 3072 | 128 | 50  | 显存受限时的设备侧归并兜底方案 |
| 4     | 本地评分 + 主机侧聚合         | 100M | 3072 | 128 | 50  | 主机侧聚合对比基线       |
| 2     | 拓扑感知绑定               | 100M | 4096 | 128 | 50  | I/O 通道绑定对比实验    |
| 4     | 拓扑感知绑定               | 100M | 4096 | 128 | 50  | I/O 通道绑定对比实验    |
| 2     | Shared-path conflict | 100M | 4096 | 128 | 50  | 共享路径冲突对比基线      |
| 4     | Shared-path conflict | 100M | 4096 | 128 | 50  | 共享路径冲突对比基线      |

(b) 方案配置说明

| 方案                   | 索引组织         | 归并路径                    | I/O 绑定               | 定位                      |
|----------------------|--------------|-------------------------|----------------------|-------------------------|
| Replicated           | Full replica | None                    | Topology-aware       | 主扩展方案，验证随 GPU 数量增加的吞吐收益 |
| 本地评分 + 设备侧归并         | 分片索引         | NCCL all-gather + 设备侧归并 | 拓扑感知                 | 显存受限时的设备侧归并兜底路径         |
| 本地评分 + 主机侧聚合         | 分片索引         | 主机侧聚合                   | 拓扑感知                 | 用于对比 CPU 回到数据平面的额外代价    |
| 拓扑感知绑定               | 完整副本         | 无                       | 拓扑感知                 | 验证 I/O 通道绑定对扩展效率的影响     |
| Shared-path conflict | Full replica | None                    | Shared PCIe/SSD path | 作为共享路径冲突下的对比基线          |

注：多 GPU 实验规模口径统一按“真实集 → 1M 合成数据 → 10M 合成数据 → 50M 合成数据 → 100M 合成数据”组织。10M 主要服务于索引复制主方案扩展曲线，100M 主要服务于兜底归并与路径冲突对比。

## 附录 C 缩写、变量与配置对照

本附录列出正文中频繁出现的关键变量、配置项及其含义，便于后续统一实验脚本和结果复核。

| 变量或配置项             | 含义                                                                              |
|--------------------|---------------------------------------------------------------------------------|
| FAISS-GPU          | FAISS 的 GPU 检索实现；本文中主要用于前期 ANN 候选召回接口、设备侧输入输出路径、候选 ID/距离返回格式以及 PyTOD 评分衔接的初步验证。 |
| FlashANNS          | 完整 FlashTOD 路径中的主要 ANN 候选召回后端，用于大规模外存驻留或超显存索引条件下的高吞吐候选召回。                       |
| 初步融合方案             | FAISS-GPU ANN API 与 PyTOD 的简单串接，用于验证候选召回—候选评分链路可行性。                             |
| 完整融合方案             | 以 FlashANNS 为候选召回后端、以 PyTOD 为异常评分主干，并结合 GPU 主导数据路径和设备侧候选组织后的完整 FlashTOD 实现。     |
| batch_size         | 单次提交到候选召回或精排路径中的批处理大小。                                                          |
| rerank_budget      | 进入高精度精排阶段的候选数量预算。                                                               |
| nlist              | 索引聚类桶数量或粗排分区规模参数。                                                               |
| nprobe             | 查询阶段实际访问的粗排桶数量。                                                                 |
| device_id          | 当前任务绑定的 GPU 设备标识。                                                               |
| replica_mode       | 多 GPU 复制式多实例执行模式开关。                                                             |
| bounded_batch_size | 在时延约束下允许形成的微批上限。                                                                |

## 在学期间完成的相关学术成果



## 致 谢